DON HONORIO VENTURA STATE UNIVERSITY
Bacolor, Pampanga

College of Engineering and Architecture
Department of Electronics Engineering

INVENTORY MONITORING SYSTEM WITH RFID ACCESS AND QR CODE FOR ELECTRONICS ENGINEERING DEPARTMENT

Blancada, Jerome S.

Cayanan, Josh Bradly M.

Cruz, Armel John V.

Dela Cruz, Enriet B.

Dela Cruz, Rainner C.

Gatchalian, Julian Miguel G.

April 2023

ENMAR TUAZON, ECE

INVENTORY MONITORING SYSTEM WITH RFID ACCESS AND QR CODE FOR ELECTRONICS ENGINEERING DEPARTMENT

A Thesis
Presented to the
Department of Electronics Engineering
College of Engineering and Architecture
Don Honorio Ventura Technological State University

In Partial Fulfilment
Of the Course Requirements for the Degree of
Bachelor of Science in Electronics Engineering

Blancada, Jerome S.
Cayanan, Josh Bradly M.
Cruz, Armel John V.
Dela Cruz, Enriet B.
Dela Cruz, Rainner C.
Gatchalian, Julian Miguel G.

April 2023

APPROVAL SHEET

This thesis entitled **"INVENTORY MONITORING SYSTEM WITH RFID ACCESS AND QR CODE FOR ELECTRONICS ENGINEERING DEPARTMENT",** prepared and submitted by **Enriet B. Dela Cruz, Jerome S. Blancada, Josh Bradly M. Cayanan, Armel John V. Cruz, Rainner C. Dela Cruz, and Julian Miguel G. Gatchalian**, in partial fulfillment of the requirements for the degree **Bachelor of Science in Electronics Engineering**, has been examined and recommended for Oral Examination.

ENMAR TUAZON, ECE
Adviser

APPROVAL

Accepted and approved by the Panel of Examiners with a grade of **Passed** on **May 30, 2024.**

Engr. Elias Francis Vergara
Panel Chair

Engr. Anthony Tolentino
Panel Member

Engr. Nilo Manuntag
Panel Member

Accepted in partial fulfillment of the requirements for the degree Bachelor of Science in Electronics Engineering**.**

Enmar T. Tuazon, ECE
EcE Thesis Coordinator

Nilo Q. Manuntag, PECE
OIC-Chairperson, EcE Department

JUN P. FLORES, PECE MEP-EE
Dean, College of Engineering and Architecture

ACKNOWLEDGEMENT

First and foremost, the researchers would like to express their gratitude to the **Lord Almighty** who gave them wisdom, knowledge, and understanding to finish this research study.

The researchers would also like to thank the following people who provided assistance all throughout the journey of this study:

To their research and class adviser, **Engr. Enmar Tuazon**, who continuously supported, and guided them until the end of this research study. The one who shows encouragement, patience, and enthusiasm in all the time of conducting this study. Her expertise really helped a lot and working under her supervision is a great honor;

To their Panel of Examiners, **Engr. Elias Francis Vergara, Engr. Anthony Tolentino**, **and Engr. Nilo Manuntag**, who gave insightful comments and suggestions, and motivation to face the challenges that made this research study better;

To their laboratory technician, **June Paul Velarde**, who shared his knowledge and expertise that made this study more reliable;

To **their parents**, who served as their inspiration and has always been there to support, love, care, and be proud of their children's milestones;

And to people who were not mentioned but still have been a great help, the researchers would like to express their deep gratitude for all of them.

The researchers would like to express their sincerest gratitude to following who were mentioned above, as the study would not be possible without them.

CERTIFICATE OF ORIGINALITY

This is to certify that the research work presented in this thesis entitled "**INVENTORY MONITORING SYSTEM WITH RFID ACCESS AND QR CODE FOR ELECTRONICS ENGINEERING DEPARTMENT"** for the award of Bachelor of Science in Electronics Engineering of Don Honorio Ventura State University from Bacolor, Pampanga, embodies the result of original and scholarly work carried out by the undersigned. This thesis does not contain words or ideas taken from published sources or written works by other persons which have been accepted as basis for the award of any degree from other education institutions, except where proper referencing and acknowledgment were made.

Blancada, Jerome S.

Cayanan, Josh Bradly M.

Cruz, Armel John V.

Dela Cruz, Enriet B.

Dela Cruz, Rainner C.

Gatchalian, Julian Miguel G.

Researchers

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF APPENDICES

Abstract

This research paper presents the Inventory Monitoring System with RFID Access and QR Code for Electronics Engineering Department, a digital solution designed to modernize equipment borrowing and returning by replacing the existing manual paper-based system. The system utilizes RFID access and QR codes to streamline its functions. The hardware design incorporates an ESP32 chip acting as a Wi-Fi module for the system, an SD card for storage of generated data, and an MFRC522 RFID reader for tag identification and users' authentication. The researchers also developed an Android application that integrates functionalities for equipment borrowing, returning, and real-time reporting with timestamps. The application was incorporated with QR scanner to facilitate the borrowing and returning processes of laboratory equipment. This mobile-based offline application also enables accurate monitoring of transactions made within the system. This system empowers faculties and laboratory technicians to efficiently track and manage all borrowing and returning transactions for laboratory equipment. To ensure its effectiveness and performance, the system was tested and evaluated across 51 trials. It is tested and evaluated for its accuracy, timeliness and performance, ensuring its effectiveness and reliability in monitoring equipment in the Electronics Engineering Department. The system addresses current shortcomings of the manual system by providing real-time equipment status updates, eliminating manual record-keeping and centralizing borrowing and returning activities in a single Android application. Ultimately, its borrowing processes is 4x faster, and its returning processes is 9x faster than the currently applied method.

Keywords: Inventory Monitoring System, RFID, QR code, ESP32, SD card, and Android Application

CHAPTER 1

INTRODUCTION

This chapter introduces the Background of the Study, Statement of the Problem, Project Objectives, Scope and Limitations, and Significance of the Study. The researchers also outline the research Conceptual Framework to provide an overview of the succeeding chapters.

Background of the Study

Ensuring efficient asset management is crucial for institutions. This includes keeping track of all the long-lasting equipment and supplies. Many organizations, like construction companies, still rely on manual methods for inventory control. These methods could be faster and more labor-intensive. By adopting an automated system, institutions can significantly reduce the time spent checking and tracking assets, leading to a more streamlined and efficient operation (Xie 2015). Globally, there's a growing trend towards automation and digitalization across various sectors, including laboratory management. This shift offers significant benefits such as improved efficiency, enhanced data accuracy, and real-time information access.

Recent advancements in technology offer promising solutions for improved inventory management. Nowadays, the inventory management system has become easier and more systematic with RFID technology. Radio Frequency Identification (RFID) technology offers a promising solution to these inefficiencies. RFID technology utilizes tags on equipment that can be scanned by readers, enabling automatic identification and data capture, leading to faster and more accurate inventory tracking ensuring they are always in the right place. RFID is an evolving mechanism that is being implemented in almost every field. It is a contactless communication mechanism (Kumbhakarna & Chaudhari, 2017). This eliminates discrepancies and human errors that plague manual systems. As a result, implementing

RFID can significantly improve inventory management efficiency, enhance the accuracy and quality of asset tracking, and minimize human error.

QR codes simplify equipment management by offering a convenient way to look up information, track equipment locations, and maintain inventory records. Their versatility in storing various data types allows for efficient recording of details about different equipment and even personalized information. This translates to significant time and labor savings in managing equipment, similar to the benefits seen in operating room instrument management (Li-Fang et al. 2021). QR codes offer a cost-effective alternative for basic identification. They can be printed on equipment or packaging and scanned with smartphones or dedicated readers, linking to detailed inventory information within a database.

In this paper, the researchers proposed a laboratory monitoring system specifically designed for the DHVSU Electronics Engineering department, utilizing both RFID access and QR code functionalities that will significantly benefit the department and its constituents. This proposed system aimed to manage and control access to laboratory equipment. Functioning as an app-based monitoring and management system, it stores detailed inventory data and facilitates the physical distribution of equipment. Laboratory technicians will have access to the history view of borrowed equipment status through the Android application, including when it was borrowed, its purpose, the subject, the borrower and the instructor that handles the student within that time. This system can digitize the current borrowing system of the institution. It will improve data accuracy by minimizing human error, enhancing efficiency through faster check-in/out processes, reduce loss with real-time tracking, and increase inventory visibility for informed decision-making regarding equipment allocation.

Conceptual Framework

The basis of the study is shown in the framework below.

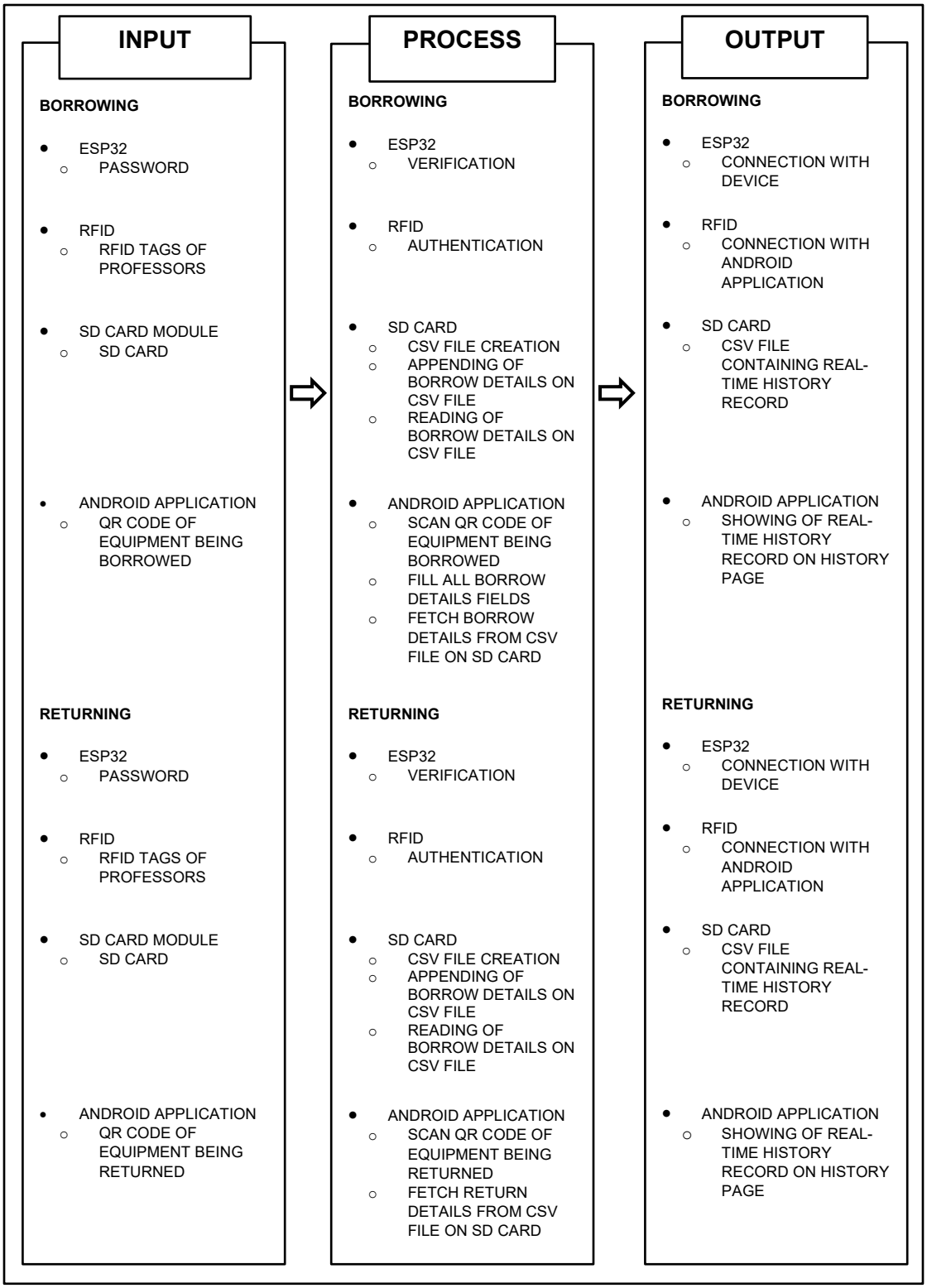


Figure 1: Paradigm of the Study

This thesis employed the Input, Process, and Output (IPO) model as shown in Figure 1. The input box shows the materials and triggers needed to start the function of the system both when borrowind and returning a laboratory equipment. The process box contains the processes that the hardware components and the Android application performs in order to work hand-in-hand with each other. After processing all inputs, the output box presents the connection of the hardware with the Android application, and the generated data of the system, which can be accessed both in the SD card and in the Android application.

Statement of the Problem

The current approach to borrowing laboratory equipment in the Electronics Engineering Department (ECE) of Don Honorio Ventura State University (DHVSU) causes inconveniences such as the need to fill out borrowing information on traditional paper. Furthermore, all information about the equipment and borrowing history is stored in written format, resulting in environmental waste when these forms are no longer needed. Moreover, there is an additional workload for the laboratory technician who must manually monitor the laboratory equipment, as well as a lengthy process of borrowing for students and ECE professors.

This study aimed to provide a system for the ECE department that paved the way to go digital in the process of borrowing laboratory equipment and to make the process faster and easier, without sacrificing the accuracy and scalability of the current process. To ensure that the objectives are met, this study sought answers to the following:

1. The current method of verifying eligibility to use laboratory equipment at our institution relies on manual processes, which are time-consuming and prone to human errors.

2. The department currently lacks an efficient system for monitoring laboratory equipment, leading to difficulties in tracking the condition and status of the equipment.
3. The current process of monitoring and borrowing of laboratory equipment do not leverage the use of technology making it inaccessible for remote monitoring.
4. The current inventory system lacks real-time reporting capabilities for all borrowing activities, which could potentially result in discrepancies and hinder transparency and efficiency.

Project Objectives

General Objective

To develop an inventory monitoring system using RFID access and QR code for the Electronics Engineering Department.

Specific Objectives

Specifically, the study seeks to address the stated problem by achieving the following objectives.

1. To utilize RFID tags for verification of eligibility to use laboratory equipment.
2. To monitor laboratory equipment in the Electronics Engineering Department using QR codes.
3. To develop an Android application for remote monitoring of the inventory system.
4. To generate real-time and time-stamped reports of all the activities within the inventory system.

Scope and Limitations

Scope

The study prioritized the development of Inventory Monitoring System with RFID Access and QR Codes for the Electronics Engineering Department of this university. The main goal of this study is to ensure the usability, accuracy, timeliness, and performance of the system in generating reliable data that can be accessed for monitoring purposes. An ESP32 board is used to serve as the access point in which the devices connect to be able to access the Android application's features. Moreover, RFID tags were used as identity tag for faculties who wish to access the system. There are a total of 10 tags that were part of this study, wherein each one is designated to a respective faculty member. Additionally, QR codes were optimized for efficient equipment identification. Furtheremore, an Android application is developed for the digital processes of the system of borrowing and returning laboratory equipment and for remote monitoring of the history records. Ultimately, an SD card with a size of 16 GB is used as a storage for the generated real-time reports. Only micro-SD cards are allowed to be used in the system.

Limitations

The researchers established limitations to ensure that the research project would be manageable. The study is limited to the Analog Laboratory only of the Electronics Engineering Department of DHVSU. In addition, not all laboratory equipment placed in the Analog Laboratory were part of the study. The researchers only chose specific laboratory equipment which includes digital and analog trainers, function generators, Lenovo laptops, digital oscilloscope, digital multitester, MCU trainers, and power supplies only, with a total of 51 laboratory equipment. However, adding new laboratory equipment for monitoring is still feasible by generating QR codes using Microsoft PowerPoint. Having said that, generating QR codes is not a part of the system of this thesis. Moreover, borrowers can only

borrow one (1) equipment per transaction. To borrow multiple equipment, the borrower needs to repeatedly scan QR codes and fill-up the needed details.

The ESP32 device which is used as access point can connect a maximum of 4 devices only. Registration of RFID tags is hard-coded. Thus, registration when adding new RFID tags needs to be written in the Arduino code as well. Unlike other inventory systems, there is no Masterlist for all the QR codes in this thesis. The information of a laboratory equipment is embedded on the QR code itself. Therefore, updating of equipment information is manually done using Microsoft PowerPoint or other software that can generate QR. The QR feature also have the ability to read all QR codes. Hence, as long as the needed information are embedded on the QR code, the QR scanner regards it as a valid QR and actually shows up the details page where the user can input details. The needed information are Inventory Code, Equipment Name, and Remarks. Mobile phones that utilize the Android 10 operating system or higher are the only allowable devices to use the application. Additionaly, other operating systems such as Apple IOS are not compatible. The study was not also implemented directly and in person in the Analog Laboratoy using real equipment due to the limitations of online class. The testing of the system relied entirely on the utilization of dummy equipment affixed with printed QR codes.

Significance of the Study

The inventory monitoring system can generate real-time reports of the activities made in the system. It can also monitor laboratory equipment to manage them in an organized manner effectively. Since all of these can be done remotely in a single Android phone using the mobile application and just by tapping the RFID tags, the ease of access to important data is viable. Thus, it is beneficial to the following:

ECE students. The need for filling up information on a piece of paper when borrowing equipment to use in laboratory activities is removed due to the implementation of the project.

ECE Professors. The use of laboratory equipment provides practical application and hands-on experience for students and enhances the quality of teaching of the professors. This approach also allows the professors to enrich the students' learning experience with a realistic substantial foundation instead of purely theoretical concepts. With the help of the inventory system, the borrowing and returning process of specific laboratory equipment used in professors' discussions can be swift and straightforward. Additionally, the ease of access of professors to the system can be done by just scanning the RFID tags.

Laboratory Technician. Reduced workload is one of the major advantages of the inventory system for the laboratory technician. Monitoring work is a lot easier since all is being done in the mobile application. Moreover, real-time generation of updates with time stamps helps in managing activities made in borrowing and returning laboratory equipment.

ECE Department. The encouragement for more sustainable and effective learning is being implemented through hands-on workshops and practice in using laboratory equipment. Learning would not be just made theoretically but also in practical application. The department can maintain a relaxed attitude when letting students borrow laboratory equipment.

Environment. Implementing the product is a movement and a step forward towards a digital world. Since physical papers are not already needed when filling up important details when borrowing laboratory equipment, waste products are lessened which in turn benefits the environment.

Definition of Terms

The following terms are defined operationally and conceptually to properly guide the readers of this study:

3D Case – A protective enclosure for electronic hardware, typically designed using three-dimensional modeling software and fabricated using 3D printing technology.

Access Point – A device, typically a wireless router or network node, that allows other devices to connect to a wired network or the internet wirelessly.

Annealing PLA – strengthens the PLA against and handle heat, allowing the parts to hold their shape at higher temperature.

Android Application – A software application designed to run on devices that use the Android operating system, such as smartphones and tablets.

Back-end – The underlying infrastructure or processes that support the functionality of a software application, often unseen by users.

CSV (Comma-Separated Values) – A plain-text file format used to store tabular data, where each line represents a row of the table, and values within each row are separated by commas.

Experimental Approach – A research method involving controlled tests and trials to evaluate the functionality and performance of a system or process.

Forbidden Response – An HTTP status code indicating that the server understood the request, but refuses to authorize it, often used to deny access to unauthorized users or devices.

Front-end – The user interface or visible part of a software application, where users interact with the system.

HTTP (Hypertext Transfer Protocol) – A protocol used for transmitting data over the internet, commonly used for communication between web servers and clients, or between devices in a network.

HTTP Code – A numerical status code returned by a server in response to a client's request, indicating the outcome of the request, such as success, failure, or redirection.

Inventory Monitoring System – A system designed to track and manage inventory levels, movements, and usage, typically utilizing technology such as RFID and QR codes for efficient monitoring and management.

Inventory – A detailed record or list of all the items or materials within a particular system, such as equipment in a laboratory or goods in a store.

Iterative Design – A design approach that involves repeating the design process, making improvements and refinements based on feedback and testing results, ensuring gradual enhancement of the system's features and functionality.

ML Kit – Google's Machine Learning Kit for mobile applications, providing developers with tools and APIs for integrating machine learning capabilities into Android applications.

PCB (Printed Circuit Board) – A board made of non-conductive material with conductive pathways etched or printed on it, used to mechanically support and electrically connect electronic components

QR Code (Quick Response Code) – A type of barcode that can store information, often used for storing URLs or other data for quick retrieval using a smartphone or dedicated scanner.

Real-time Reporting – Reporting method that provides information immediately as it occurs, without delay, ensuring up-to-date and accurate data.

Real-time – Referring to actions or processes that occur instantaneously or without noticeable delay, providing immediate feedback or results.

Remote Monitoring – The process of observing and managing a system or process from a distant location, often facilitated by technology like mobile apps or networked devices.

RFID (Radio Frequency Identification) – A technology that uses electromagnetic fields to automatically identify and track tags attached to objects. In this context, RFID tags are utilized for inventory management.

SD Card – A small, portable memory card used for storing data, often used in electronic devices such as cameras, smartphones, and the inventory monitoring system to store real-time reports and data.

SPI (Serial Peripheral Interface) – A synchronous serial communication interface used to transfer data between microcontrollers and peripheral devices.

SSID (Service Set Identifier) – A unique identifier assigned to a wireless network, allowing devices to connect to and communicate with the network.

Budget Allocation

Table 1: List of Materials and Pricing

Materials	Quantity	Unit Price	Amount
ESP32	1	Php 160.00	Php 160.00
RFID Reader	1	Php 76.00	Php 76.00
RFID Tags	10	Php 15.00	Php 150.00
SD Card Module	1	Php 64.00	Php 64.00
16 GB SDHC Card	1	Php 170.00	Php 170.00
Charger Adaptor	1	Php 30.00	Php 30.00
Micro-USB Cord	1	Php 35.00	Php 35.00
PCB and Etching	1	Php 50.00	Php 50.00
3D Printing	1	Php 150.00	Php 150.00
		Total	**Php 885.00**

CHAPTER 2

LITERATURE REVIEW

This chapter delves into a comprehensive review of existing literature and studies relevant to the research topic. By meticulously analyzing these sources, it will synthesize key findings, identify areas where further investigation is needed, and ultimately, provide a strong foundation for the upcoming research endeavor.

STATE OF THE ART

Inventory is the raw materials, components, and finished goods a company sells or uses in production. Within a commercial entity, inventory is a physical manifestation of its product offerings, functioning as a readily available pool of resources. This includes not just finished goods, but the building blocks of their creation, a dynamic reserve of electrical and hardware components. Each distinct variation be it type, dimension, color, or configuration represents a unique item within this strategic stockpile.

Erlangga et al. (2022) outline two primary modes of inventory monitoring: direct and indirect. Direct methods involve real-time observation of incoming and outgoing goods, akin to a meticulous census. Conversely, indirect monitoring relies on secondary sources such as written reports, verbal briefings, or interviews with personnel involved in the inventory workflow. The advantage of implementing an inventory monitoring system lies in its ability to generate immediate insight into stock availability. This empowers businesses to make agile decisions and streamline the report-generation process. Companies should prioritize investments in cutting-edge technologies that facilitate real-time data visualization to maximize the system's potential.

Efficient and modern inventory management systems are becoming increasingly essential not just for large corporations, but also for academic institutions like universities.

According to Procedures (2019), traditional methods of ensuring security and safety in laboratory settings have relied on manual equipment tracking, involving tedious processes of pen-and-paper documentation and tagging each item with an identification number. However, as highlighted by Shao & Wang (2021), the complexity of laboratory environments, including the presence of various equipment, network configurations, and maintenance needs, underscores the necessity for digital solutions. Implementing digital technologies not only enhances efficiency in management but also plays a crucial role in ensuring safety, security, and cost reduction associated with repairs.

Components

This section contains the related studies highlighting the specifications of the main components, which are utilized in the research.

A. ESP 32

The ESP32, created by Espressif Systems, is a powerful and affordable option for building devices that connect and it combines a variety of features that are very useful for IoT applications. The ESP32 utilizes a dual-core Tensilica Xtensa LX6 microprocessor that offers increased processing power, enabling efficient multitasking and handling of complex tasks. ESP32 has built-in Wi-Fi and Bluetooth interfaces that simplify connection and communication with other devices or networks. The ESP32 integrates Wi-Fi and Bluetooth for straightforward connection and communication with other devices or networks and it provides Bluetooth Classic and Bluetooth Low Energy (BLE) connectivity options. It also offers numerous general-purpose input/output (GPIO) pins for connecting and controlling external devices and sensors. These pins support various protocols such as SPI, I2C, UART, and PWM. The ESP32 features low-power operation modes that extend battery life, making it well-suited for projects with limited power or battery operation. It can be programmed using

various development environments and languages, with C++ being the most common. Popular choices for programming the ESP32 include the Arduino IDE and PlatformIO.

Magdy (2023) mentioned that the ESP32 can be configured as a wireless access point, eliminating the need for an external router. In this Access Point (AP) mode, also called SoftAP, the ESP32 itself broadcasts a Wi-Fi network. Other devices, like computers, smartphones, or even other ESP32s (stations), can then connect to this local network and communicate directly with the ESP32. This creates a self-contained network ideal for data exchange within a limited range, without internet access. Crucially, the ESP32 acts as a router in AP mode, assigning IP addresses to connected devices. However, this mode has a limitation of supporting a maximum of five clients.

As stated by Babiuch, Foltynek, & Smutny (2019), the ESP32 is a powerful upgrade to the ESP8266 chip, packing a lot of functionality onto a single minicomputer chip (SoC). It boasts built-in Wi-Fi and Bluetooth for wireless connections, along with dual cores for faster processing. Compared to its predecessor, the ESP32 has nearly double the connection points for peripherals and can control more devices. It also comes with four times more memory, allowing it to store more programs and data. This makes the ESP32 perfect for various applications like smart home control, wearable tech, internet-connected devices, and even monitoring systems thanks to its wireless capabilities.

B. SD Card

Santos and Santos (2022) discuss that the Arduino microSD card module bridges the voltage gap between standard microSD cards (3.3V) and Arduino circuits (often 5V) by incorporating a voltage regulator that steps down the voltage to a safe level for the card. This module is particularly useful for projects that require data storage. Using the SD library, your Arduino can create and write files to the microSD card, enabling long-term data logging

capabilities. The SD card module utilizes a common communication protocol (SPI) to interact with the card. It recommends formatting the micro-SD card using the official SD card formatter for optimal compatibility. The SD card module can handle micro-SD cards up to 32GB in capacity.

As mentioned by M. Broell, L., Hanshans, C., & Kimmerle, D. (2023) an SD card, short for Secure Digital Memory Card, operates based on flash storage technology, serving as a digital storage device. The majority of microcontrollers typically only accommodate the SPI bus for storing data onto an SD card. However, the ESP32, a popular choice for IoT applications, offers compatibility with both the SPI and SD buses. Allafi, I., & Iqbal, T. (2017) also indicated that the ESP32 lacks a slot for inserting an SD card; SPI pins are utilized to link an SD card reader.

C. RFID System

Radio Frequency Identification (RFID) has become a popular technology for applications like supply chain tracking, warehouse management, and inventory control. It works by attaching unique ID tags to objects, which are then detected wirelessly by RFID readers. These readers have a limited range, so large-scale systems often deploy multiple readers to work together and cover a wider area. RFID's convenience is another advantage, allowing for quick setup of temporary systems using mobile readers. However, a key challenge for RFID systems is identification throughput, or how many tags can be identified per unit of time. This is heavily impacted by signal collisions, and most existing protocols focus on resolving these collisions to improve throughput for a single reader (Liu et.al, 2016).

Nedelkovski, (2017) An RFID system has two main components, a tag, and a reader. The RFID reader comprises a radio frequency module, a control unit, and an antenna coil

that emits a high-frequency electromagnetic field. Conversely, the tag typically constitutes a passive element, consisting solely of an antenna and an electronic microchip. When the tag approaches the transceiver's electromagnetic field, voltage is induced in its antenna coil, thereby providing power to the microchip. According to Soni, S., Soni, R., & Waoo, A. A. (2021). RFID technologies are recognized for their efficiency and enhanced security compared to alternative networks. They find extensive applications across various sectors including public transportation, industrial automation, animal identification, ticketing, inventory management, electronic immobilization, access control, and asset and personnel tracking.

D. 3D Case

A review conducted by Hasan et al. (2024), this review investigates the potential of recycled PLA for 3D printing by analyzing its mechanical and thermal properties (strength and heat resistance) and its environmental impact. While recycled PLA might be slightly weaker, it offers a significant sustainability advantage by potentially reducing the carbon footprint of 3D printing. Overall, the wider use of recycled PLA presents a promising path towards more eco-friendly 3D printing.

In the study of Suder et al. (2021), they conducted tests showed that regular PLA 3D printed parts could only handle the lowest heat compared to other PLAs. Even with a small weight on them, these parts would not deform up to 75°C, but any hotter and they would start to soften and lose their shape. PLA-HD is the most heat resistant type among those tested, tolerating temperatures up to 165°C. Regular PLA comes in second at 150°C. However, all three PLAs suffer from weakness around screw connections, deforming at a much lower 60°C regardless of the PLA type used. This means that even the most heat resistant PLA might not be suitable if you plan to use screws to assemble the parts. Studies confirm that

annealing PLA boosts its heat tolerance, resulting in more stable shapes at hotter temperatures but still depends on the amount of stress applied.

Software

This section contains the related studies highlighting the programming languages that are used in the research.

A. Kotlin

A modern, general-purpose programming language called Kotlin can be compiled to run on various platforms, including the Java Virtual Machine (JVM), which makes it suitable for Android development. It also compiles to JavaScript and native code. Developed by JetBrains in 2010, Kotlin became an open-source language in 2016 with the official release of version 1.0. As a modern language, Kotlin offers several benefits for Android app development without causing compatibility issues (Putranto et al., 2020).

Kotlin, emerging in 2011, stands as a contemporary programming language offering an alternative to Java. According to the State of Developer Ecosystem report in 2018, Kotlin sees primary usage in mobile and server applications, predominantly operating on Oreo and Nougat for Android and JDK 8 for servers. Its adoption has led to a reduction in common Java frustrations, enhancing the safety of development processes. Kotlin presents several benefits to Java developers, particularly in terms of code conciseness. (Ardito, L., et. al., 2020).

Table 2: Advantages of Kotlin in Android Application Development

Feature	Kotlin	Java	PHP	Python
Best for Android	Yes	Yes (original)	No	No
Code Length	Less	More	More	More
Modern Features	More	Less	Less	Less
Integrates with Java	Yes	N/A	No	No

As the go-to development environment for Android apps, Android Studio and Kotlin form a perfect duo. Kotlin feels like a native language for the platform, seamlessly integrating with its functionalities. Compared to Java, Kotlin achieves the same results with less code. This leads to cleaner, more readable code that is easier to maintain, ultimately reducing bugs and development time. Kotlin boasts modern features like null-safety, extension functions, and lambdas, making the code more robust and elegant compared to Java. Another benefit for developers with Java experience is that Kotlin plays well with Java code. Existing Java libraries and frameworks can be seamlessly integrated into a Kotlin project, easing the transition for familiar developers. While PHP and Python are powerful languages, they are geared more towards web development or backend applications and are not ideal for building native Android apps.

Bose (2018) discusses that Kotlin stands out for its maturity and focus on improving Android development. Unlike some languages, Kotlin's development process involved extensive testing and refinement before its official release, suggesting fewer inherent issues.

Kotlin simplifies development by being concise and modern, offering features that enhance developer productivity. The compiler actively aids in reducing errors by catching potential problems early, which saves time and effort in debugging. With built-in safeguards against common coding mistakes, Kotlin inherently produces more robust and stable applications. Finally, Kotlin's conciseness, requiring less code to achieve the same results, improves code readability and maintainability for developers.

B. Google ML

According to T. Sproull and D. Shook (2023), Google ML Kit offers robust machine-learning capabilities for applications on both iOS and Android platforms, catering to developers of all skill levels, whether seasoned or new to machine learning. With minimal code, users can integrate these efficient and user-friendly machine-learning tools into their apps, and via the official Google ML Kit website ML Kit has a barcode API for scanning barcodes and QR codes. It detects them in the camera preview frame, reads the embedded information or URL, and can automatically open the link or read the information aloud using Google TTS. Accessing the smartphone's camera is all it needs.

C. C++

In recent years, the C++ programming language has undergone significant modernization and improvement. Despite the prevalence of IoT hardware capable of running higher-level languages like Python, Java, or JavaScript, C++ still offers distinct advantages. These include a minimal memory footprint at runtime, the ability for explicit memory allocation control if desired, and close proximity to hardware, facilitating low-level driver development for sensors and other peripherals (U., Schafer, 2019).

Existing Works

Multiple research endeavors have explored diverse strategies to tackle the challenges imposed by manual equipment management systems in computer laboratories.

In the study conducted by Kumbhakarna & Chaudhary, (2017) the researchers created a system utilizing RFID technology to effectively oversee and manage all inventory within a college laboratory. It assists lab assistants in maintaining detailed records and overseeing inventory, while also enabling students to easily locate and verify the availability of equipment and other items. Inventory data is stored on a PC and accessible via a graphical user interface (GUI), facilitating tasks such as adding, searching, issuing, returning, and reviewing the history of specific items or student interactions. It uses RFID technology as its foundation, this solution is cost-effective and scalable, featuring both software and hardware components. Multiple antennas can be connected to a single RFID reader, expanding coverage for comprehensive inventory tracking.

Muyumba & Phiri (2017) compared the literature on various inventory management technologies such as RFID, Barcode, and NFC, and examined existing systems for potential adoption of automated inventory management for the Zambia Air Force (ZAF). A baseline study revealed challenges with the current manual system, including human errors and pilferage. They developed and tested a prototype system, finding it faster, more efficient, and more reliable. However, their focus is on computerizing inventory management using cloud and barcode technology. Barcodes are highlighted as effective tools for providing commanders with timely and accurate inventory data, enhancing overall efficiency.

Alvareze, N.,(2023) has stated that traditional methods like manual tracking with pen and paper or spreadsheets have limitations such as time consumption and lack of real-time updates. The study addressed these challenges by developing an IoT-based equipment

tracking system for Université du Lac Tanganyika (ULT). Using RFID technology, equipment tags that are automatically tracked, preventing unauthorized removal. Agile's Scrum framework guided the system development, while qualitative research methods collected functional and non-functional requirements. An ESP32 microcontroller and AES encryption ensure secure data transmission, with an automatic lock-unlock feature adding further security. A user-friendly web application was developed for equipment management. They concluded that in addition to minimizing the risk of human errors associated with manual paper-based methods, the system has notably decreased the duration required for manual tracking tasks and enhanced the effectiveness of lab management.

Rabiah, N.N et. al (2022), developed a website-based application for inventory management in the State Polytechnic of Sriwijaya specifically the Laboratory of Telecommunication Engineering Study Program that is equipped with instruments vital for practical coursework, which is seen as a promising alternative to the outdated manual system for equipment borrowing, which requires students to complete loan forms provided by lab technicians. Development employed a prototype method, utilizing the Laravel framework for web-based application design. The resulting system aims to aid both technicians and students, integrating QR codes and RFID technology with a centralized database and website interface. Testing confirmed the functionality of the application, demonstrating efficient scan results with QR codes and RFID technology. Overall, the implementation of inventory management utilizing QR codes and RFID technology offers a viable solution for enhancing the processes within the telecommunications engineering study program laboratory.

Li et al. (2020) focus on developing a cloud-based IoT laboratory management system, on the cloud-computing platform of Sichuan University of Science & Engineering where it integrates technologies such as radio frequency identification (RFID), wireless

sensor networks, cameras, and the Internet. It aims to enhance the management of scientific research and teaching laboratories by improving equipment monitoring and utilization efficiency while ensuring precise equipment lifecycle maintenance. The system architecture, topology, and software functions have been meticulously designed. By adopting a Software as a Service (SaaS) model in cloud computing, the system not only reduces development and maintenance costs but also enhances security and stability.

Raad et al. (2019) established a cloud-based IoT high-value laboratory equipment inventory system that features both smart cards and RFID tags used for checkout authentication, the system utilizes low-cost, battery-less tags affixed to the equipment, along with an Arduino microcontroller connected to a GSM shield and uses antenna in determining the equipment's location. Additionally, it can send SMS alerts in case of theft or unauthorized borrowing. The research was conducted in KFUPM University in Saudi Arabia.

SYNTHESIS OF THE REVIEWED LITERATURE

The review of related literature and studies not only provides valuable insights into the current state of laboratory equipment management but also carries significant implications for the present study. By examining existing research and practical applications of innovative technologies in laboratory settings, this literature review aims to identify gaps, opportunities, and best practices that can inform the development and implementation of an effective laboratory equipment management system in the present context. Through a synthesis of relevant literature, this study seeks to build upon existing knowledge, contribute to the advancement of laboratory operations, and inventory management practices.

The works of various authors and researchers, such as Kumbhakarna & Chaudhary (2017), Alvareze, N. (2023), Rabiah, N.N et. al (2022), Li et al. (2020), and Raad et al. (2019), have developed IoT cloud-based monitoring systems that utilize RFID tags and monitor

inventory through web-based management. However, these systems highlight certain limitations, such as the dependency on the Internet and security concerns due to data being sent to the cloud (Madison, A., 2023). Additionally, complex user interfaces or a lack of user-friendly design can hinder the ease of use and adoption of these monitoring systems by laboratory staff. In such cases, a potential alternative is the development of a mobile application that can run offline and does not require a browser to operate its graphical user interface (GUI) (Hamza, Z.A & Hammad, M., 2020). This alternative, equipped with an SD card, enhances the benefits by providing offline access to data, making it invaluable in areas with limited or no internet connectivity. Moreover, SD cards offer enhanced security by giving users direct control over their data and boast faster data transfer speeds compared to cloud storage (Madison, A., 2023). While Muyumba & Phiri (2017) emphasize on the adoption of cloud and barcode technology to improve efficiency, promising but According to Sing, A., (2019) QR Codes outperform them by utilizing both horizontal and vertical patterns to store information which allows QR codes to hold more data.

Considering the existing gaps mentioned in the existing literature, the study entitled Inventory Monitoring System with RFID Access and QR Code for Electronics Engineering Department aims to address these challenges by developing an offline mobile laboratory equipment monitoring system utilizing RFID, ESP32, QR Codes, and SD cards.

Table 3: Comparison between the present study and existing works

Existing studies	**Scope**	**Differences**
RFID basedLab inventory management system (2017)	Development of an RFID-based system for inventory management in a college laboratory, aiming to improve efficiency and accuracy in tracking equipment. The system provides detailed records accessible via a graphical user interface, facilitating various inventory-related tasks.	While both studies utilize RFID technology for equipment monitoring, the present study stands out for its focus on developing an offline mobile-based system. Unlike the PC-based system in Kumbhakarna & Chaudhary's study, the present study offers mobility and flexibility through mobile devices.
A Web-based Inventory Control System Using Cloud Architecture and Barcode Technology for Zambia Air Force (2017)	Comparative analysis of inventory management technologies and development of a prototype system for the Zambia Air Force (ZAF). The study focuses on addressing the challenges of manual systems through	Muyumba & Phiri's study focuses on computerizing inventory management using cloud and barcode technology, whereas the present study emphasizes an offline mobile-based approach and offers QR codes as

	automation using cloud and barcode technology, emphasizing efficiency and accuracy in inventory control.	an improvement for Barcodes.
IoT-based computer laboratory equipment tracking system: a case of Université du lac Tanganyika (2023)	IoT-based equipment tracking system to overcome limitations of traditional manual tracking methods. The system utilizes RFID technology for automated tracking, enhancing security and efficiency in equipment management. It employs an Agile development approach and qualitative research methods to ensure functionality and usability.	Alvareze develops an IoT-based equipment tracking system using RFID technology, aiming to overcome limitations of traditional manual tracking methods. In contrast, the present study introduces an offline mobile-based monitoring system, offering portability and independence from centralized infrastructure.

Web-Based Laboratory Inventory Application Using QR Code and RFID in Telecommunication Engineering Laboratories/Workshops (2022)	Creation of a website-based application for inventory management in a laboratory. The system integrates QR Code and RFID technology to streamline inventory processes and improve efficiency in equipment borrowing and management. Development involves the use of the Laravel framework and prototype methodology.	Rabiah et al. focus on developing a website-based application for inventory management, while the present study centers on creating an offline mobile-based system. The use of RFID, ESP32, QR Codes, and SD cards in the present study provides a versatile solution suitable for environments where internet connectivity may be unreliable.
Design of smart laboratory management system based on cloud computing and internet of things technology (2020)	Development of a cloud-based IoT laboratory management system. The system aims to optimize equipment monitoring and utilization efficiency using RFID, wireless sensor networks, and camera technology. It	Li et al. developed a cloud-based IoT laboratory management system, leveraging RFID and other technologies for equipment monitoring. In contrast, the present study introduces an offline mobile-based

	adopts a Software as a Service (SaaS) model for cost-effectiveness and security.	approach, offering mobility and independence from cloud infrastructure, which may be beneficial in remote or resource-constrained settings
An IoT Based Inventory System for High Value Laboratory Equipment (2019)	Establishment of a cloud-based IoT inventory system for high-value laboratory equipment. The system incorporates smart cards and RFID tags for checkout authentication and utilizes GSM communication for real-time alerts. It aims to enhance security and prevent unauthorized borrowing or theft of equipment.	Raad et al. established a cloud-based IoT inventory system for laboratory equipment, focusing on security features and real-time alerts. The present study, in contrast, focuses on developing an offline mobile-based monitoring system, offering portability and independence from cloud or internet connectivity for equipment monitoring and tracking.

CHAPTER 3

METHODOLOGY

This chapter presents the strategies and methodology used in conducting a quantitative approach in the research project titled “Inventory Monitoring System with RFID Access and QR Code for Electronics Engineering Department”. This chapter also features the theoretical considerations and processes of the inventory monitoring system. The research design, research instruments, data gathering procedure, and the processes in developing the system are also depicted in this chapter.

Research Design

This study used a quantitative research design to assess the functionality and performance of the whole inventory monitoring system with RFID access and QR code scanner designed for the Electronics Engineering Department of this university. The quantitative data gathered from different trials and testing of this research project were analyzed in accordance with the objectives of this research. The researchers seized an experimental approach to evaluate the generated data both in the SD card and Android application. This ensures the accurate and real-time nature of the generated data. Additionally, the reliability of the RFID system and QR code scanner were rigorously tested, evaluating their ability to accurately read and recognize tags and codes. The monitoring capability of the Android application was also investigated. By analyzing the results of the gathered data, the researchers were able to provide a comprehensive interpretation of the proposed system’s reliability, functionality, and consistency.

Data Gathering Procedure

The data gathering procedure implemented in this study focuses on testing and assessing the system's functionality and behavior in various approaches. The data gathering procedure involved the following steps:

1. Group discussion and research.
    1.1. Literature review of existing knowledge and systems to have an in-depth knowledge about existing inventory monitoring systems.
    1.2. Identify all the hardware and software needed for the Inventory Monitoring System with RFID Access and QR Code Scanner for the Electronics Engineering Department to work.
        1.2.1. ESP32
        1.2.2. RFID
        1.2.3. QR code scanner
        1.2.4. SD card module
        1.2.5. Android application
2. Programming and integration of each hardware and software to form a single system.
3. Acquire important data for RFID and QR.
    3.1. Read the UID of each RFID tag by configuring the ESP32 for this function.
    3.2. Generation of QR codes of the laboratory equipment being monitored using Microsoft PowerPoint for testing.
4. Testing of the whole inventory monitoring system.
    4.1. Evaluate the functionality of the RFID System by tapping different tags with different UIDs.
    4.2. Evaluate the functionality of the QR Code Scanner by:
        4.2.1. By scanning all of the QR codes generated containing valuable laboratory equipment information.

4.2.2. QR Codes were tested in the borrowing and returning process.

4.3. Evaluate the reliability of the SD card by checking the CSV file if it contains all the borrow and return details inside it.

4.4. Evaluate the functionality of the Android application by verifying if it is suitable for borrowing, and returning processes.

4.5. Evaluate the accuracy and timeliness of all generated data.

5. Implementation of the Inventory Monitoring System with RFID Access and QR Code Scanner for Electronics Engineering Department.

Data Gathering Tool

To quantify the gathered data in this study, the researchers performed a number of trials in the whole inventory system. This approach supported the quantitative experimental design that is utilized in this study. The results in each trial were recorded and the system's behavior in each trial was compared with other results from a different trial. This comparison strengthens the performance reliability of the system showcasing its consistency. Furthermore, different behavior of the system was observed when it was tested using different approaches and at different times. Ultimately, the tools used for gathering data are as follows:

1. Serial Monitor

The Arduino IDE that is used in programming the hardware in C/C++ language includes a built-in serial monitor with it. During data gathering, this tool is used to check if the RFID reader can read tags, and if data it collects are accurate. The response of the system when a tag is tapped is also verified using this tool, whether the response is "granted" for registered tags, and "forbidden" for unregistered tags. The HTTP requests of the application to the hardware are also verified using this tool, as well as the response of the hardware to the requests of the application.

2. Android application

All history records are being saved in the history page of the application, together with all the relevant details of the transaction. The data that the application collects are verified if they are accurate and real-time.

3. CSV file

All transactions are also being recorded in the CSV file inside the SD Card. It is utilized to validate the data gathered in terms of accuracy and timeliness.

Data Analysis

This section defines the metrics used to interpret the data gathered in this study to evaluate the system's functionality and reliability. The presented metrics helped in quantifying the data obtained from different trials of the system.

Table 4: Interpretation of RFID response

Expected Response	Interpretation
Granted	Successful connection
Forbidden	Unsuccessful connection

Table 4 shows the interpretation of the expected responses from the RFID. When a tag is registered, the response is granted and it will allow successful connection with the application. On the other hand, if a tag is unregistered, the response is forbidden, and it will not allow connection with the Android application.

Table 5: Interpretation of QR response

Expected Response	Interpretation
Valid	Valid QR code
Invalid	Invalid QR code

The interpretation of the QR response is shown in Table 5. The QR gives valid response when the QR code is readable, and it contains the important details which are Inventory Code, Equipment Name, and Remarks. If a QR code is valid, the details page will show up in the application wherein the user inputs the necessary details before borrowing. However, the QR code is read as invalid by the system if it does not contain the information mentioned. Additionally, a “Invalid QR” message will show up.

Table 6: Interpretation of data in Android application and CSV file

Expected Response	Interpretation
Reflected	Accurate/On-time
Not Reflected	Inaccurate/Not On-time

The data shown in the Android application and CSV file were evaluated based on Table 6. If the borrowing and returning information is shown in the Android application and CSV file, the response is regarded as “Reflected”, which also means that the data is accurate and the generation of data is real-time, without any delay or errors. Conversely, if the information is not shown in the Android application and CSV file, it is regarded as “Not Reflected”. Moreover, it is also regarded as not reflected if the data is inaccurate, delayed or advanced in terms of timing, and if there are many errors in the reflected data.

Statistical Treatment

The statistical treatment implemented in this study leveraged the use of mean or average for the times gathered during the comparison of the current method of borrowing and returning laboratory equipment applied in the Analog Laboratory of the Electronics Engineering Department, over the digitized process using the proposed system in this thesis. The formula used is shown below:

$$Mean\ (or\ average) = \frac{\sum_{i=1}^{n} x_i}{n}$$

In this equation, ∑ denotes summation, x_i represents each value in the data set and n is the number of values in the data set.

Theoretical Considerations

Several considerations influenced the whole development process of this research. The principles that were considered outlines the functional requirement needed to be achieved and possessed by the proposed system to ensure its reliability.

1. Accurate

The system should generate accurate data as an output based on the inputs from the user. The RFID reader should accurately read eligible and not eligible tags in accessing the system. The QR scanner must be competent in scanning QR codes of the laboratory equipment. The Android application and CSV file of the SD card must yield and display the accurate data from borrowing, and returning of laboratory equipment.

2. Timeliness

Producing real-time and time-stamped reports further guarantee the monitoring capability of the system. This applies to the data shown in the Android application, as well as the data that are written in the CSV file of the SD Card.

3. Performance

Maximizing the system's performance ensures fast responses from the RFID reader, and QR code scanner. It also allows minimal latency over the system such that the responses are delivered quickly when a request is made. Moreover, this guarantee that the data are written in the CSV file, and shown in the Android application in real-time.

Design Considerations

This segment outlines the considered concepts in designing the Inventory Monitoring System with RFID Access and QR Codes for Electronics Engineering Department. The features and functionalities of the system are shown in this section. The key factors on the utilization of each component of the system are also discussed.

A. System Features

The inventory monitoring system provides the following features:

ESP32. Utilization of ESP32 board that is configured as access point and provides an IP Address where Android mobile devices can connect to communicate with the application.

RFID. Use of RFID tags that can be tapped into the RFID scanner to verify the user's eligibility to access the Android application. Tapping registered tags allows the user to access the Android application, and tapping unregistered tags shows a forbidden message and denies access to the Android application.

SD Card. A SD card that saves all the borrow and return transactions made in the application in a CSV file in real-time. To view the data of the CSV file, the SD card must be unmounted from the SD card module and use a SD card reader to access the file in a desktop or other device that is supported with any spreadsheet software.

Android Application. An Android application developed solely to communicate with the hardware. The application acts as the front-end of the system, handling all the inputs and outputs, and sending of requests that the hardware process to provide accurate responses, acting as the back-end of the inventory monitoring system. The Android application also have the following features:

- Only allow registered tags to access its features. Unregistered tags are forbidden to make any transactions within the application.
- A history page is present inside the Android application which contains all the transactions made within the system. This page is for monitoring purposes. This page also shows if a borrowed equipment is already returned or not. Subsequently if the equipment is not yet returned, the returning processes is also triggered in this page by tapping the “Return” button which then opens the QR scanner for the verification of the borrowed equipment.
- A QR code scanner inside the Android application which can be used to obtain the information of the QR codes placed in each equipment of the Electronics Engineering Department. QR scanning feature is used for borrowing and returning processes of laboratory equipment

B. System Functionalities

The following are the functionalities offered by the system:

Borrowing. The borrowing process of the system acts as a facilitator for the borrowing of laboratory equipment for the Electronics Engineering Department. Utilization of the system indicates the simplification of borrowing process while ensuring the proper and effective management of laboratory equipment. Real-time data from borrowing guarantee a supervised access to the equipment while maintaining the proper knowledge for students.

Returning. The returning functionality of the system assures the integrity of transactions from borrowing equipment. It allows validation and tracking of the usage of laboratory equipment. Real-time data generated from this function establish effective tracking of equipment usage. This functionality also monitors borrowing transactions made within the system and certifies their return.

C. Component Utilization Factors

The following are the factors taken into considerations when choosing the right component:

ESP32. This component is used mainly because of its integrated Wi-Fi capability. ESP32 board can be configured as access point, enabling the device to create its own Wi-Fi network that other devices can connect to. It is an ideal choice for this thesis, allowing it to communicate with the Android application installed in a mobile device. Even though ESP32 also utilizes Bluetooth technology, its Wi-Fi technology was mainly considered since Bluetooth only allows single device to connect, whereas Wi-Fi enables up to 4 devices to connect simultaneously. It is also made with micro-USB which can be connected to a 5V power supply to power it up.

RFID. This technology is utilized to allow seamless authentication of users within the system. The MFRC522 RFID reader module can read RFID tags that operates in the 13.56 MHz range. This allows faster access on the system for eligible users by just simply tapping the registered RFID tag, eliminating the need for user registration and login process. Each tag UIDs are hard coded in the C/C++ code in the Arduino IDE which indicates the tags' registration and eligibility in accessing the application.

SD Card Module. This component accommodates the SD card that is implemented in the system. It provides a venue where the SD card can be mounted to be able to explore its function. Even though the maximum SDHC size capacity of the SD Card Module is 32 GB, this thesis utilizes 16 GB SDHC card to reserve a room for error for the

processing performance of the ESP32. The SDHC card is used to act as a storage for all the transactions made within the system. This component is mainly for external storage.

Android Application. Acting as the front-end of the system, the android application interacts with the user. It provides a setting for the user to explore for borrowing, returning and monitoring. It is considered to be the face of the system as it allows digitized process for an inventory monitoring system and enables remote monitoring of laboratory equipment. Since Android 10 and higher accommodates a large number of Android phones nowadays, it was chosen as the minimum Android operating system of the device in order to install the application.

3D printing. Polylactic Acid (PLA) filament was chosen for 3D printing due to its eco-friendly nature. PLA is a biodegradable material, making it a sustainable choice. Additionally, PLA offers a good balance of strength and durability, a key factor for prototyping projects favored by hobbyists and professionals alike. To address potential heat concerns, considering PLA's 60°C deformation temperature, ventilation holes were integrated into the design to improve airflow and prevent overheating.

D. Programming Language

The following explains the considerations made when choosing the programming language to use in developing the system:

C/C++. Arduino IDE was the environment used in programming the hardware part of the system which uses a programming language similar to C/C++. It is chosen for its ease of integration with the components used, due to the built-in libraries offered by the Arduino IDE. It also supports network communication using HTTP requests and HTTP codes as responses.

Kotlin. Due to their focus on web development, PHP and Python were not suitable choices for this Android application. Kotlin, a modern and expressive language designed for Android development, emerged as the ideal solution. The decision was further strengthened by the prevalence of Android Studio, a powerful IDE that seamlessly integrates with Kotlin. Moreover, Kotlin's familiarity with Java developers, combined with its advanced features, made it a compelling choice for the project.

Block Diagram

The block diagram pictured in this section provides the operational overview of the control system implemented in this study. It is a combination of open-loop and closed-loop system as shown in the diagram.

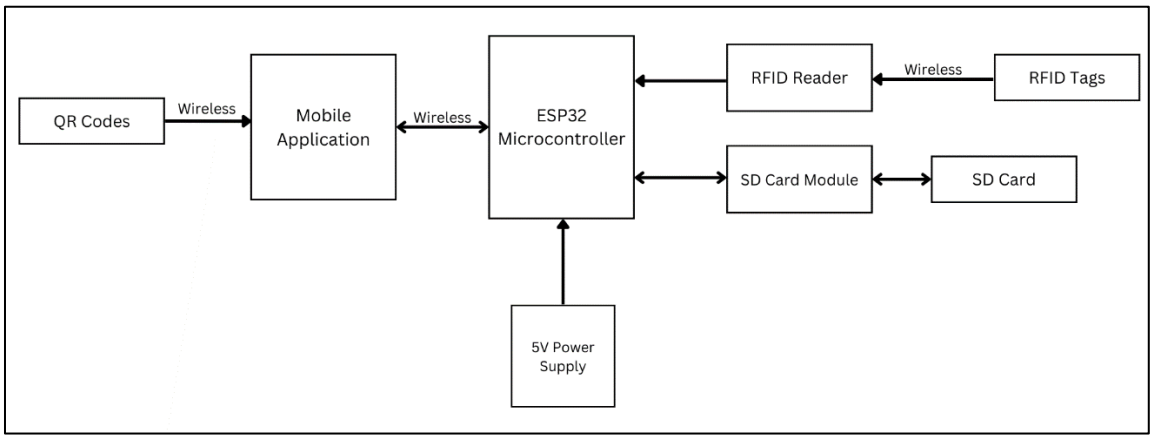


Figure 2: Block Diagram of the Inventory Monitoring System

Illustrated in Figure 2 is the flow of data and processes in the inventory monitoring system. The RFID reader process the RFID tags for authentication whereas the SD card module is a place to mount and unmount the SDHC Card for reading and appending transactions within the system. Both RFID tags and SD card are inputs in the hardware part of the system and ESP32 handles them as the central processing unit (CPU) of the system. The mobile application also interacts with the ESP32 when sending requests and receiving

responses wirelessly by means of HTTP protocol. QR codes are the inputs for the mobile application which are being scanned using the QR code scanner feature of the application.

Collaboration Diagram

This section showcases the functional visualization of the ESP32 as access point in the system, portraying its communication with other devices.

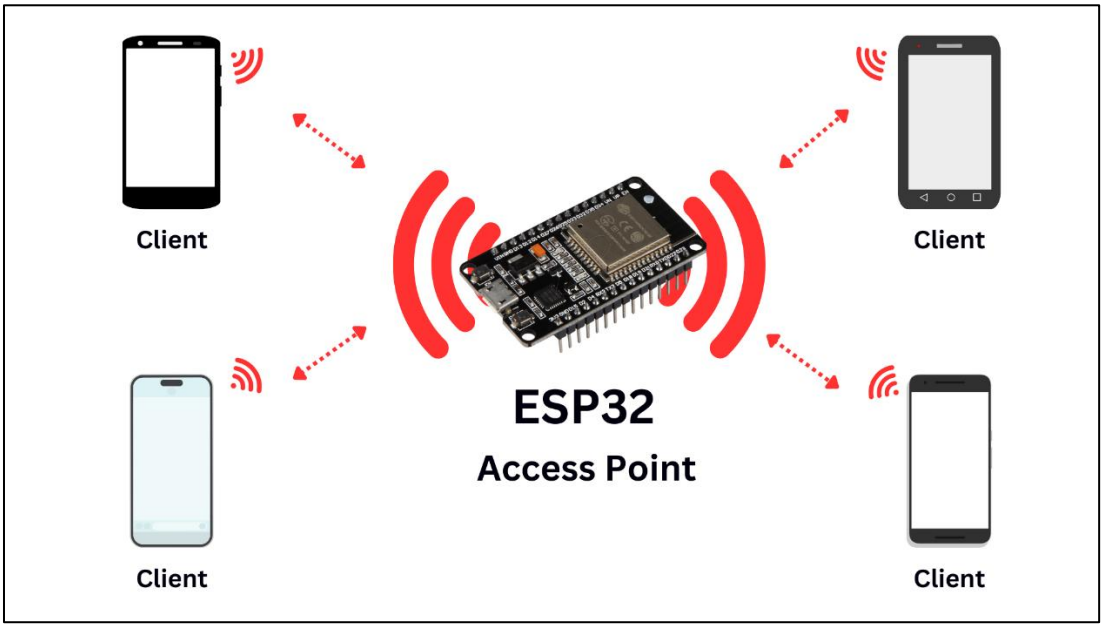


Figure 3: ESP32 as Access Point

When configured as an access point, ESP32 creates its own Wi-Fi network, along with its own SSID, password, and IP address as portrayed in Figure 3. The figure visualizes a total of 4 devices that can connect to the ESP32 using the configured SSID and password. The communication between the ESP32 and a device is then possible due to the configured IP address. All of 4 devices are able to connect to the ESP32 and utilize the Android application on each device simultaneously.

Schematic Diagram

This section displays the schematic diagram of the hardware along with the pin configuration. The components included in the hardware are ESP32, MFRC522 RFID Reader, and SD Card Module. The schematic diagram and the actual circuit design provides a detailed layout of the electrical connection of the components.

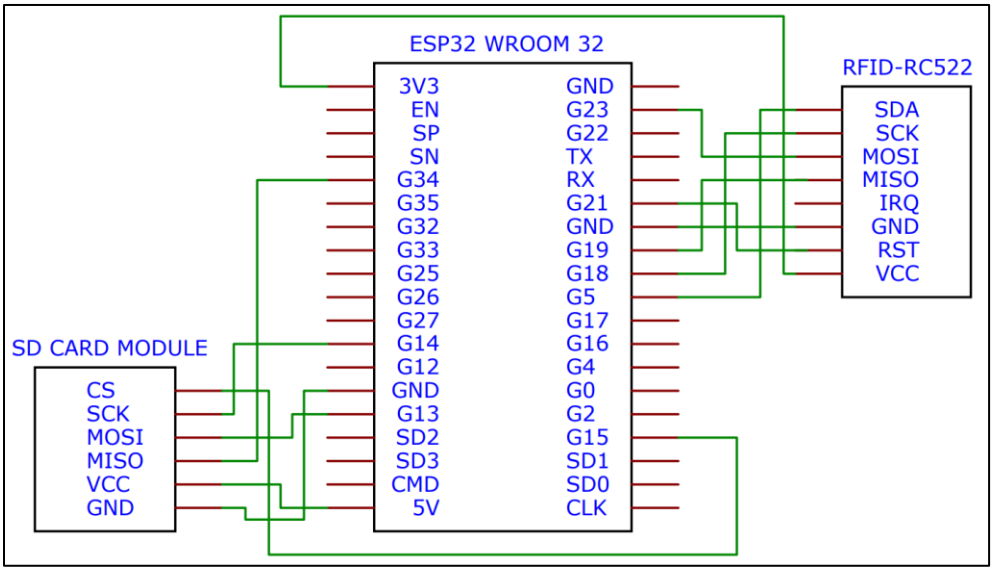


Figure 4: Schematic Diagram of the Hardware

The schematic diagram shown in Figure 4 portrays the wiring diagram of the hardware. The SPI pins of the MFRC522 RFID reader are all connected to respective SPI compatible pins of the ESP32. Other SPI pins of ESP32 are connected to the SPI pins of the SD Card Module allowing communication over the hardware.

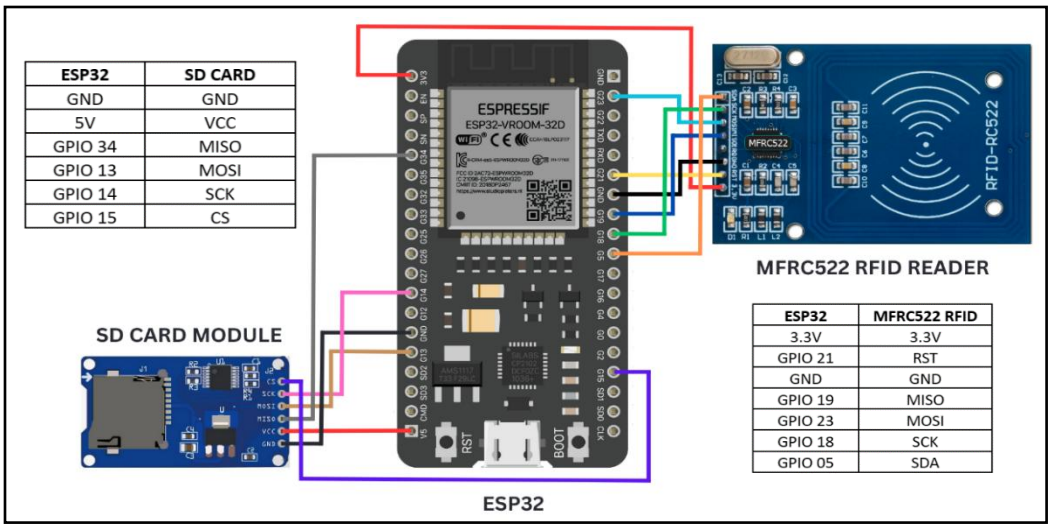


Figure 5: Actual Circuit Design and Pin Configuration of the Hardware

The actual circuit configuration of the MFRC522 RFID reader and the SD card module to the ESP32 as shown in Figure 5. Since both the RFID and SD card module uses SPI communication, the RFID uses the software SPI (SSPI) pins of the ESP32 for straightforward coding without the need to define them in the code anymore. The SD card module on the other hand utilizes the hardware SPI (HSPI) pins of the ESP32, but pin definition in the code is needed for the SD card to work. By utilizing the SSPI and HSPI pins of the ESP32, both the RFID and SD card worked accordingly and concurrently using a single ESP32 board.

Sequence Diagram

This section of the research paper depicts an in-depth visualization of the hardware and Android application communication to each other, outlining the HTTP requests from the application that triggers specific HTTP response from the hardware. HTTP APIs were utilized on these processes.

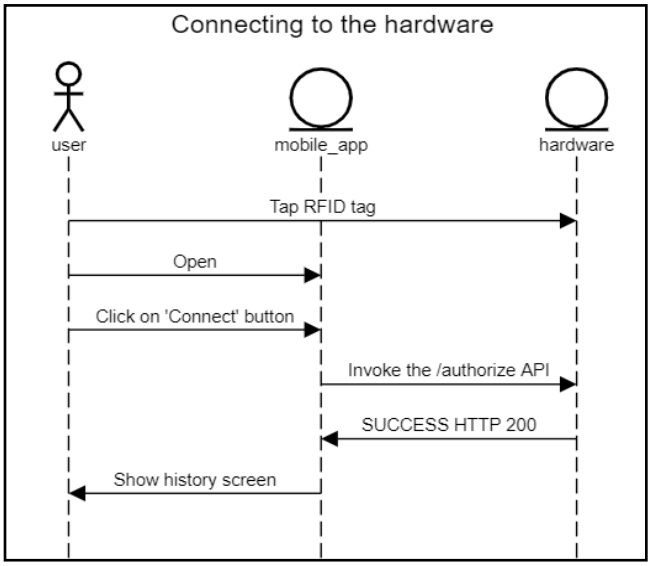


Figure 6: Connecting to the Hardware Sequence Diagram

Figure 6 shows the sequence diagram for the connection process of the device to the hardware. The user taps a RFID tag in the hardware, opens the Android application, and clicks the “Connect” button to trigger a connection for the system. Upon clicking on the “Connect” button, the hardware then processes the request using the /authorize API and sends a HTTP 200 success response if the tapped tag is registered, and HTTP 404 forbidden response if the tapped tag is unregistered. The end result of this process is the showing of the history screen in the Android application.

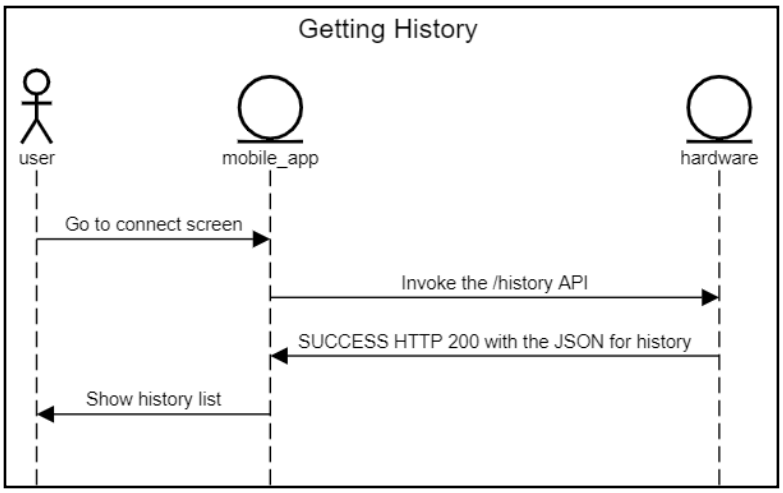


Figure 7: Getting History Sequence Diagram

The sequence diagram for the getting of history process of the application is presented in Figure 7. After a successful connection in the connect screen, the application automatically requests the /history API to the hardware and it will send the HTTP 200 successful response containing all the history information that the hardware reads from the CSV file in the SD card. The resulting page for this process is the history page showing the history list.

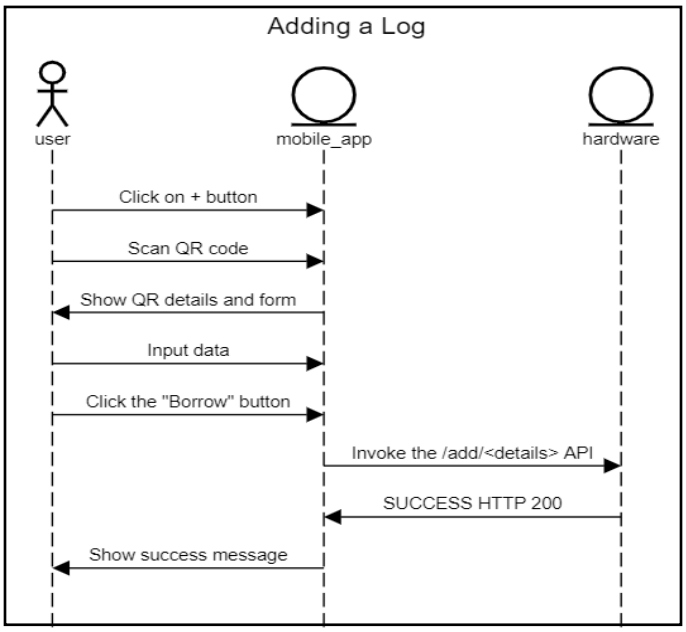


Figure 8: Adding a Log Sequence Diagram

The borrowing procedure is shown in the Adding a Log Sequence Diagram presented in Figure 8. The user clicks on the “+” button that triggers the QR code scanner to open. After successful scanning of the QR code, the QR details and form shows up wherein the user needs to input data that are necessary for borrower’s information for tracking and monitoring of the laboratory equipment. Upon filling up all the necessary information, the “Borrow” button activates which the user can click to successfully borrow an equipment. By click the “Borrow” button, the hardware receives the /add/<details> API and sends a HTTP 200 success response that causes the success message to show.

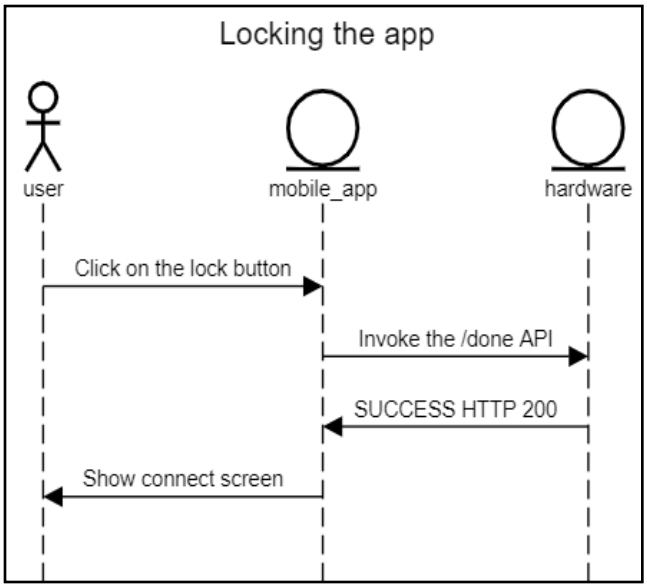


Figure 9: Locking the Application Sequence Diagram

Locking the application sequence diagram in Figure 9 displays the hardware and application process to end the borrowing process. After clicking on the lock button, the hardware receives a /done API and sends a HTTP 200 success response which leads to the connect screen of the application.

3D Design

This section exhibits the 3D casing of the hardware along with the components used.

Inside View

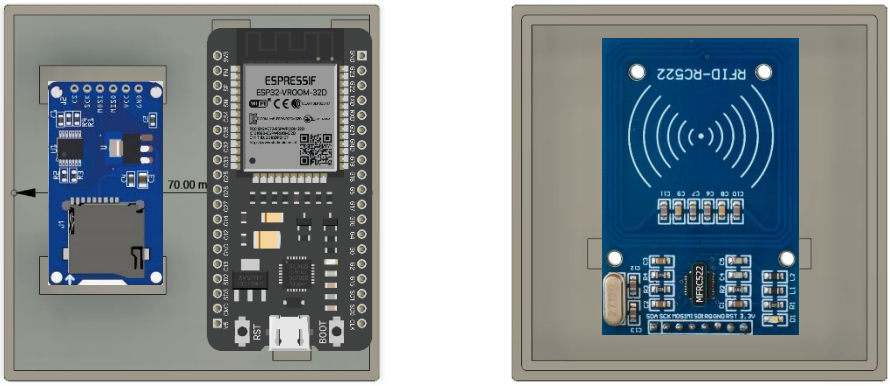


Figure 10: Inside View of the 3D Casing of the Hardware

Outside View

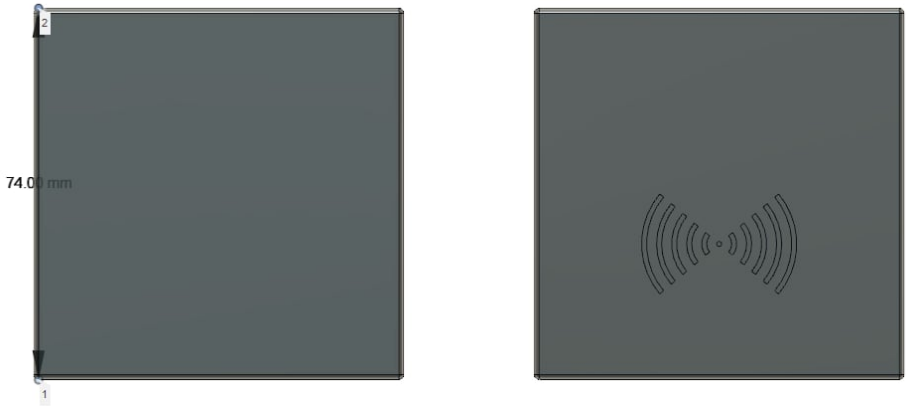


Figure 11: Outside View of the 3D Casing of the Hardware

Figure 10 and 11 exhibits the 3D casing of the hardware. It shows the inside and outside view of the case along with the placements of the ESP32, SD card module, and RFID reader. 3D casing was used to ensure the safety of the hardware.

Flowchart

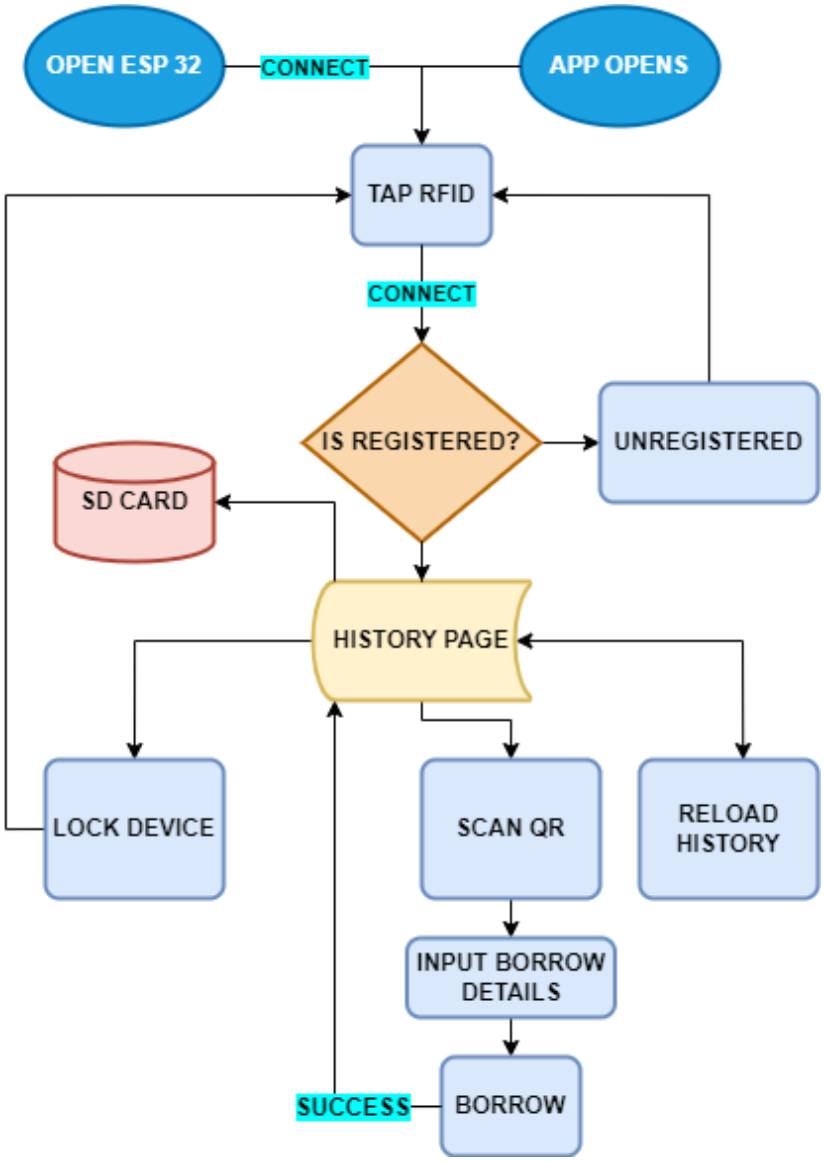


Figure 12: Flowchart of the Borrowing Process

The process of using the system when borrowing is shown in the flowchart displayed in Figure 12.

OPEN ESP32: The system is electrically powered. This also initializes and starts the RFID reader and SD card.

CONNECT: The mobile device connects to the ESP32 using the SSID and password of the Wi-Fi network.

APP OPENS: Open the Android application.

- Note that the OPEN ESP32 and APP OPENS events can occur simultaneously.

TAP RFID: Place a RFID tag in the RFID reader.

CONNECT: Click “Connect” on the application.

- **IF REGISTERED:** Redirects to the History Page of the application.
- **IF UNREGISTERED:** Forbids the access to the application, need to tap a registered tag.

HISTORY PAGE: Fetch the data of the CSV file contained in the SD Card and show them in the application.

- **RELOAD HISTORY BUTTON:** Refresh the history page.
- **LOCK DEVICE BUTTON:** Locks the Android application and redirects it to the connection screen where the connection resets.
- **SCAN QR:** Scan the QR of the equipment being borrowed.
    - **INPUT BORROW DETAILS:** Input borrow details being asked.
    - **BORROW:** Click “Burrow” button
    - **SUCCESS:** Prompt of success borrowing, redirects to History Page.

SD CARD: Storage where all transactions are saved. All details shown in the History Page of the application are fetched from here.

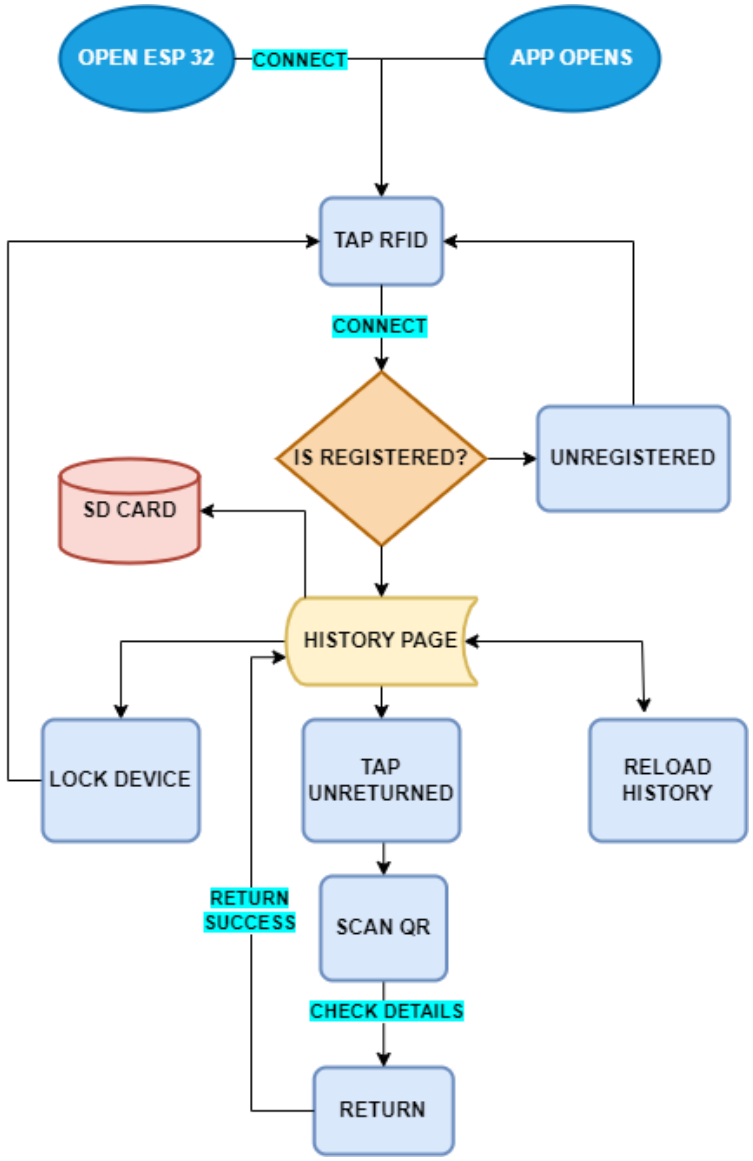


Figure 13: Flowchart of the Returning Process

The returning process of the system is shown in the flowchart displayed in Figure 13.

OPEN ESP32: The system is electrically powered. This also initializes and starts the RFID reader and SD card.

CONNECT: The mobile device connects to the ESP32 using the SSID and password of the Wi-Fi network.

APP OPENS: Open the Android application.

- Note that the OPEN ESP32 and APP OPENS events can occur simultaneously.

TAP RFID: Place a RFID tag in the RFID reader.

CONNECT: Click “Connect” on the application.

- **IF REGISTERED:** Redirects to the History Page of the application.
- **IF UNREGISTERED:** Forbids the access to the application, need to tap a registered tag.

HISTORY PAGE: Fetch the data of the CSV file contained in the SD Card and show them in the application.

- **RELOAD HISTORY BUTTON:** Refresh the history page.
- **LOCK DEVICE BUTTON:** Locks the Android application and redirects it to the connection screen where the connection resets.
- **TAP UNRETURNED:** Tap “Return” on the details card of borrow transaction.
    - **SCAN QR:** Scan the QR code of the equipment being returned. The QR scanner will compare the QR code of the equipment borrowed to the QR of the equipment being returned. Therefore, the QR codes should match for successful returning process.
    - **CHECK DETAILS:** Check for return details.
    - **RETURN:** Click the “Return” button. Successful return redirects to the History Page.

SD CARD: Storage where all transactions are saved. All details shown in the History Page of the application are fetched from here.

Developing the System

Phase I: Planning Phase

The planning phase included the brainstorming and researching on how the system is expected to work and all the materials needed in producing the inventory monitoring system, both in hardware and software. Related literatures were also studied to pin point the gaps with existing inventory monitoring systems. This phase served as the foundation of this research, which allowed the researchers to determine the needed features of the system to be able to properly monitor equipment in the Electronics Engineering Department of this university. Discussions and research were held prior to the production of the inventory monitoring system.

Phase II: Design Phase

For the design phase of this research, the researchers focused on designing an inventory monitoring system that utilizes RFID access, and QR codes for the faculty, laboratory technicians, and students of the Electronics Engineering Department. It is ensured that each subsystem will work harmoniously with each other for them to work as a whole single system. Additionally, the user-friendliness of the system was considered making it easily accessible for all possible users.

Phase III: Development Phase

During this phase, the researchers focused on developing the whole inventory monitoring system. The hardware and Android application were coded simultaneously. All of the hardware used was coded using the Arduino IDE utilizing a programming language that is similar to C and C++. The Android application on the other hand was coded using the IDE named Android Studio employing Kotlin as its main programming language. While programming the hardware and Android application concurrently, their connection with each

other was ensured for them to work hand-in-hand. This is an important process for the Android application to act as the front-end and the hardware as the back-end of the whole inventory monitoring system. This means that the Android application handles all the inputs and outputs of the system, whereas the hardware works all its processes. The development phase also followed an iterative design of development. Hence, the development of the system is a repetitive procedure in terms of feature. Per added feature, the connection between the subsystems involved is tested. Accordingly, the procedure in developing the subsystems and the inventory monitoring system involved the following steps:

A. Hardware

1. ESP32

1.1. Configured the ESP32 as an Access Point by setting up an IP address, SSID, and a password from which a device can connect. In this thesis, the default IP address of the ESP32 device which is 192.168.4.1 is used.

2. RFID

2.1. Coded the MFRC522 RFID system using the ESP32 in such a way that it can read information from 13.56MHz RFID tags, and can display their UID.

2.2. Recorded each UID of the randomly chosen 10 tags.

2.3. Added a RFID feature such that when it reads a tag and verifies that the UID of the specific tag is registered, it will allow connection, and denies other tags that are not registered.

2.4. Integrate the RFID with the ESP32, and utilize HTTP codes as a response for the connection.

2.4.1. HTTP code 200 and 202 for successful connection when the tag is registered and granted access.

2.4.2. HTTP code 404 for forbidden response when the tag is unregistered and access is denied.

3. SD card

    3.1. Coded the SD card with the ESP32 such that:

        3.1.1. It can create a CSV file when it starts up.

        3.1.2. It can append or log data on the CSV file.

        3.1.3. It can read the contents of the CSV file.

4. Designing and soldering of PCB with all the hardware.

5. Designing and printing of a 3D case for the hardware.

B. Android application (iterative process)

1. Connection with RFID

    1.1. UI design and development.

    1.2. Dummy application test.

    1.3. Integration with the RFID.

2. Borrowing

    2.1. UI design and development. QR scanning feature makes use of the ML Kit of Google for Android.

    2.2. Dummy application test.

    2.3. Update together with the last version of the application.

    2.4. Integration with the RFID when granted access.

    2.5. Generation of QR Codes for the laboratory equipment that is under the scope of this study.

        2.5.1. Use of Microsoft PowerPoint in creating QR codes.

        2.5.2. The QR codes includes crucial information for determining a laboratory equipment: Inventory Code, Equipment Name, and Remarks.

3. History fetching

    3.1. UI design and development.

3.2. Dummy application test.

3.3. Update together with the last version of the application.

3.4. Integration with the SD card.

4. Lock function

4.1. UI design and development.

4.2. Dummy application test.

4.3. Update together with the last version of the application.

4.4. Hardware integration.

5. Returning

5.1. UI design and development.

5.2. Update together with the last version of the application.

5.3. Application test.

Phase IV: Testing and Evaluation

The Inventory Monitoring System for Electronics Engineering Department was tested and evaluated per subsystem. The procedures it undergoes are as follows:

1. ESP32
    a. Tested if it creates its own Wi-Fi network with its own SSDI, password, and IP address.
    b. Verified if a device can connect using the configured Wi-Fi network.
2. RFID
    a. Tested using 10 registered tags and 10 unregistered tags with different UIDs.
    b. Verified if it accurately grants access to the registered tags, and not allow unregistered tags.
3. SD Card
    a. Tested if it creates a CSV file upon initialization.

b. Tested for its ability to append transaction data into the CSV file in real-time.
c. Tested for its ability to read transaction data into the CSV file that is fetched to the history page of the application.
d. Verified if the data it appends and read are accurate.

4. Android Application
    a. Tested for its ability to allow registered tags to enter the system, and forbid unregistered tags to use its features.
    b. Tested its borrowing and returning capabilities by the use of its QR scanning feature to read QR codes.
    c. Tested its QR scanning feature if it can read the QR codes generated using Microsoft PowerPoint.
    d. Verified if it can fetch the contents of the CSV file in the SD card and display them into its history page to monitor borrowing and returning transactions made using the system.
    e. Verified if its data generation capability is accurate and real-time.
    f. Tested using 4 devices simultaneously and verified if it can communicate with them concurrently.
5. Current Method vs the Proposed Inventory Monitoring System as a Whole
    a. Compared the two in terms of speed in seconds to verify which process is faster.
    b. Verified its consistency with different trials.
    c. The process is done for both borrowing and returning.

CHAPTER 4

RESULTS AND DISCUSSIONS

This chapter exhibits all the gathered data during the testing and implementation of the Inventory Monitoring System with RFID Access and QR Code Scanner for Electronics Engineering Department with a detailed discussion of the results from a series of trials of testing the system. Tables are provided for comprehensive insights of the data in a more simplified manner. Alongside the tables are the discussion of each. All the results are also analyzed and interpreted in this chapter to determine the functionality, consistency, and reliability of the system and its subsystems.

Presentation of Data

Table 7: RFID Verification of Eligibility for Registered Tags

Tag UID	Trial per RFID tag	Granted	Forbidden
69F61B7A	5	5	0
69D7987A	5	5	0
7933F27A	5	5	0
793BFD7A	5	5	0
FAFE6080	5	5	0
E28C219	5	5	0
71456227	5	5	0
7914417A	5	5	0
69E2BC7A	5	5	0
6983327A	5	5	0

The RFID access component of the inventory monitoring system underwent five (5) tests for each registered tag with different UIDs to confirm its accessibility and ensure that registered tags are eligible for entry into the system, as illustrated in Table 7. The first column of the table lists all the UIDs of each registered tag. The second column shows how many trials the RFID tags went through. The third column indicates the number of times the system responded with "Granted" out of the five trials conducted. Conversely, the fourth column indicates the number of times the system responded with "Forbidden" out of the five trials. As shown in the table, the RFID reader granted access five times for all registered tags, and it did not respond with "Forbidden" for any tag during these five trials.

Table 8: RFID Verification of Eligibility for Unregistered Tags

Tag UID	Trial per RFID tag	Granted	Forbidden
79AAB7A	5	0	5
595F607A	5	0	5
89D307A	5	0	5
69765E7A	5	0	5
1937807A	5	0	5
8976967A	5	0	5
7964577A	5	0	5
79361E7A	5	0	5
894587A	5	0	5
9CEDA7A	5	0	5

For verification purposes, the RFID access component was also tested with unregistered tags to ensure that it does not grant access to them, as shown in Table 8. Again, column one displays the tag IDs of the ten (10) unregistered tags, each having

different UIDs than the registered tags and column two indicates the number of trials the tags underwent. Columns three and four indicate the number of times the system responded with "Granted" and "Forbidden," respectively, for each unregistered tag. During the five trials conducted, the RFID reader responded with "Forbidden" for each unregistered tag and did not provide a "Granted" response, demonstrating that these unregistered tags are not eligible to access the system.

Table 9: QR Code Scanner Functionality for All Monitored Laboratory Equipment

Item	Equipment Code	Trial per Equipment	Valid	Invalid
1	**DT-AT 2017-03**	2	2	0
2	**DT-AT 2017-02**	2	2	0
3	**LM 2321-01**	2	2	0
4	**LM2330-04**	2	2	0
5	**LM2330-03**	2	2	0
6	**LM2230-01**	2	2	0
7	**AT2013-08**	2	2	0
8	**ATF20B-06**	2	2	0
9	**ATF20B-05**	2	2	0
10	**ATF20B-04**	2	2	0
11	**ATF20B-03**	2	2	0
12	**ATF20B-01**	2	2	0
13	**AT2013-07**	2	2	0
14	**AT2013-06**	2	2	0
15	**AT2013-05**	2	2	0
16	**AT2013-04**	2	2	0

17	**AT2013-03**	2	2	0
18	**AT2013-02**	2	2	0
19	**AT2013-01**	2	2	0
20	**3ARB2017-02**	2	2	0
21	**3ARB2017-01**	2	2	0
22	**MT2013-04**	2	2	0
23	**MT2013-03**	2	2	0
24	**MT2013-02**	2	2	0
25	**MT2013-01**	2	2	0
26	**LLT2013-06**	2	2	0
27	**LLT2013-05**	2	2	0
28	**LLT2013-04**	2	2	0
29	**LLT2013-03**	2	2	0
30	**LLT2013-02**	2	2	0
31	**LLT2013-01**	2	2	0
32	**DT2013-08**	2	2	0
33	**DT2013-06**	2	2	0
34	**DT2013-05**	2	2	0
35	**DT2013-04**	2	2	0
36	**DT2013-03**	2	2	0
37	**DT2013-02**	2	2	0
38	**DT2013-01**	2	2	0
39	**ADS1102CML-01**	2	2	0
40	**ADS1102CML-02**	2	2	0
41	**ADS1102CML-03**	2	2	0

42	**ADS1102CML-04**	2	2	0
43	**ADS1102CML-05**	2	2	0
44	**ADS1102CML-06**	2	2	0
45	**MT2018-07**	2	2	0
46	**MT2018-06**	2	2	0
47	**MT2018-05**	2	2	0
48	**MT2018-04**	2	2	0
49	**MT2018-03**	2	2	0
50	**MT2018-02**	2	2	0
51	**MT2018-01**	2	2	0

Table 9 depicts all lists of equipment being monitored in this study, along with the results of the QR code scanning feature of the Android application for two trials each. Shown in the first column is the number of equipment monitored which is 51 items. The second column lists all equipment codes, with various QR codes each, that underwent testing in the system. All QR codes were scanned using the QR code scanner of the application two times as displayed in the third column. Out of the two trials, the equipment were successfully scanned two (2) times which is depicted in the fourth column. The fifth column indicates zero (0) unsuccesful result for the QR codes. The QR scanner always reads the QR code presented to it. However, succesful result only means that the page where the user inputs the borrow details is opened. Consequently, unsuccessful result do not open that page, instead it shows a “Invalid QR code” message. This only means that the QR code scanner can successfully scan the QR codes of each laboratory equipment being monitored under this study, and allows the user to input details, especifically the student borrower name, the subject, and the purpose.

Table 10: Status of the Borrow Details in the Android Application

Details	Trials				
	1-10	11-20	21-30	31-40	41-51
Date and Time Borrowed	Reflected	Reflected	Reflected	Reflected	Reflected
Date and Time Returned	Reflected	Reflected	Reflected	Reflected	Reflected
Tag ID	Reflected	Reflected	Reflected	Reflected	Reflected
Student Name	Reflected	Reflected	Reflected	Reflected	Reflected
Professor Name	Reflected	Reflected	Reflected	Reflected	Reflected
Subject	Reflected	Reflected	Reflected	Reflected	Reflected
Purpose	Reflected	Reflected	Reflected	Reflected	Reflected
Inventory Code	Reflected	Reflected	Reflected	Reflected	Reflected
Equipment Name	Reflected	Reflected	Reflected	Reflected	Reflected

Table 10 confirms a positive success rate in reflecting borrowing details throughout 51 trials on the Android application, indicating accurate data retrieval and integration between the backend and user interface. For each transaction, the application conveniently records all relevant details, including date and time borrowed, date and time returned, tag

ID, student name, professor name, subject, purpose, inventory code, and equipment name, as shown in the table.

Table 11: Number of Devices that can Access the Application Simultaneously

Devices	History	Borrowing Interface
1	Reflected	Reflected
2	Reflected	Reflected
3	Reflected	Reflected
4	Reflected	Reflected

Table 11 shows that multiple devices, up to 4 devices simultaneously, can access the history of the borrowing process that is saved in the SD card and scan a new QR code to proceed to the borrowing form. The first column indicated the number of devices that access the system simultaneously, the second column was the history, across all devices, history was reflected, and the last column depicted that the borrowing interface of the application was reflected across four trial devices.

Table 12: Status of the Borrow Details in CSV File of the SD Card

Details	Trials				
	1-10	11-20	21-30	31-40	41-51
Date and Time Borrowed	Reflected	Reflected	Reflected	Reflected	Reflected
Date and Time Returned	Reflected	Reflected	Reflected	Reflected	Reflected
Tag ID	Reflected	Reflected	Reflected	Reflected	Reflected

Student Name	Reflected	Reflected	Reflected	Reflected	Reflected
Professor Name	Reflected	Reflected	Reflected	Reflected	Reflected
Subject	Reflected	Reflected	Reflected	Reflected	Reflected
Purpose	Reflected	Reflected	Reflected	Reflected	Reflected
Inventory Code	Reflected	Reflected	Reflected	Reflected	Reflected
Equipment Name	Reflected	Reflected	Reflected	Reflected	Reflected
Remarks	Reflected	Reflected	Reflected	Reflected	Reflected

The results in Table 12 show that all borrowing details were saved flawlessly to a CSV file on the SD card over 51 trials. This suggests the system can accurately retrieve data and seamlessly link the backend with the user interface. Trials 1 to 51 shows the date and time borrowed, date and time returned, tag ID, student name, professor name, subject, purpose, inventory code, equipment name and remarks are all reflected on the CSV file of the SD card, which can be viewed using a spreadsheet software such as Microsoft Excel.

Table 13: Speed of Borrowing Time in Seconds

Trial	Current Method	Digitized
1	138.98	39.30
2	171.31	41.43
3	160.23	32.92

4	142.98	30.32
5	155.90	37.31
6	141.76	34.02
7	129.08	28.90
8	134.95	29.53
9	145.67	30.82
10	162.02	34.85
11	156.42	38.21
12	178.03	42.32
13	167.82	40.45
14	145.35	39.99
15	189.33	43.54
16	190.21	49.80
17	184.05	48.07
18	145.91	35.55
19	176.32	36.32
20	216.75	50.21
21	154.82	41.21
22	166.43	39.31
23	189.40	41.56
24	145.67	33.65
25	134.89	32.88
26	168.97	38.22
27	143.12	35.32
28	133.56	31.29

29	152.35	39.42
30	178.31	40.21
31	173.35	39.09
32	187.24	40.53
33	161.45	31.21
34	121.91	28.42
35	190.53	43.93
36	176.14	39.42
37	126.27	30.01
38	175.02	35.92
39	157.95	33.67
40	149.09	30.78
41	186.80	37.61
42	156.78	32.65
43	169.01	34.52
44	142.79	40.50
45	176.33	39.22
46	122.98	29.49
47	136.77	30.42
48	195.64	40.04
49	188.92	35.59
50	167.91	39.91
51	166.30	34.29
Mean:	161.37	39.94

Table 13 presents the time taken for the logging and borrowing process of equipment across trials 1 through 51, utilizing both the current method and digitized systems. The average duration of the borrowing process for each student was calculated by determining the mean of all these processes, expressed in seconds. By comparing the average time in seconds of the current method and the digitized method using the proposed project, it can be said that the digitized method is 4x faster than the current method.

Table 14: Speed of Returning Time in Seconds

Trial	Current Method	Digitized
1	118.08	10.30
2	131.56	8.43
3	120.09	10.92
4	132.22	12.32
5	115.12	10.31
6	71.45	11.02
7	91.65	7.90
8	160.02	8.53
9	98.32	10.82
10	62.89	9.85
11	71.82	12.21
12	79.26	11.32
13	68.46	9.45
14	125.04	8.99
15	88.32	10.54
16	91.61	8.80
17	86.01	9.07

18	92.54	9.55
19	79.76	7.32
20	87.79	11.05
21	76.24	11.21
22	66.79	10.31
23	87.79	13.56
24	95.34	10.65
25	134.23	9.88
26	98.93	11.22
27	93.54	9.32
28	73.21	8.29
29	112.13	10.42
30	128.11	7.21
31	83.34	9.09
32	87.21	10.53
33	61.75	13.21
34	81.20	9.42
35	90.37	10.93
36	75.53	12.42
37	106.23	7.81
38	115.87	11.92
39	57.45	10.67
40	69.78	8.78
41	86.93	12.61
42	103.05	9.65

43	69.34	11.52
44	92.56	10.50
45	76.87	12.22
46	82.53	8.49
47	96.01	7.42
48	95.98	11.04
49	88.67	12.59
50	67.91	10.91
51	76.43	8.29
Mean:	92.22	10.21

Table 14 shows the comparison of the time taken for the returning process of equipment using the current method vs the digitized method using the proposed project. Across trial 1 through 51, the proposed system shows faster speed in when returning a laboratory equipment as compared to the conventional method of borrowing. By looking at the average time, the digitized method is 9x faster than the current method of returning process.

Interpretation of Data

The tables along with the detailed discussions that were presented shows positive standing for the proposed Inventory Monitoring System with RFID Access and QR Code for Electronics Engineering. The accuracy, timeliness, and performance of the system was tested and verified as shown on the tables along with the discussions of each. Testing the system with many trials verifies its consistency. Thus, the following are the interpretation that can be made based on the data generated:

A. RFID

- Table 7 and 8 shows that the RFID reader can acuurately identify registered and unregistered tags.
- In terms of performance, it allows only registered tags to access the system and forbids unregistered tags showcasing its reliability.

B. Android Application

a. QR scanning feature

- Table 9 shows that the QR scanner can accurately read the information embedded on the QR codes, which are Inventory Code, Equipment Name, and Remarks.
- In terms or performance, the QR scanner reads tag with the 3 information as Valid, and reads QR code without the 3 information as invalid, not allowing other QR codes that are not supported by the proposed system to be processed.

b. Monitoring feature

- Table 10 confirms that the application accurately displays the borrow and return details.
- The timeliness of the system was also verified since the date and time borrowed and the date and time returned of the laboratory equipment are generated in real-time.

c. Simultaneous Usage

- Table 11 displays that the Android Application can be used simultaneously using a maximum of 4 devices connected to the ESP32.

C. SD Card

- Table 12 displays that the CSV file reflects the accurate, and real-time data generated from borrowing and returning laboratory equipment.

D. Borrowing and Returning

- Table 13 shows that the proposed system is 4x faster than the current method of borrowing, featuring the much better performance of the system as compared to manual methods of borrowing.
- Table 14 indicates that the system is 9x faster than the current method of returning laboratory equipment.

CHAPTER 5

SUMMARY OF FINDINGS, CONCLUSIONS AND RECOMMENDATIONS

This chapter delves into the core of the project, presenting summary of findings, and conclusion of the project's overall performance. To propel future advancements, recommendations for further exploration are presented.

Summary of Findings

The following are the findings of the study:

1. The use of RFID tags allows faster authentication for the use the application. It grants access to the registered tags and forbids unregistered tags. The registration of tags is hard-coded using Arduino IDE. The system was set to have a timeout after 5 minutes of inactivity, thus re-tapping of the RFID tag is required. It's crucial to note that if a new RFID tag is tapped before clicking the "Borrow" button when borrowing, the system prioritizes the most recently scanned tag. This means the borrower information saved in the CSV would correspond to the last tapped RFID, not necessarily the tag used to connect to the application.
2. The QR code scanner can read all QR codes allowing seamless monitoring and tracking of laboratory equipment conditions and status. However, QR codes that have the Inventory Code, Equipment Name, and Remaks embedded into it are the only QR codes that are considered as valid QR codes. The system consider other QR codes that do not have these information as invalid QR codes.
3. The Android application enables remote monitoring of the transactions done within the system. It handles the borrowing and returning functions of the system and accurately displays the generated data from the transaction in real-time. A maximum of 4 devices can access the application and its function simultaneously. This digitized method of borrowing and returning came out ot be a faster process

than the conventional method employed in the Analog Laboratory of the Electronics Engineering Department.

4. The CSV file in the SD card showed real-time and time-stamped reports of all the borrowing and returning activities made within the system. All important transaction details are reflected to it enabling transparency over the system. The contents of the CSV file can be viewed by using a spreadsheet software such as Microsoft Excel.

Conclusions

The Inventory Monitoring System with RFID Access and QR code was designed to revolutionize the Electronics Engineering Department's equipment borrowing process by transitioning it from a paper-based system to a digitized one. This innovative solution tackles several shortcomings of the current manual system. Firstly, the manual process relies on handwritten records, which are prone to errors and difficulties in data retrieval. Secondly, the current system lacks real-time equipment status updates. Finally, the system is heavily dependent on a single laboratory technician, creating a bottleneck and potentially leading to insufficient monitoring and documentation of borrowed items. This, in turn, complicates the tracking process and increases the risk of misplaced or lost equipment. The proposed Inventory Monitoring System with RFID Access and QR Codes for Electronics Engineering Department aimed to address these issues by introducing a more efficient, transparent, and reliable method for managing the department's valuable laboratory equipment.

The proposed system addresses the shortcomings of the current borrowing and returning process by leveraging advanced technologies. It provides real-time, and time-stamped reports for improved data accuracy and accountability. RFID tags and QR code authentication eliminated manual record-keeping and human error, while the user-friendly application centralized borrowing activities and offer easy access for all users. To ensure

system reliability, the application and hardware components underwent rigorous testing and debugging protocols, including average time testing to identify its performance speed. This comprehensive upgrade aimed to simplify and expedite the borrowing and returning process, enhance data security, and ultimately improve user experience within the department.

Testing of the Application-based Inventory Monitoring System for the Electronics Engineering Department of the Don Honorio Ventura State University yielded positive results, confirming its potential to revolutionize inventory management and borrowing and returning process. The system seamlessly integrates user-friendly interfaces that streamlines the user experience, while verification mechanisms ensure data security. Furthermore, QR code functionalities optimize the existing inventory monitoring, and borrowing and returning methods. These positive outcomes demonstrate the system's ability to address the current borrowing and returning challenges of the department and its potential to elevate operational efficiency within the institution, paving the way for a more smooth and secure approach to inventory monitoring, and borrowing and returning process in educational settings.

Recommendations

Several recommendations for future exploration and further development of the systems are as follows:

1. Registration of RFID tags using the Android application, not hard-coded.
2. Utilization of Masterlist of laboratory equipment along with corresponding QR codes, for scalability and expanding purposes over the system. This also allows easy updates for equipment conditions.
3. Use of GPS for real-time location tracking of a borrowed equipment.

REFERENCES

Xie, H. (2015). Empirical Study Of An Automated Inventory Management System With Bayesian Inference Algorithm. International Journal of Research in Engineering and Technology, 04(10), 398–405. https://doi.org/10.15623/ijret.2015.0410065

G. A. Kumbhakarna and R. P. Chaudhari, "RFID based lab inventory management system," 2017 International Conference on Information, Communication, Instrumentation and Control (ICICIC), Indore, India, 2017, pp. 1-3, doi: 10.1109/ICOMICON.2017.8279154.

Ma, Lifang & Mu, Yonghong & Wei, Ling & Wang, Xiumei. (2021). Practical Application of QR Code Electronic Manuals in Equipment Management and Training. Frontiers in Public Health. 9. 726063. 10.3389/fpubh.2021.726063.

Raad, W., Bueno-Delgado, M. V., Deriche, M., & Suliman, W. (2019). An IoT Based Inventory System for High Value Laboratory Equipment. 2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS). https://doi.org/10.1109/iotsms48152.2019.8939259

Erlangga, S. B., Yunita, A., & Satriana, S. R. (2022). Development of Automatic Real Time Inventory Monitoring System using RFID Technology in Warehouse. JOIV : International Journal on Informatics Visualization, 6(3), 636. https://doi.org/10.30630/joiv.6.3.1231

Shao, D., & Wang, J. (2021). Discussion on Laboratory Computer Management and Maintenance of Computer Course. In 6th Annual International Conference on Social Science and Contemporary Humanity Development, 262-266.

Procedures, E. M. (2019). Equipment Management Services Procedure Manual. Utah State University, 1–32.

Madison, A. (2023). Choose wisely: SD vs cloud storage. Retrieved from https://www.q-see.com/blogs/news/choose-wisely-sd-vs-cloud-

Paul, S., Chatterjee, A., & Guha, D. (2019). STUDY OF SMART INVENTORY MANAGEMENT SYSTEM BASED ON THE INTERNET OF THINGS (IOT). International Journal on Recent Trends in Business and Tourism, 3(3), 27–34. http://ejournal.lucp.net/index.php/ijrtbt/article/view/749

Babiuch, M., Foltynek, P., & Smutny, P. (2019). Using the ESP32 Microcontroller for Data Processing. 2019 20th International Carpathian Control Conference (ICCC). doi:10.1109/carpathiancc.2019.8765944

M. Broell, L., Hanshans, C., & Kimmerle, D. (2023). IoT on an ESP32: Optimization Methods Regarding Battery Life and Write Speed to an SD-Card. IntechOpen. doi: 10.5772/intechopen.110752

Allafi, I., & Iqbal, T. (2017). Design and implementation of a low-cost web server using ESP32 for real-time photovoltaic system monitoring. 2017 IEEE Electrical Power and Energy Conference (EPEC). doi:10.1109/epec.2017.8286184

Dejan Nedelkovski, (2017), "How RFID Works and How To Make an Arduino based RFID Door Lock" Arduino Tutorials – How To Mechatronics", Retrieved 29 April 2024 from ://howtomechatronics.com/tutorials/arduino/rfid-works-make-arduino-based-rfid-door-lock.

Soni, S., Soni, R., & Waoo, A. A. (2021). RFID-based digital door locking system. Indian Journal of Microprocessors and Microcontroller (IJMM), 1(2), 17-21

Luca Ardito, Riccardo Coppola, Giovanni Malnati, Marco Torchiano, Effectiveness of Kotlin vs. Java in android app development tasks, Information and Software Technology, Volume 127, 2020, 106374, ISSN0950-5849, https://doi.org/10.1016/j.infsof.2020.106374.

Ntafatiro, A. (2023). Iot-based computer laboratory equipment tracking system: a case of Université du lac Tanganyika (Doctoral dissertation, NM-AIST).

Li, S., Gao, X., Wang, W., & Zhang, X. (2020, June). Design of smart laboratory management system based on cloud computing and internet of things technology. In Journal of Physics: Conference Series (Vol. 1549, No. 2, p. 022107). IOP Publishing.

Todd Sproull and Doug Shook. (2023). Machine Learning on the Move: Teaching ML Kit for Firebase in a Mobile Apps Course. In Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 2 (SIGCSE 2023). Association for Computing Machinery, New York, NY, USA, 1182. https://doi.org/10.1145/3545947.3569615

ESP32 Wi-Fi & Bluetooth SOC | Espressif Systems. (n.d.). https://www.espressif.com/en/products/socs/esp32

Magdy, K. (2023, August 17). ESP32 WiFi Tutorial & Library Examples (Arduino IDE). DeepBlue. https://deepbluembedded.com/esp32-wifi-library-examples-tutorial-arduino/#getting-started-with-esp32-wifi

Santos, S., & Santos, S. (2022, September 20). Guide to SD Card Module with Arduino | Random Nerd Tutorials. Random Nerd Tutorials. https://randomnerdtutorials.com/guide-to-sd-card-module-with-arduino/#Introducing%20The%20SD%20Card%20Module

X. Liu, Bin Xiao, Feng Zhu and Shigeng Zhang, "Let's work together: Fast tag identification by interference elimination for multiple RFID readers," 2016 IEEE 24th International Conference on Network Protocols (ICNP), Singapore, 2016, pp. 1-10, doi: 10.1109/ICNP.2016.7784427.

Putranto, B. P. D., Saptoto, R., Jakaria, O. C., & Andriyani, W. (2020). A comparative study of Java and Kotlin for Android mobile application development. 2020 3rd International Seminar on Research of Information Technology and Intelligent Systems (ISRITI). https://doi.org/10.1109/isriti51436.2020.9315483\

Bose, S. (2018). A COMPARATIVE STUDY: JAVA VS KOTLIN PROGRAMMING IN ANDROID APPLICATION DEVELOPMENT. International Journal of Advanced Research in Computer Science, 9(3), 41–45. https://doi.org/10.26483/ijarcs.v9i3.5978

Suder, J., Bobovsky, Z., Mlotek, J., Vocetka, M., Zeman, Z., & Safar, M. (2021). EXPERIMENTAL ANALYSIS OF TEMPERATURE RESISTANCE OF 3D PRINTED PLA COMPONENTS. MM Science Journal, 2021(1), 4322–4327. https://doi.org/10.17973/mmsj.2021_03_2021004

Hasan, M. R., Davies, I. J., Pramanik, A., John, M., & Biswas, W. K. (2024). Potential of recycled PLA in 3D printing: a review. Sustainable Manufacturing and Service Economics, 3, 100020. https://doi.org/10.1016/j.smse.2024.100020

Google. (n.d.). Barcode scanning | ML kit | google for developers. Google. https://developers.google.com/ml-kit/vision/barcode-scanning?authuser=1

U. Schafer (2019). Teaching Modern C++ with Flipped Classroom and Enjoyable IoT Hardware. 2019 IEEE Global Engineering Education Conference (EDUCON). doi:10.1109/educon.2019.8725068

APPENDIX A

TITLE PROPOSAL APPROVAL SHEET

APPENIX B

USER MANUAL

The screenshots and operation of the implemented system are introduced and depicted underneath.

I. INSTALLATION GUIDE

The user will need to download the app via the Google Drive link provided. The APK file for the installer is shown on Figure 14.

https://drive.google.com/drive/folders/1l1iibeDq3tZBVk8V5LyfCH-nsKWtpKeH?usp=sharing

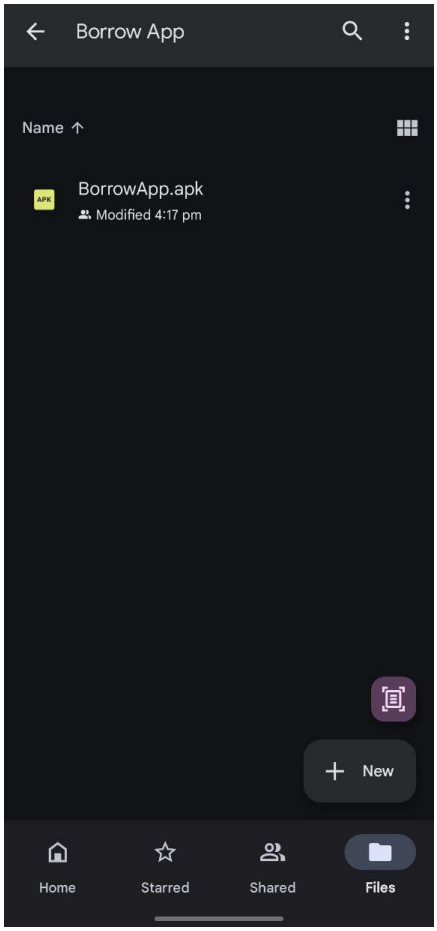


Figure 14: Google Drive Folder for APK installer

II. INSTALLATION PROMPT FOR THE APPLICATION (Android OS)

The user will then open the file explorer to install the APK in order to start the use of the application. The prompt for the installation process for the application is shown in Figure 15.

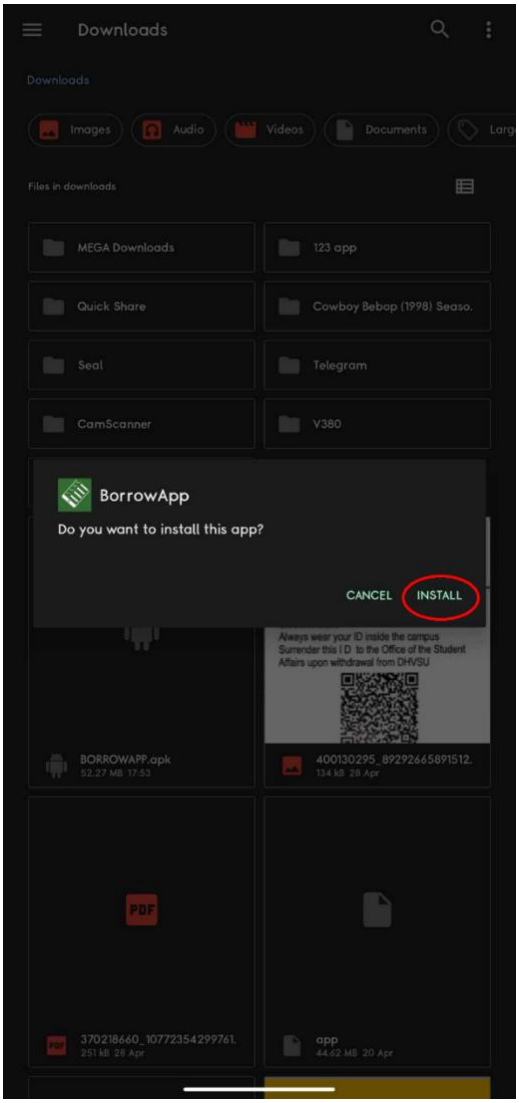


Figure 15: Installation Prompt for the Application

III. APPLICATION ICON AND LAUNCHER ON THE DEVICE

The icon will then be shown on either the applications drawer or on the device's home screen shown in Figure 16 and Figure 17.

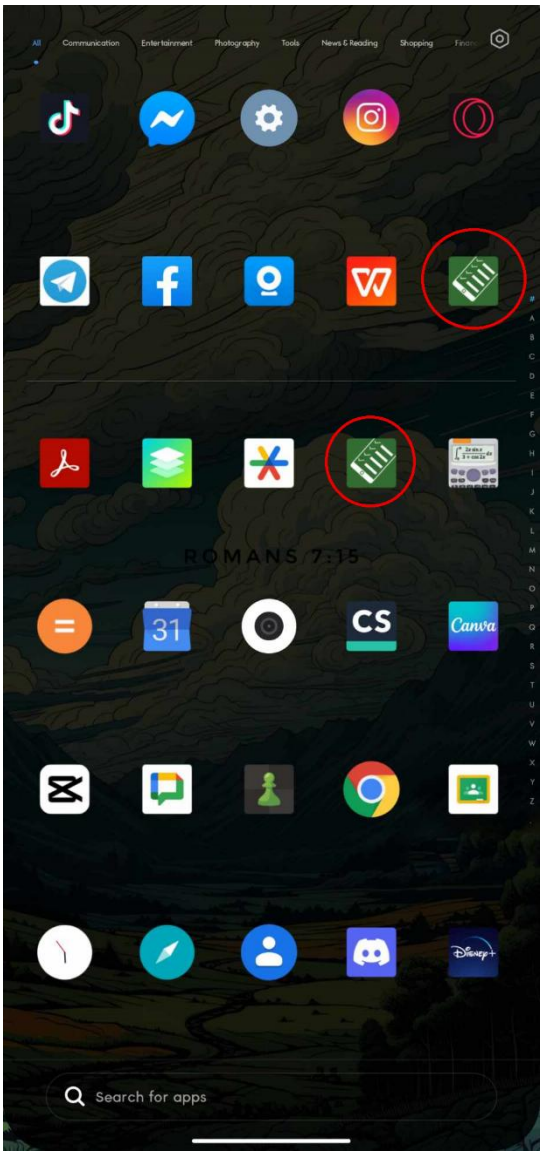


Figure 16: Application Icon on the Application Drawer

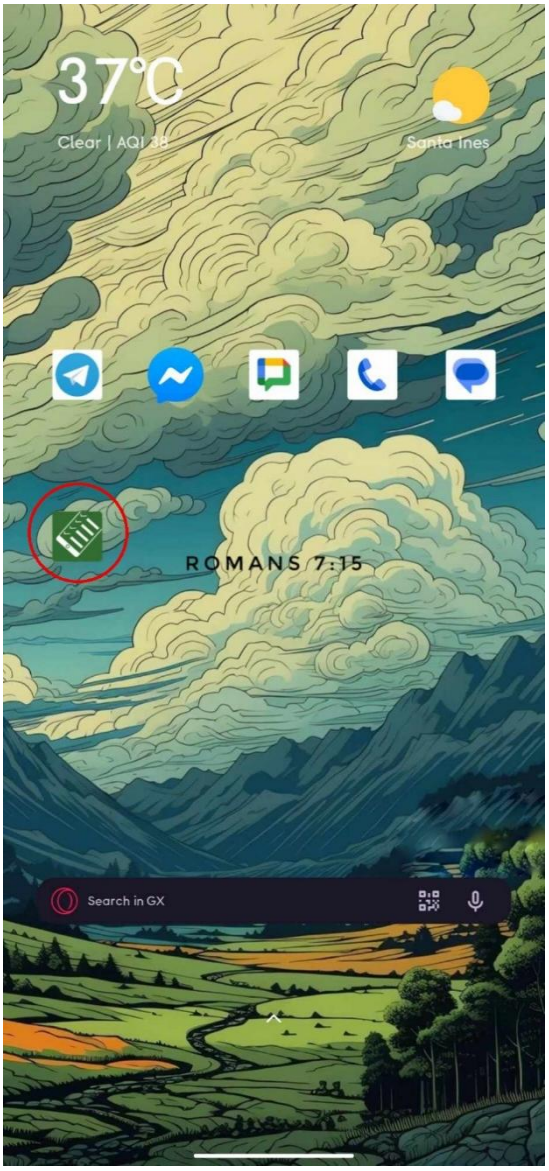


Figure 17: Application Icon on the Home Screen

IV. ESP 32 WIFI CONNECTION

After the installation the user has to connect to the ESP32 module via WIFI in order to access the application. ESP32 connection is shown on Figure 18.

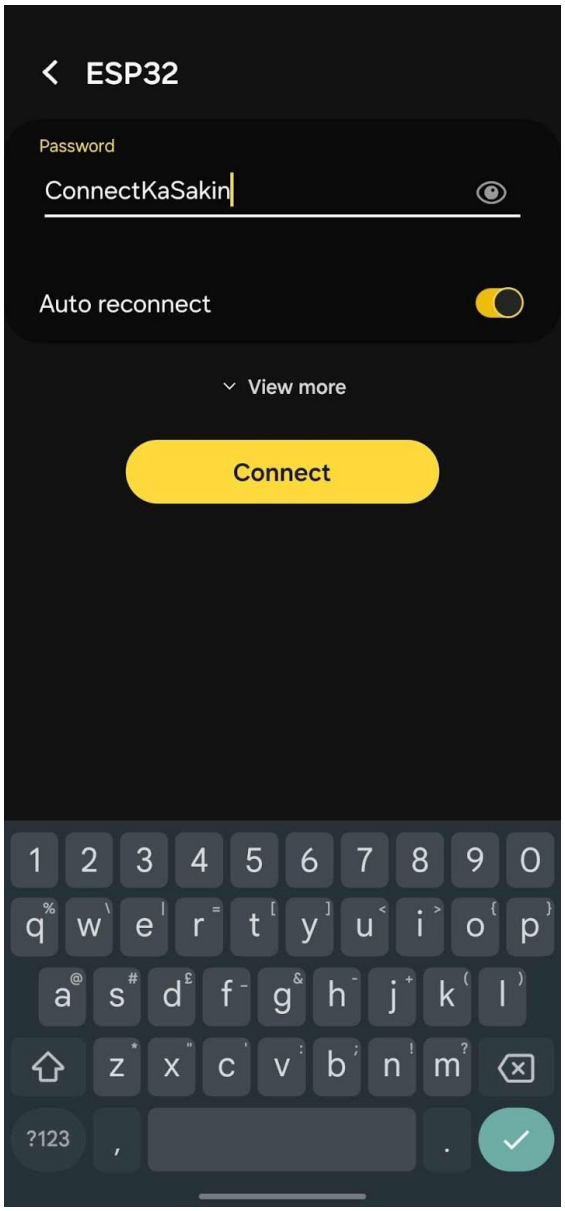


Figure 18: ESP32 Connection via WIFI on Android Device.

V. HARDWARE AND SOFTWARE COMMUNICATION

The user can access the application while connected to the ESP32 module. Once connected, the user can initiate interactions by tapping an RFID tag on the scanner and subsequently pressing the "CONNECT" button to commence app utilization. Both functions can be seen on Figure 19 and Figure 20.

Figure 19: Interaction of RFID Tag to the RFID Scanner

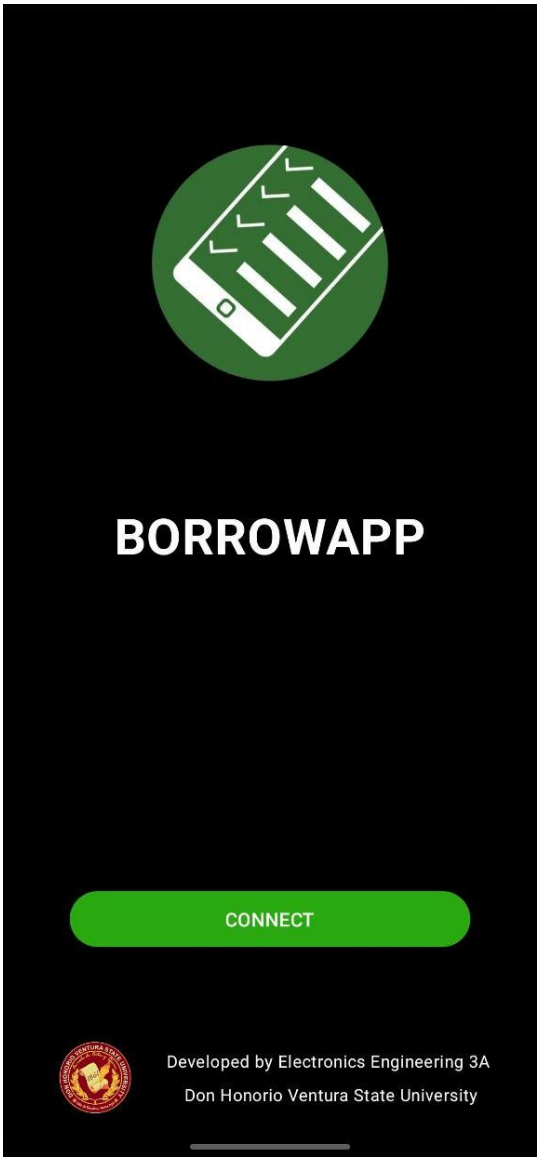


Figure 20: Application Prompt for the RFID Scanner

VI. HOME SCREEN FOR THE APPLICATION

Upon successful connection between the app and the ESP32 module, the user is directed to the homepage. Here, they encounter a display of the borrowing history along with three function buttons for further interaction. The homepage and history page are both displayed on the screen on Figure 21. The three function buttons are explained in Figure 22.

Figure 21: Homepage and History Page of the Application

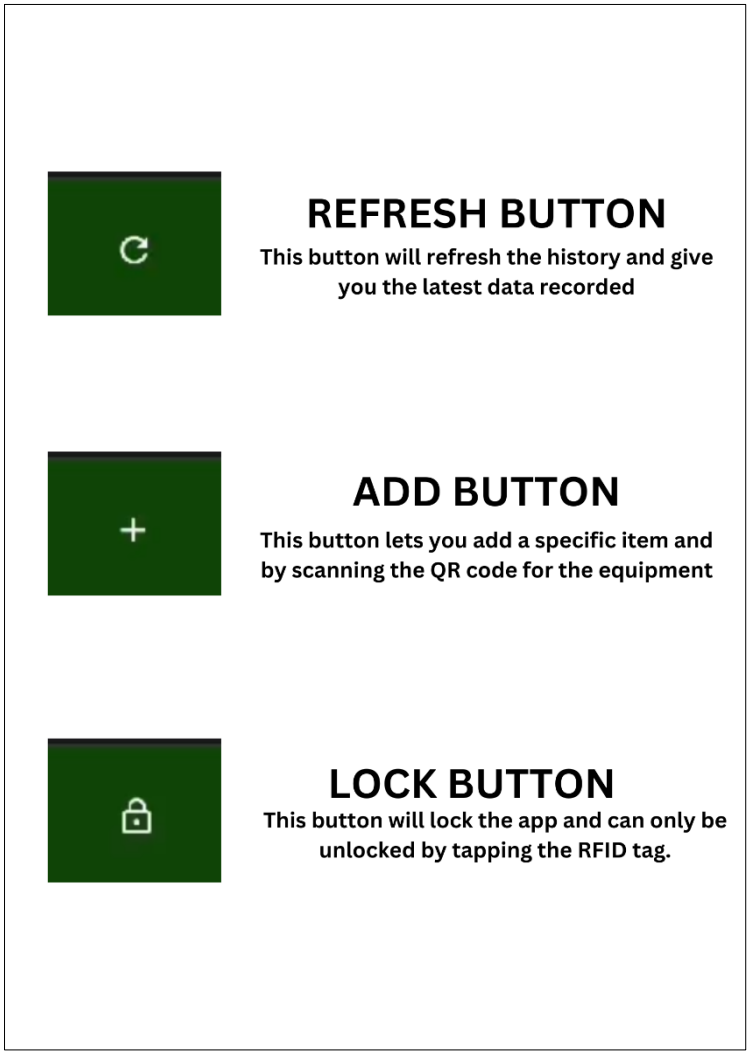


Figure 22: The Three Buttons Along with Their Functions

VII. BORROWING PROCESS

To initiate the borrowing process, the user should press the "+" button shown in Figure 23. Subsequently, the QR scanner interface will open, enabling the user to start scanning QR codes. Tap again the “+” button to scan a QR code as shown in Figure 24.

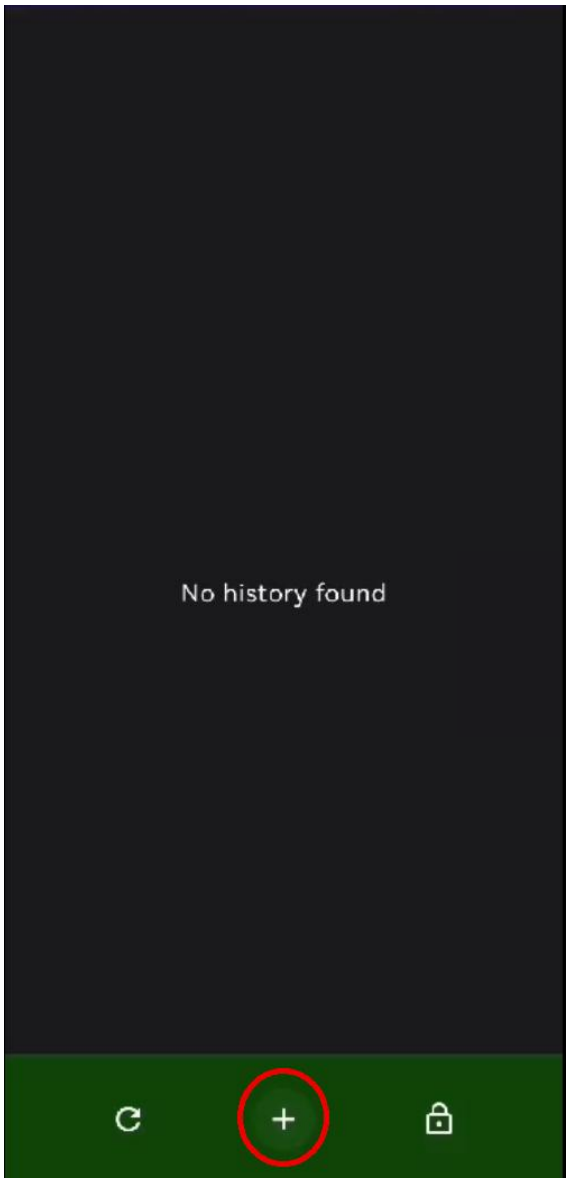


Figure 23: Homepage Highlighting the Button to Enable the QR Code Scanner

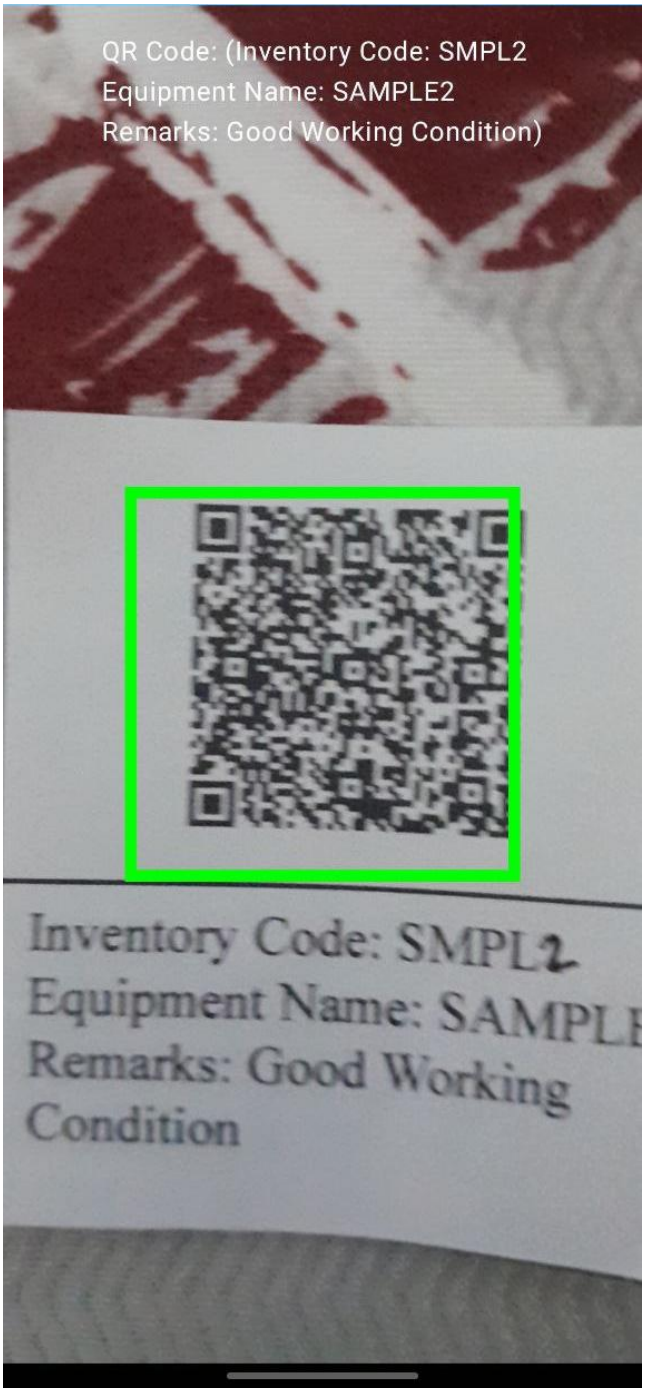


Figure 24: Functioning QR Code Scanner from the Application

VIII. USER INFORMATION FOR INVENTORY LOGGING

After successfully scanning the QR code a form will pop up, prompting the user to fill in the required information as listed which is shown in Figure 25. The user needs to fill up all fields (Student, Subject, and Purpose) before tapping “Borrow” as shown in Figure 26. Successful transaction will redirect back to the history page.

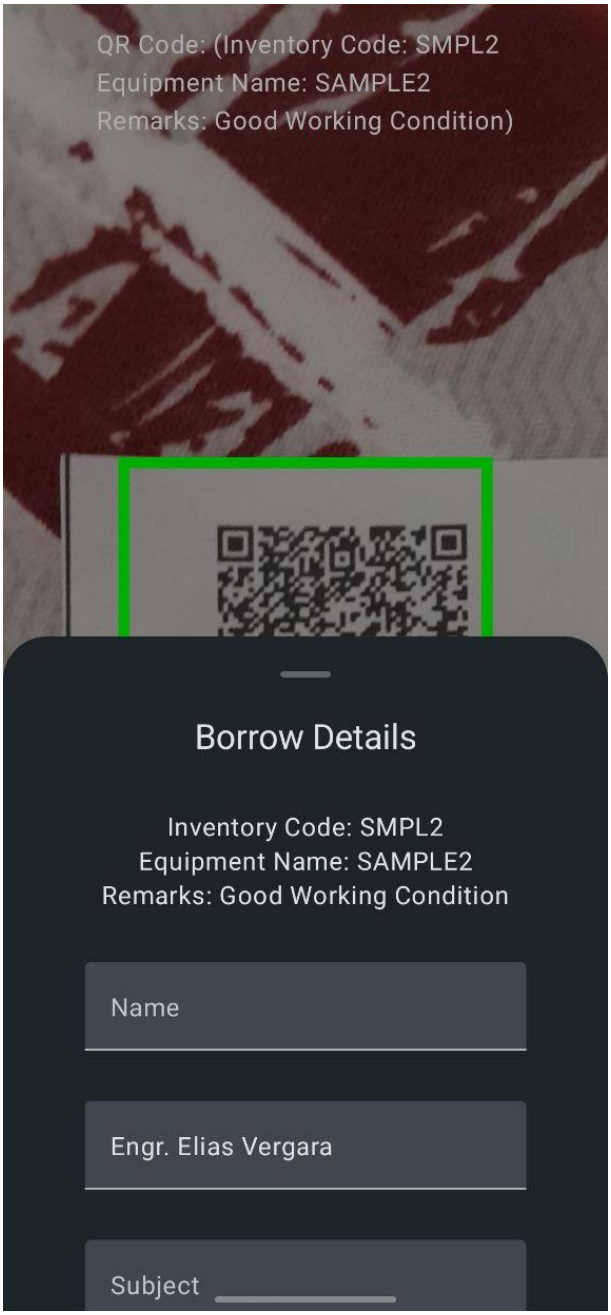


Figure 25: Information from the QR Code After the Successful Scan

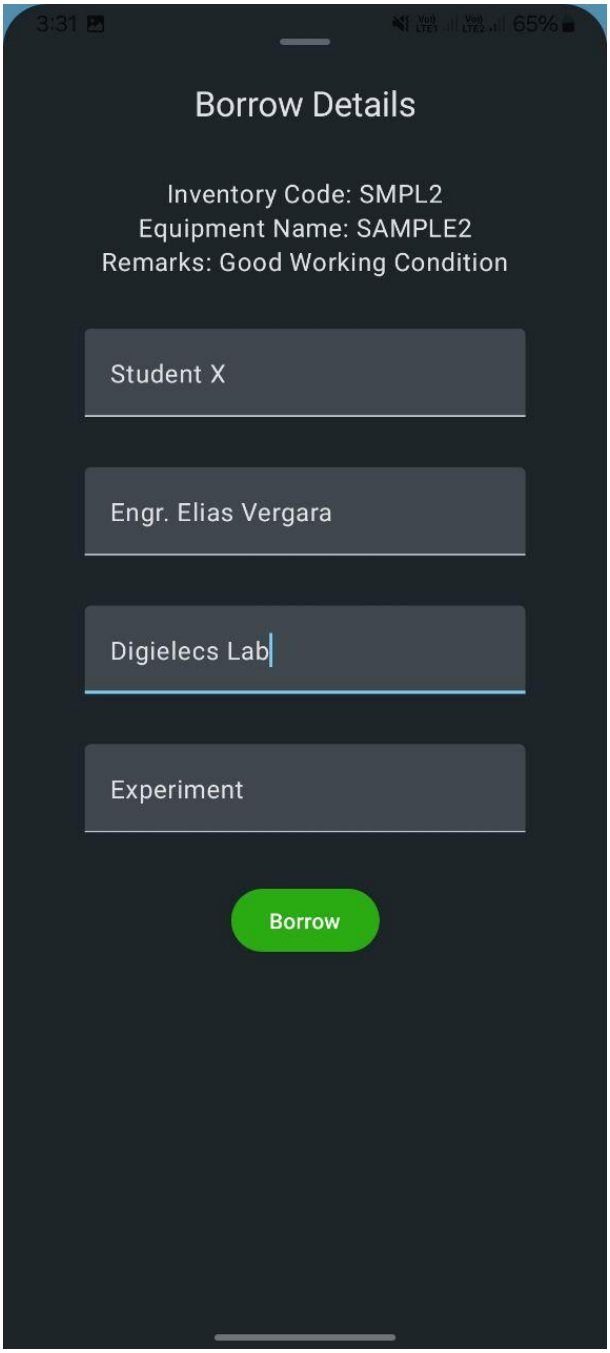


Figure 26: User Interface for the Information for the Borrower

IX. HOMEPAGE SHOWING THE UPDATED HISTORY AFTER BORROWING

The history page is now updated and filled with detail card of the new borrow transaction made as shown in Figure 27. All unreturned transactions are labeled as “UNRETURNED”. The user can tap the details card of a specific transaction to view more details as depicted in Figure 28.

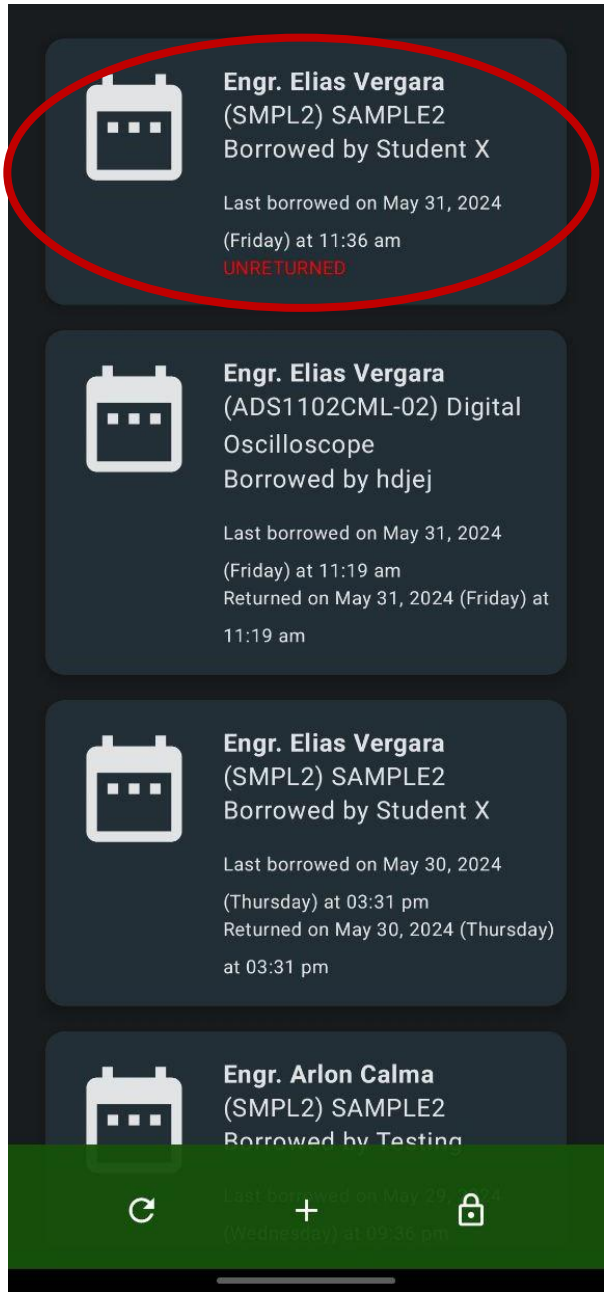


Figure 27: Homepage with the Updated History Log

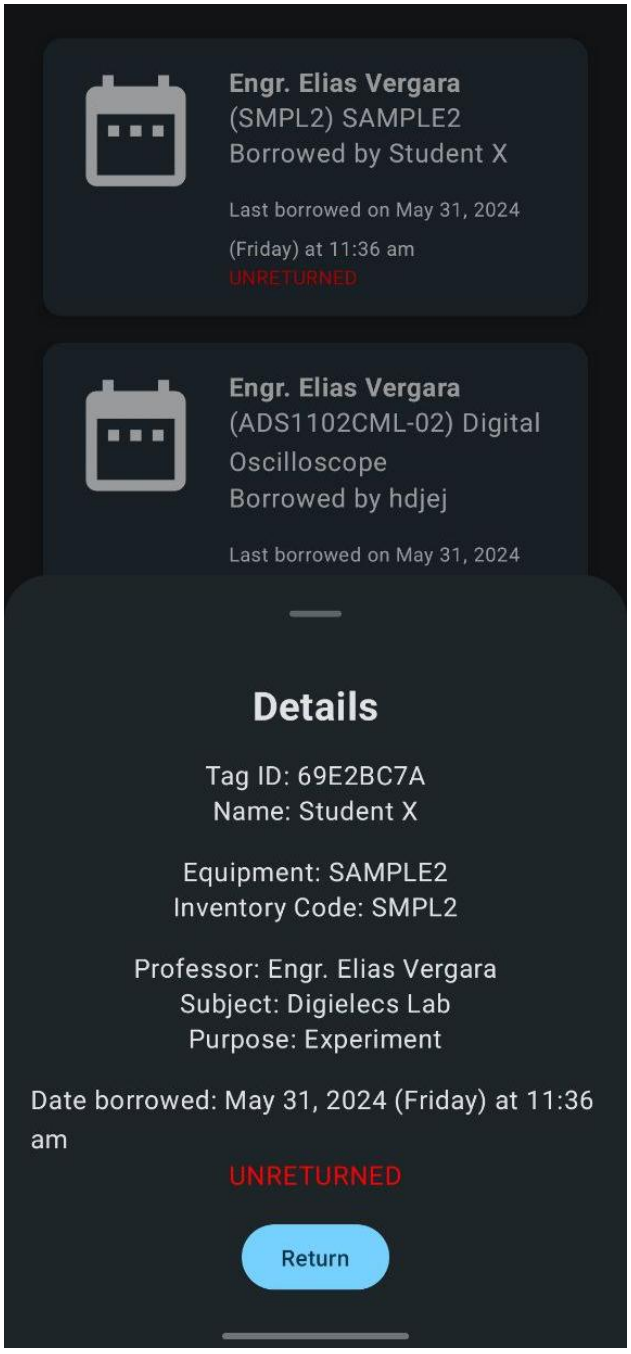


Figure 28: Details Card of the Transaction

X. RETURNING PROCESS

To start the returning process, the user needs to tap the details card of the specific laboratory equipment being returned and click the “Return” button as shown in Figure 29. The QR scanner will then be opened again upon clicking. The user will need to scan the QR code of the equipment as displayed in Figure 30. After scanning, the borrow details will be shown again, and click “Return” to finish the returning process as depicted in Figure 31. Upon completion, the application redirects again to the history page.

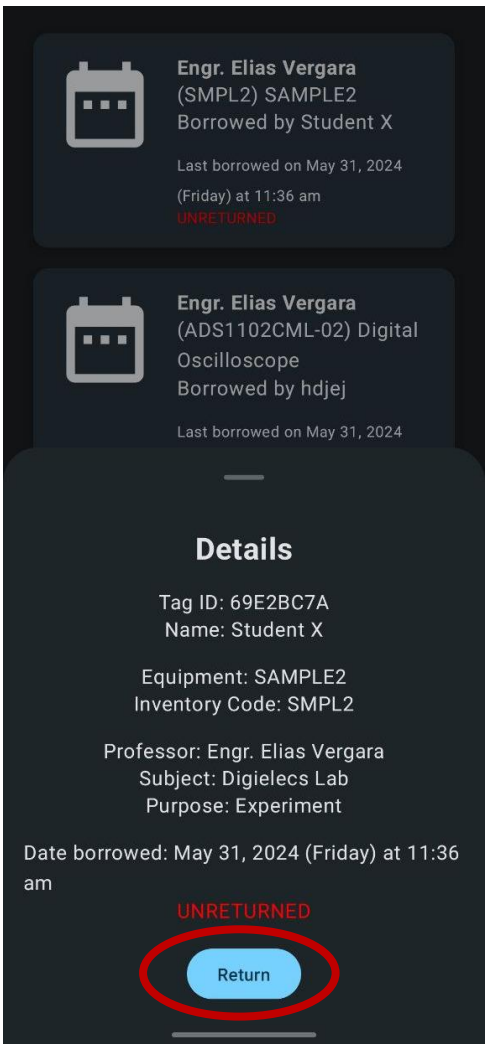


Figure 29: Details Card Highlighting the Return Button to Start Returning Process

Figure 30: QR Scanner in Returning for Verification

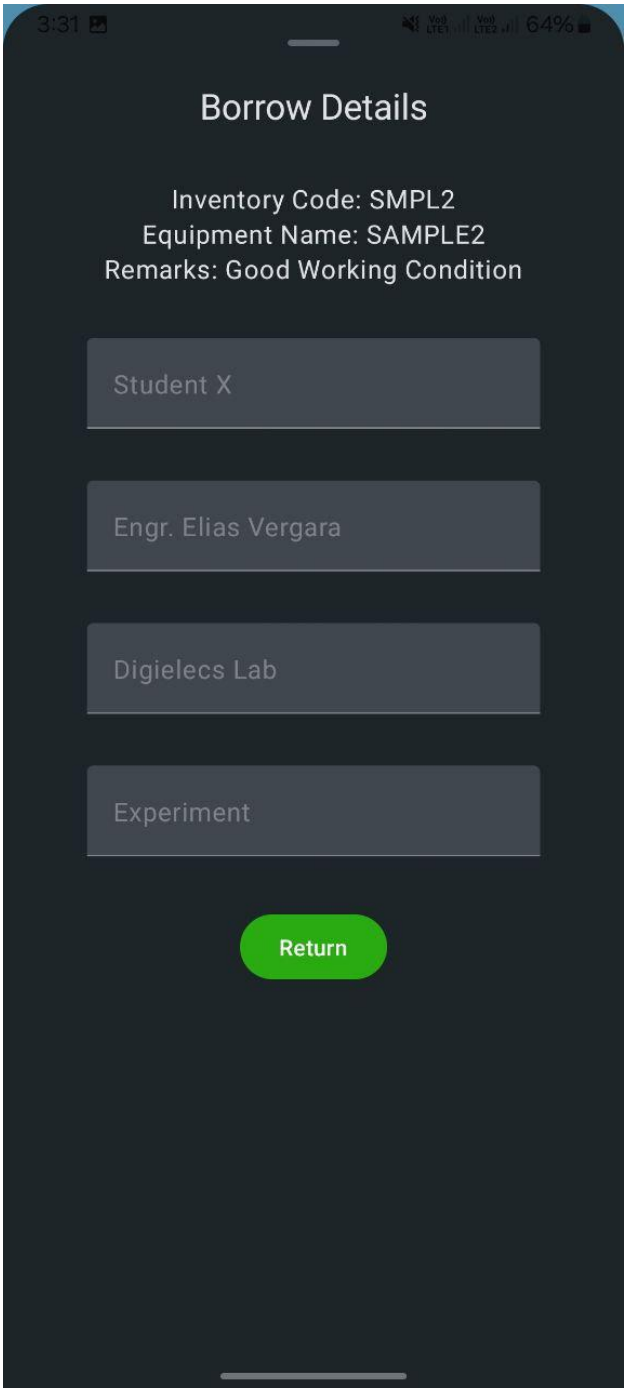


Figure 31: Last Step of Returning Process

XI. HOMEPAGE SHOWING THE UPDATED HISTORY AFTER RETURNING

The history is updated and the label “UNRETURNED” is removed from the details card as shown in Figure 32. The user can also tap on the details card for more detailed viewing of the transaction record as displayed in Figure 33.

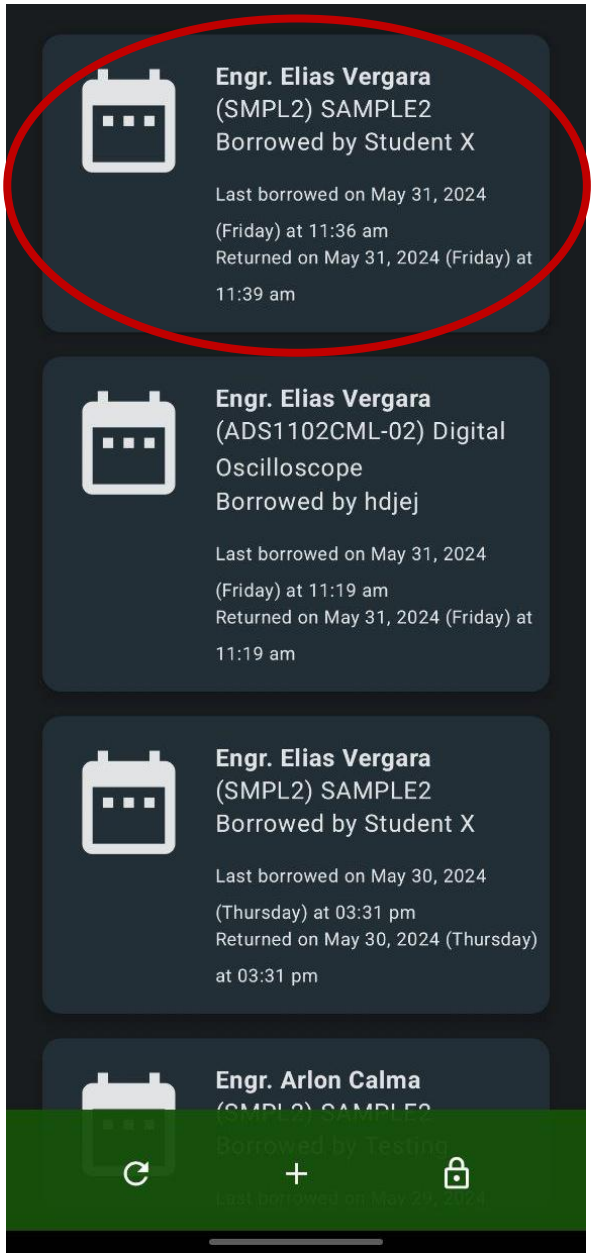


Figure 32: Updated History Page After Returning

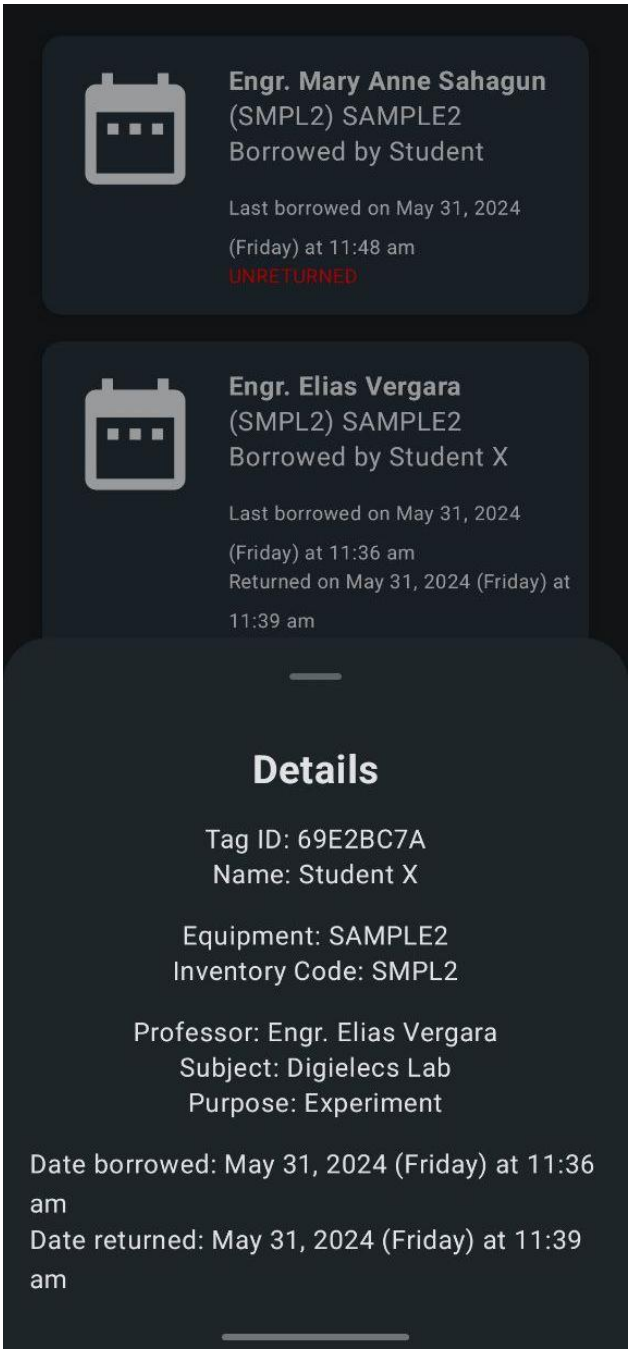


Figure 33: Updated Details Card After Returning

XII. OTHER INSTRUCTIONS

A. TIMEOUT

The hardware was set to initiate a timeout every 5 mins of inactivity. When the hardware reached this point, the user should just re-tap the RFID tag into the RFID scanner and this will solve the problem. Figure 34 shows what the screen looks like when the timeout was reached when borrowing and when viewing the history page.

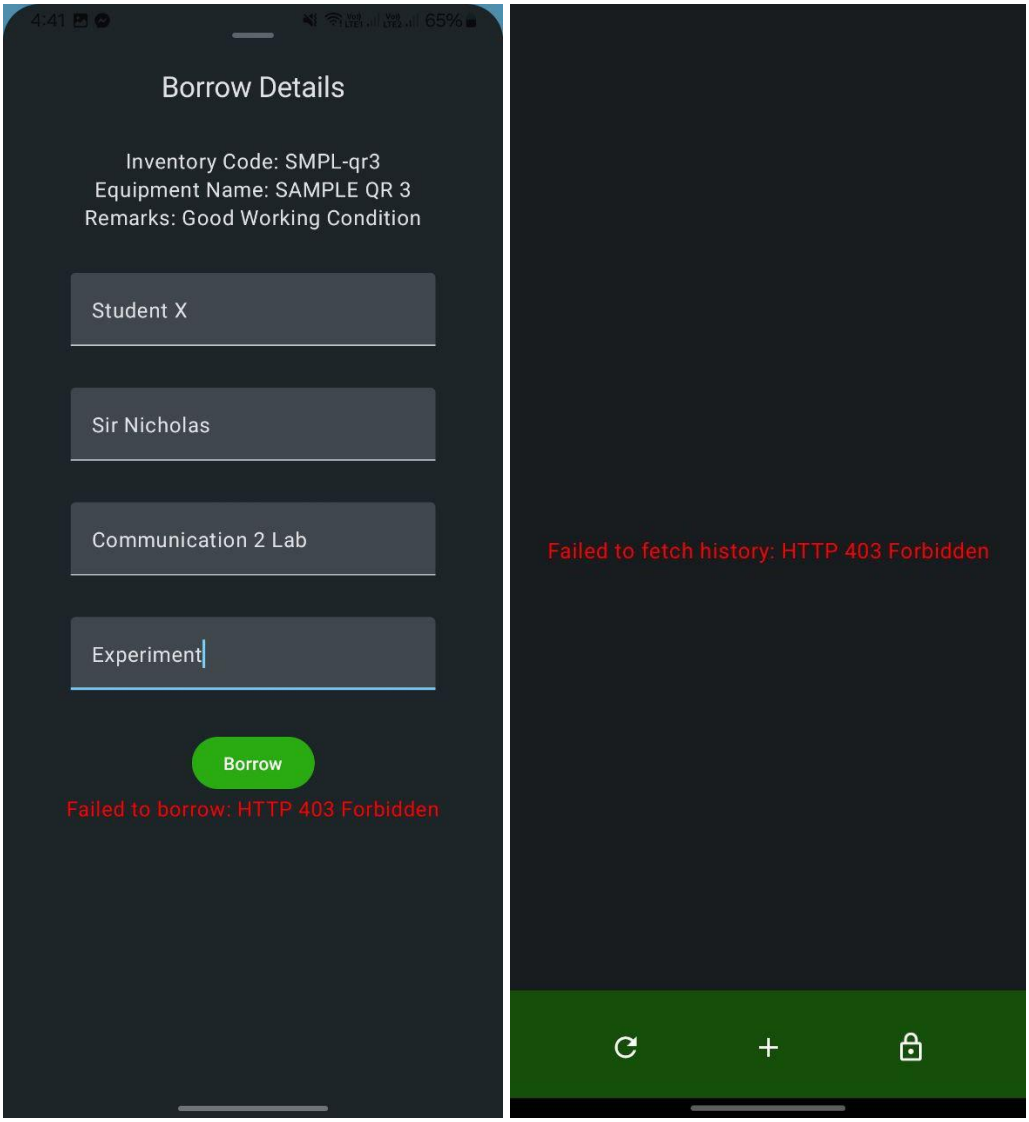


Figure 34: Timeout Within the Application

B. UNREGISTERED TAG

Unregistered RFID tags are not allowed to access the application as shown in Figure 35. The only solution for this is to tap a registered RFID tag.

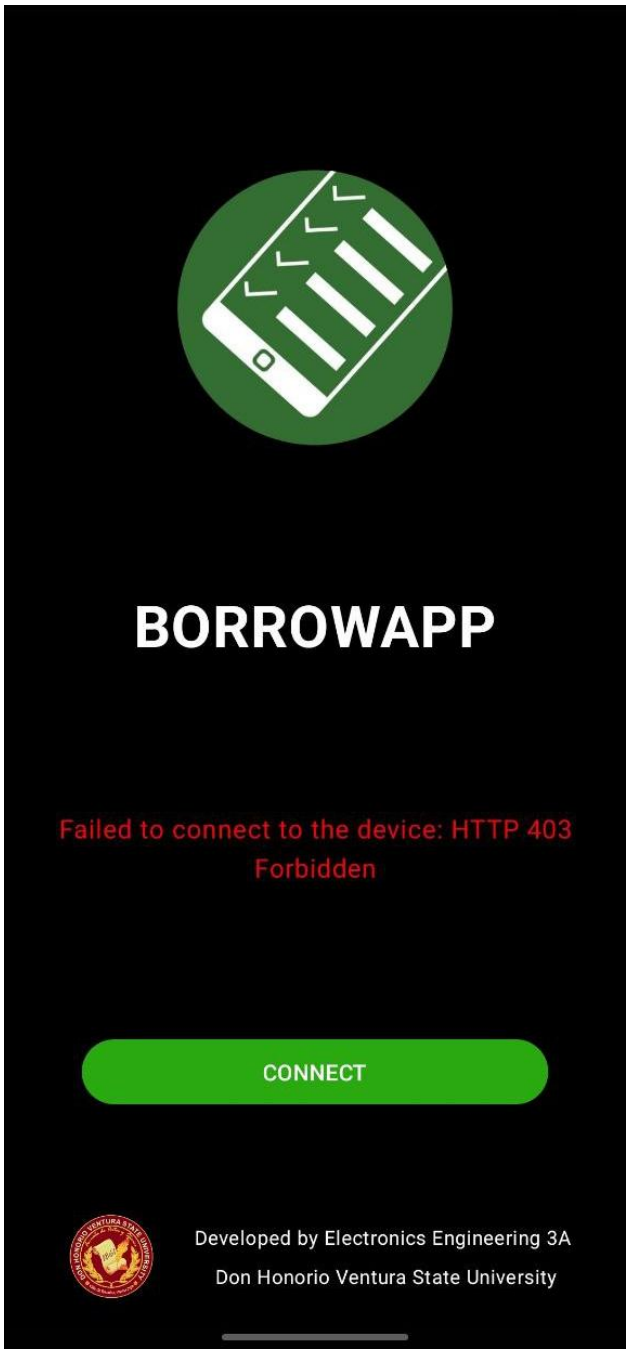


Figure 35: Forbidden Message for Unregistered RFID Tags

C. FILLING UP FIELDS WHEN BORROWING

The user should fill all fields for successful borrowing. If the “Borrow” button was tapped even the fields are not all filled, a message will show on the screen as displayed in Figure 36.

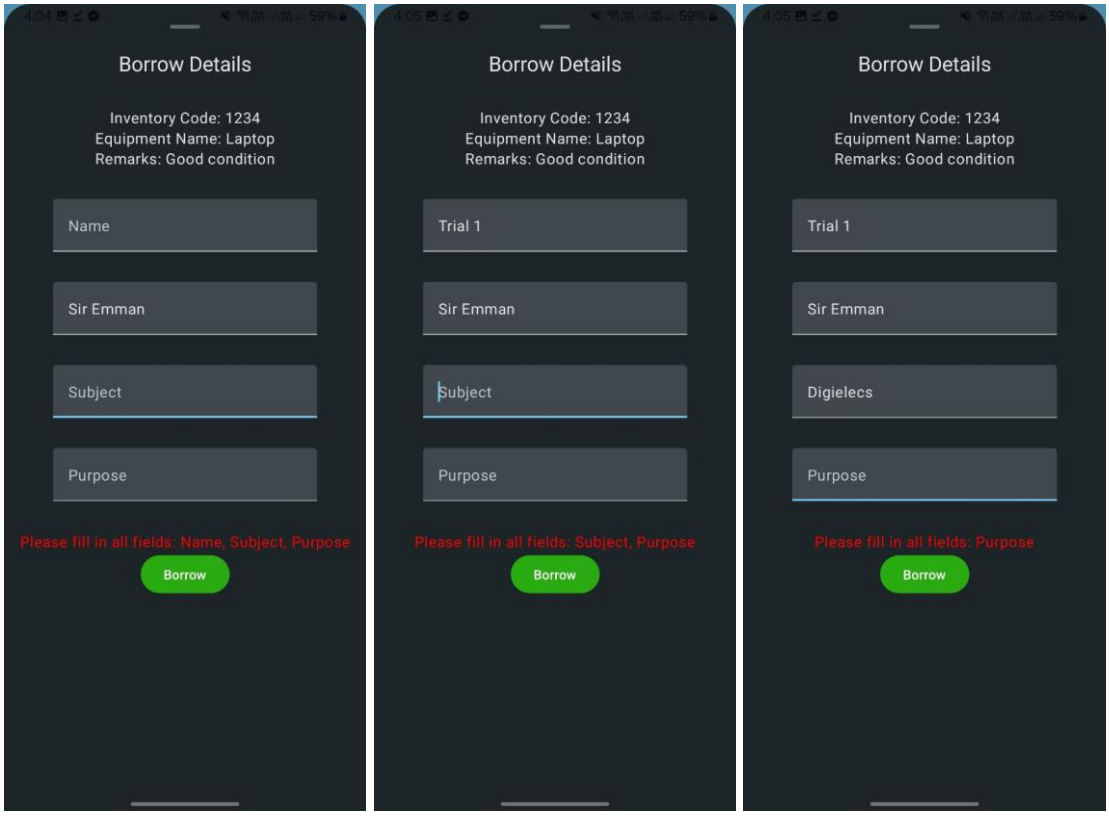


Figure 36: Message Requiring to Fill All Fields

D. QR SCANNING PROBLEMS

- BORROWING

Inventory Code, Equipment Name, and Remarks should be embedded on the QR code for the QR scanner to read it as a valid QR. If the 3 information are missing on the QR code, a message will show on the screen as displayed in Figure 37.

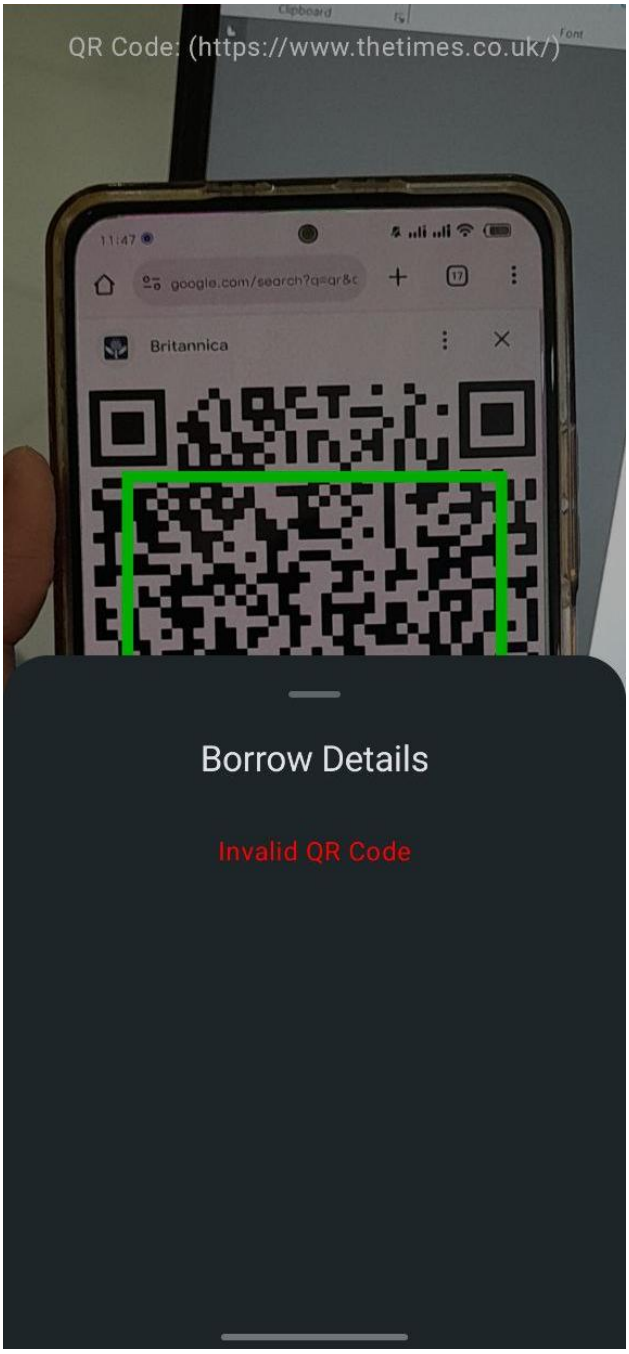


Figure 37: Message for Invalid QR Code When Borrowing

- RETURNING

The scanned QR code of the borrowed equipment should match the scanned QR code when it is returned for verification. If the QR codes do not match, a message will show on the screen as displayed in Figure 38.

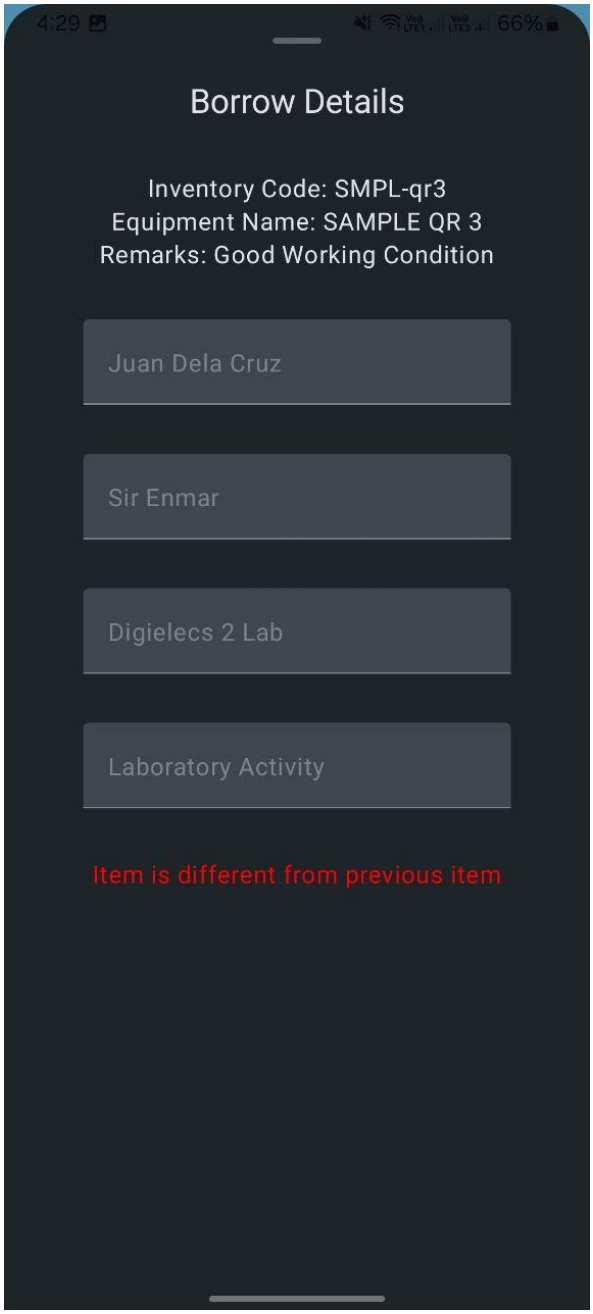


Figure 38: Message If QR Codes Do Not Match When Returning

APPENDIX C

DATA SHEETS

I. ESP32 WROOM 32

Table 1: ESP32-WROOM-32 (ESP-WROOM-32) Specifications

Categories	Items	Specifications
Certification	RF certification	FCC/CE/IC/TELEC/KCC/SRRC/NCC
	Wi-Fi certification	Wi-Fi Alliance
	Bluetooth certification	BQB
	Green certification	RoHS/REACH
Wi-Fi	Protocols	802.11 b/g/n (802.11n up to 150 Mbps)
		A-MPDU and A-MSDU aggregation and 0.4 μs guard interval support
	Frequency range	2.4 GHz ~ 2.5 GHz
Bluetooth	Protocols	Bluetooth v4.2 BR/EDR and BLE specification
	Radio	NZIF receiver with -97 dBm sensitivity
		Class-1, class-2 and class-3 transmitter
		AFH
	Audio	CVSD and SBC

Espressif Systems 1 ESP-WROOM-32 Datasheet V2.4

1. OVERVIEW

Categories	Items	Specifications
Hardware	Module interface	SD card, UART, SPI, SDIO, I2C, LED PWM, Motor PWM, I2S, IR
		GPIO, capacitive touch sensor, ADC, DAC
	On-chip sensor	Hall sensor, temperature sensor
	On-board clock	40 MHz crystal
	Operating voltage/Power supply	2.7 ~ 3.6V
	Operating current	Average: 80 mA
	Minimum current delivered by power supply	500 mA
	Operating temperature range	-40°C ~ +85°C
	Ambient temperature range	Normal temperature
	Package size	18±0.2 mm x 25.5±0.2 mm x 3.1±0.15 mm
Software	Wi-Fi mode	Station/SoftAP/SoftAP+Station/P2P
	Wi-Fi Security	WPA/WPA2/WPA2-Enterprise/WPS
	Encryption	AES/RSA/ECC/SHA
	Firmware upgrade	UART Download / OTA (download and write firmware via network or host)
	Software development	Supports Cloud Server Development / SDK for custom firmware development
	Network protocols	IPv4, IPv6, SSL, TCP/UDP/HTTP/FTP/MQTT
	User configuration	AT instruction set, cloud server, Android/iOS app

II. SD Card module

Features and Specifications of Micro SD Card Adapter Module

This section mentions some of the features and specifications of the Micro SD Card Adapter Module.

1. Operating Voltage: 4.5V - 5.5V DC
2. Current Requirement: 0.2-200 mA
3. 3.3 V on-board Voltage Regulator
4. Supports FAT file system
5. Supports micro SD up to 2GB
6. Supports Micro SDHC up to 32GB

Pin Configuration of Micro SD Card Adapter Module

The module contains 6 pins for power and communicating with the controller. The table below describes the pin type and role of each pin on the module.

Pin Type	Pin Description
GND	Ground
VCC	Voltage Input
MISO	Master In Slave Out(SPI)
MOSI	Master Out Slave In(SPI)
SCK	Serial Clock(SPI)
CS	Chip Select(SPI)

III. MFRC522 RFID reader

RC522 Pin Configuration

Pin Number	Pin Name	Description
1	Vcc	Used to Power the module, typically 3.3V is used
2	RST	Reset pin – used to reset or power down the module
3	Ground	Connected to Ground of system
4	IRQ	Interrupt pin – used to wake up the module when a device comes into range
5	MISO/SCL/Tx	MISO pin when used for SPI communication, acts as SCL for I2c and Tx for UART.
6	MOSI	Master out slave in pin for SPI communication
7	SCK	Serial Clock pin – used to provide clock source
8	SS/SDA/Rx	Acts as Serial input (SS) for SPI communication, SDA for IIC and Rx during UART

RC522 Features

- 13.56MHz RFID module
- Operating voltage: 2.5V to 3.3V
- Communication : SPI, I2C protocol, UART
- Maximum Data Rate: 10Mbps
- Read Range: 5cm
- Current Consumption: 13-26mA
- Power down mode consumption: 10uA (min)

APPENDIX D

SOURCE CODES FOR APPLICATION

APIService.kt

```kotlin
package com.example.borrowapp

import ...

private val retrofit =
    Retrofit.Builder() Retrofit.Builder
        .baseUrl("http://192.168.4.1/") Retrofit.Builder
//            .baseUrl("http://54.199.243.150/")
        .addConverterFactory(GsonConverterFactory.create()).build()

val apiService = retrofit.create(APIService::class.java)

interface APIService {
    @GET("authorize")
    suspend fun authorize(): AuthorizeResponse

    @GET("add/{timein}/{timeout}/{tagid}/{studentName}/{profName}/{subject}/{purpose}/{inventoryCode}/{equipmentName}/{remarks}")
    suspend fun add(
        @Path("timein") timeIn: String,
        @Path("timeout") timeOut: String,
        @Path("tagid") tagId: String,
        @Path("studentName") studentName: String,
        @Path("profName") profName: String,
        @Path("subject") subject: String,
        @Path("purpose") purpose: String,
        @Path("inventoryCode") inventoryCode: String,
        @Path("equipmentName") equipmentName: String,
        @Path("remarks") remarks: String
    ): DoneResponse

    @GET("history")
    suspend fun getHistory(): HistoryResponse

    @GET("done")
    suspend fun done(): DoneResponse
}
```

AppState.kt

```kotlin
package com.example.borrowapp

import java.io.Serializable

data class AppState(val currentID: String? = null,
                    val loading: Boolean = false,
                    val history: List<HistoryItem> = emptyList(),
                    val error: String? = null,
                    val historyError: String? = null,
                    var loaded: Boolean = false,
                    val connected: Boolean = false)

// API Responses
data class AuthorizeResponse(val id: String? = null)

data class HistoryResponse(val history: String? = null)


data class DoneResponse(val message: String? = null)


data class HistoryItem(val tagId: String = "",
                       val date: String = "",
                       val dateOut: String = "",
                       val name: String = "",
                       val professor: String = "",
                       val purpose: String = "",
                       val subject: String = "",
                       val inventoryCode: String = "",
                       val equipmentName: String = "",
                       val remarks: String = ""): Serializable
```

BottomBar.kt

```kotlin
package com.example.borrowapp

import ...


@Preview(showBackground = true)
@Composable
fun BottomBarPreview() {
    BottomBar({}, {}, navController: null)
}

@Composable
fun BottomBar(onLogout: () -> Unit, onRefresh: () -> Unit, navController: NavHostController?) {
    val context = LocalContext.current

    Row(
        modifier = Modifier
            .fillMaxWidth()
            .fillMaxHeight(),
        horizontalArrangement = Arrangement.SpaceEvenly,
        verticalAlignment = Alignment.CenterVertically
    ) { this: RowScope
//        IconButton(onClick = {
//            navController?.navigate("history_screen")
//        }) {
//            Icon(imageVector = Icons.Default.List,
//                 contentDescription = "History",
//                 tint = Color.White)
//
//        }
        // icon button for refresh
        IconButton(onClick = { onRefresh() }) {
            Icon(
                imageVector = Icons.Default.Refresh,
                contentDescription = "Refresh",
                tint = Color.White
            )
        }
        IconButton(onClick = {
            val cameraIntent = Intent(context, Camera::class.java)
            startActivity(context, cameraIntent, options: null)
        }) {
            Icon(imageVector = Icons.Default.Add, contentDescription = "Borrow", tint = Color.White)
        }

        IconButton(onClick = { onLogout() }) {
            Icon(
                imageVector = Icons.Outlined.Lock,
                contentDescription = "Refresh",
                tint = Color.White
            )
        }
    }
}
```

Camera.kt

```
package com.example.borrowapp

import ...

class Camera : AppCompatActivity() {
    @OptIn(ExperimentalMaterial3Api::class)
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val options = BarcodeScannerOptions.Builder()
            .setBarcodeFormats(Barcode.FORMAT_QR_CODE, Barcode.FORMAT_AZTEC)
            .enableAllPotentialBarcodes().build()

        if (!hasRequiredPermissions()) {
            ActivityCompat.requestPermissions( activity: this, CAMERAX_PERMISSIONS,  requestCode: 0)
        }
        setContent {
            BorrowAppTheme { // A surface container using the 'background' color from the theme
                Surface(
                    modifier = Modifier.fillMaxSize(), color = MaterialTheme.colorScheme.background
                ) {
                    val cameraViewModel: CameraViewModel = viewModel()

                    val controller = remember {
                        LifecycleCameraController(applicationContext).apply { this: LifecycleCameraController
                            setEnabledUseCases(CameraController.IMAGE_CAPTURE or CameraController.VIDEO_CAPTURE)
                        }
                    }
                    Text(text = "Camera Screen", color = Color.Black)
                    Box(modifier = Modifier.fillMaxSize()) { this: BoxScope
                        CameraPreview(controller = controller, modifier = Modifier.fillMaxSize())
                        Text(
                            text = "QR Code: (${cameraViewModel.message.value})",
                            modifier = Modifier
                                .align(Alignment.TopCenter)
                                .padding(16.dp),
                            color = Color.White
                        )
                        val sheetState = rememberModalBottomSheetState()
                        val scope = rememberCoroutineScope()
                        var showBottomSheet by remember { mutableStateOf( value: false) }

                        if (showBottomSheet) {
                            QRForm(
                                sheetState = sheetState,
                                data = cameraViewModel.message.value,
                                onDismissRequest = {
                                    showBottomSheet = false
                                    cameraViewModel.currentItem.value = HistoryItem()
                                    cameraViewModel.successfulMessage.value = ""
                                },
                                cameraViewModel = cameraViewModel
                            )
                        }
                        Row(
                            modifier = Modifier
                                .fillMaxWidth()
                                .align(Alignment.BottomCenter)
                                .padding(16.dp), horizontalArrangement = Arrangement.SpaceAround
                        ) { this: RowScope

                            IconButton(onClick = {
                                takePhoto(controller = controller,
                                    onPhotoTaken = cameraViewModel::onTakePhoto,
                                    showSheetForm = { showBottomSheet = true })
                            }) {
                                Icon(
                                    imageVector = Icons.Default.AddCircle,
                                    contentDescription = "Take photo"
                                )
                            }
                        }
                    }
                }
            }
        }

    }

    private fun hasRequiredPermissions(): Boolean {
        return CAMERAX_PERMISSIONS.all { it: String
            ContextCompat.checkSelfPermission(
                applicationContext, it
            ) == PackageManager.PERMISSION_GRANTED
        }
    }

    private fun takePhoto(
        controller: LifecycleCameraController,
        onPhotoTaken: (Bitmap) -> Unit,
        showSheetForm: () -> Unit
    ) {
        controller.takePicture(ContextCompat.getMainExecutor(applicationContext),
            object : ImageCapture.OnImageCapturedCallback() {
                override fun onCaptureSuccess(image: ImageProxy) {
                    super.onCaptureSuccess(image)
                    onPhotoTaken(image.toBitmap())
                    image.close()
                    showSheetForm()
                }
```

```
                override fun onError(exception: ImageCaptureException) {
                    super.onError(exception)
                    Log.e( tag: "Camera",  msg: "Couldn't take photo: ", exception)
                }
            })
    }

    companion object {
        private val CAMERAX_PERMISSIONS = arrayOf(
            Manifest.permission.CAMERA,
            Manifest.permission.RECORD_AUDIO,
        )
    }

//    private fun closeIntent(item: HistoryItem) {
//        val intent = Intent()
//        intent.putExtra("item", item)
//        setResult(RESULT_OK, intent)
//        finish()
//    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CameraPreview(controller: LifecycleCameraController, modifier: Modifier = Modifier) {
    val lifecycleOwner = LocalLifecycleOwner.current


    AndroidView(factory = { it: Context
        PreviewView(it).apply { this: PreviewView
            this.controller = controller
            controller.bindToLifecycle(lifecycleOwner)
        }
    }, modifier = modifier)
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun QRForm(
    sheetState: SheetState,
    data: String,
    onDismissRequest: () -> Unit,
    cameraViewModel: CameraViewModel
) {
    val context = LocalContext.current
    val item = remember {
        cameraViewModel.currentItem
    }
    val loading = remember {
        cameraViewModel.loading
    }
    val error = remember {
        cameraViewModel.error
    }
    val successfulMessage = remember {
        cameraViewModel.successfulMessage
    }
    val now = Instant.now()
    val after3Hours = now.plusSeconds( secondsToAdd: 3 * 60 * 60)

    val formattedNow = formatter.format(now)
    val formattedAfter3Hours = formatter.format(after3Hours)
    val parsedData: Map<String, String>

    val disabledButton =
        item.value.name.isEmpty() || item.value.professor.isEmpty() || item.value.subject.isEmpty() || item.value.purpose.isEmpty()
    try {
        parsedData = parseInventoryString(data)
//        parsedData = mapOf(
//            "Inventory Code" to "1234", "Equipment Name" to "Laptop", "Remarks" to "Good condition"
//        )
    } catch (e: IllegalArgumentException) {
        ModalBottomSheet(onDismissRequest = onDismissRequest, sheetState = sheetState) { this: ColumnScope
            Column(
                modifier = Modifier.fillMaxSize(),
                verticalArrangement = Arrangement.Top,
                horizontalAlignment = Alignment.CenterHorizontally
            ) { this: ColumnScope

                Text(text = "Borrow Details", style = MaterialTheme.typography.titleLarge)
                Spacer(modifier = Modifier.padding(16.dp))

                Text(text = "Invalid QR Code", color = Color.Red)

            }
        }
        return
    }
```

```
248
249         ModalBottomSheet(
250             onDismissRequest = onDismissRequest,
251             sheetState = sheetState,
252         ) { this: ColumnScope  // Sheet content
253             Column(
254                 modifier = Modifier.fillMaxSize(),
255                 verticalArrangement = Arrangement.Top,
256                 horizontalAlignment = Alignment.CenterHorizontally
257             ) { this: ColumnScope
258
259                 item.value = item.value.copy(
260                     inventoryCode = parsedData["Inventory Code"]!!,
261                     equipmentName = parsedData["Equipment Name"]!!,
262                     remarks = parsedData["Remarks"]!!,
263                     date = now.toString(),
264                     dateOut = after3Hours.toString()
265                 )
266
267                 Text(text = "Borrow Details", style = MaterialTheme.typography.titleLarge)
268                 Spacer(modifier = Modifier.padding(16.dp))
269
270                 Text(text = "Inventory Code: ${parsedData["Inventory Code"]}")
271                 Text(text = "Equipment Name: ${parsedData["Equipment Name"]}")
272                 Text(text = "Remarks: ${parsedData["Remarks"]}")
273                 Spacer(modifier = Modifier.padding(16.dp))
274
275                 TextField(value = item.value.name, onValueChange = { it: String
276                     item.value = item.value.copy(name = it)
277                 }, placeholder = {
278                     Text(text = "Name")
279                 })
280                 Spacer(modifier = Modifier.padding(16.dp))
281
282                 TextField(value = item.value.professor, onValueChange = { it: String
283                     item.value = item.value.copy(professor = it)
284                 }, placeholder = {
285                     Text(text = "Professor")
286                 }) // TextField for subject
287                 Spacer(modifier = Modifier.padding(16.dp))
288
289                 TextField(value = item.value.subject, onValueChange = { it: String
290                     item.value = item.value.copy(subject = it)
291                 }, placeholder = {
292                     Text(text = "Subject")
293                 })
294                 Spacer(modifier = Modifier.padding(16.dp)) // TextField for purpose
295                 TextField(value = item.value.purpose, onValueChange = { it: String
296                     item.value = item.value.copy(purpose = it)
297                 }, placeholder = {
298                     Text(text = "Purpose")
299                 })
300                 Spacer(modifier = Modifier.padding(16.dp)) // TextField for purpose
301
302                 when {
303                     loading.value -> {
304                         CircularProgressIndicator()
305                     }
306
307                     else -> {
308                         Button(enabled = !disabledButton,
309                             colors = ButtonDefaults.buttonColors(containerColor = Color( color: 0xF32CB111)),
310                             onClick = {
311 //                                (context as Activity).finish()
312                                 cameraViewModel.addToLog(item.value)
313                             }) { this: RowScope
314                             Text(text = "Borrow", color = Color.White)
315                         }
316                         when {
317                             error.value != null -> {
318                                 Text(text = "Failed to borrow: ${error.value}", color = Color.Red)
319                             }
320
321                             successfulMessage.value.isNotEmpty() -> {
322                                 Text(
323                                     text = "${successfulMessage.value}",
324                                     modifier = Modifier.align(Alignment.CenterHorizontally).padding(16.dp)
325                                 )
326                             }
327                         }
328                     }
329                 }
330             }
331         }
332
333     }
334
335     fun parseInventoryString(input: String): Map<String, String> {
336         val keys = listOf("Inventory Code", "Equipment Name", "Remarks")
337         val regex = Regex( pattern: "(Inventory Code|Equipment Name|Remarks): (.*)")
338         val matches = regex.findAll(input)
339
340         val map = matches.associate { matchResult ->
341             val (key, value) = matchResult.destructured
342             key to value.trim() "associate
343         }
344
345         for (key in keys) {
346             if (!map.containsKey(key)) {
347                 throw IllegalArgumentException("Missing key: $key")
348             }
349         }
```

CameraViewModel.kt

```kotlin
package com.example.borrowapp

import ...

class CameraViewModel : ViewModel() {
    private val _bitmaps = MutableStateFlow<List<Bitmap>>(emptyList())
    val bitmaps = _bitmaps.asStateFlow()

    private val scanner = BarcodeScanning.getClient()

    private val _message = mutableStateOf( value: "No results yet")
    val message = _message

    private val _scannedSize = mutableIntStateOf( value: 0)
    val scannedSize = _scannedSize

    private val _currentItem = mutableStateOf(HistoryItem())
    val currentItem = _currentItem

    private val _loading = mutableStateOf( value: false)
    val loading = _loading

    private val _error = mutableStateOf<String?>( value: null)
    val error = _error

    private val _successfulMessage = mutableStateOf( value: "")
    val successfulMessage = _successfulMessage


    fun onTakePhoto(bitmap: Bitmap) {
        Log.d( tag: "YEL",  msg: "onTakePhoto")
        val inputImage = InputImage.fromBitmap(bitmap,  rotationDegrees: 0)

        val result = scanner.process(inputImage).addOnSuccessListener { barcodes ->
            println("YEL-SUCCESS")
            Log.d( tag: "YEL",  msg: "${barcodes.size}")
            _scannedSize.intValue = barcodes.size
            _message.value = "No results yet"

            for (barcode in barcodes) {
                Log.d( tag: "YEL",  msg: "SUCCESS")
                Log.d( tag: "QRScanner",  msg: "Barcode: ${barcode.displayValue}")
                _message.value = "${barcode.displayValue}"
                Log.d( tag: "YEL",  msg: "${barcode.displayValue}")
            }
        }.addOnFailureListener { it: Exception
            _scannedSize.value = 0
            _message.value = "Failed to process image"
            println("YEL-FAIL")

            Log.e( tag: "YEL",  msg: "ERROR")

            Log.e( tag: "QRScanner",  msg: "Failed to process image", it)
        }
    }
```

```
68      fun addToLog(item: HistoryItem) {
69          _currentItem.value = item
70          _loading.value = true
71          _successfulMessage.value = ""
72          viewModelScope.launch { this: CoroutineScope
73              try {
74                  val response = apiService.add(
75                      item.date,
76                      item.dateOut,
77                      item.tagId,
78                      item.name,
79                      item.professor,
80                      item.subject,
81                      item.purpose,
82                      item.inventoryCode,
83                      item.equipmentName,
84                      item.remarks
85                  )
86                  _loading.value = false
87                  _error.value = null
88                  _successfulMessage.value =
89                      "SUCCESS: ${item.equipmentName} - ${item.inventoryCode} borrowed by ${item.name}"
90              } catch (e: Exception) {
91                  _loading.value = false
92                  _error.value = e.message
93                  println("YEL: Error is ${e.message}")
94              }
95          }
96
97      }
98  }
```

ConnectScreen.kt

```kotlin
package com.example.borrowapp


import ...

@Composable
fun MainScreen(appViewModel: MainViewModel = viewModel()) {
    val appState by appViewModel.appState
    val navController = rememberNavController()


    when {
        appState.connected -> {
            MainScaffold(
                viewModel = appViewModel, appState = appState, navController = navController
            )
        }

        else -> {
            Box(
                modifier = Modifier
                    .fillMaxSize()
                    .background(color = Color.Black)
            ) { this: BoxScope
                ConnectScreen(appState = appState, onAuthorize = { appViewModel.authorize() })
            }
        }
    }
}


@SuppressLint("UnusedMaterial3ScaffoldPaddingParameter")
@Composable
fun MainScaffold(viewModel: MainViewModel, appState: AppState, navController: NavHostController) {
    Scaffold(bottomBar = {
        BottomAppBar(containerColor = Color( color: 0xF3155209)) { this: RowScope
            BottomBar(
                { viewModel.logout() },
                { viewModel.fetchHistory() },
                navController = navController
            )
        }
    }) { it: PaddingValues
        Box { this: BoxScope
            NavHost(navController = navController, startDestination = "history_screen") { this: NavGraphBuilder
                composable(route = "history_screen") { this: AnimatedContentScope  it: NavBackStackEntry
                    BackHandler( enabled: true) {}
                    LaunchedEffect(key1 = "history_screen") { this: CoroutineScope
                        viewModel.fetchHistory()
                    }
                    HistoryScreen(appState = appState)
                }
                composable(route = "borrow_screen") { this: AnimatedContentScope  it: NavBackStackEntry
                    BackHandler( enabled: true) {}
                    Text(text = "This is for borrow screen")
                }
            }
        }
```

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HistoryScreen(appState: AppState) {
    val sheetState = rememberModalBottomSheetState()
    val scope = rememberCoroutineScope()
    var showBottomSheet by remember { mutableStateOf( value: false) }
    val currentlySelectedItem = remember { mutableStateOf<HistoryItem?>( value: null) }

    if (showBottomSheet && currentlySelectedItem.value != null) {
        ModalBottomSheet(
            onDismissRequest = {
                showBottomSheet = false
            }, sheetState = sheetState
        ) { this: ColumnScope
            // Sheet content
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(16.dp),
                horizontalAlignment = Alignment.CenterHorizontally
            ) { this: ColumnScope
                Text(
                    text = "Details", fontWeight = FontWeight.Bold, fontSize = 24.sp
                )
                Spacer(modifier = Modifier.height(16.dp))
                Text(
                    text = "Tag ID: ${currentlySelectedItem.value?.tagId}", fontSize = 16.sp
                )
                Text(text = "Name: ${currentlySelectedItem.value?.name}", fontSize = 16.sp)
                Spacer(modifier = Modifier.height(16.dp))

                Text(
                    text = "Equipment: ${currentlySelectedItem.value?.equipmentName}",
                    fontSize = 16.sp
                )
                Text(
                    text = "Inventory Code: ${currentlySelectedItem.value?.inventoryCode}",
                    fontSize = 16.sp
                )

                Spacer(modifier = Modifier.height(16.dp))

                Text(
                    text = "Professor: ${currentlySelectedItem.value?.professor}", fontSize = 16.sp
                )
                Text(text = "Subject: ${currentlySelectedItem.value?.subject}", fontSize = 16.sp)
                Text(text = "Purpose: ${currentlySelectedItem.value?.purpose}", fontSize = 16.sp)

                Spacer(modifier = Modifier.height(16.dp))

                val formattedIn = formatter.format(Instant.parse(currentlySelectedItem.value?.date))
                val formattedOut = formatter.format(Instant.parse(currentlySelectedItem.value?.dateOut))
                Text(text = "Date borrowed: ${formattedIn}", fontSize = 16.sp)
                Text(
                    text = "Date to be returned: ${formattedOut}",
                    fontSize = 16.sp
                )
```

```
174
175                 }
176
177             }
178         }
179         Column(
180             modifier = Modifier
181                 .fillMaxSize()
182                 .padding(16.dp),
183             horizontalAlignment = Alignment.CenterHorizontally,
184             verticalArrangement = Arrangement.Center
185         ) { this: ColumnScope
186             when {
187                 appState.loading -> {
188                     CircularProgressIndicator(color = Color.Green)
189                     Spacer(modifier = Modifier.height(32.dp))
190                     Text(text = "Fetching history. Please, wait...", color = Color.White)
191                 }
192
193                 appState.historyError != null && !appState.loading -> {
194                     Text(
195                         textAlign = TextAlign.Center,
196                         text = "Failed to fetch history: ${appState.error}",
197                         color = Color.Red
198                     )
199                 }
200
201                 appState.history.isEmpty() -> {
202                     Text(text = "No history found", color = Color.White)
203                 }
204
205                 else -> {
206                     LazyVerticalGrid(columns = GridCells.Fixed( count: 1)) { this: LazyGridScope
207                         items<HistoryItem>(items = appState.history) { this: LazyGridItemScope  item ->
208                             HistoryCard(item = item, onClick = {
209                                 currentlySelectedItem.value = item
210                                 showBottomSheet = true
211                             })
212                         }
213                     }
214                 }
215             }
216         }
217
218     }
219
220     @OptIn(ExperimentalMaterial3Api::class)
221     @Composable
222     fun HistoryCard(item: HistoryItem, onClick: () -> Unit = {}) {
223         // convert item.date string to a Java 8 date-time object
224         val dateIn = Instant.parse(item.date)
225         val dateOut = Instant.parse(item.dateOut)
226
227         val formattedIn = formatter.format(dateIn)
228         val formattedOut = formatter.format(dateOut)
229         ElevatedCard(
230             elevation = CardDefaults.cardElevation(
```

```
                defaultElevation = 6.dp
            ), onClick = { onClick() }, modifier = Modifier
                .fillMaxWidth()
                .padding(8.dp)
        ) { this: ColumnScope
            Row(horizontalArrangement = Arrangement.SpaceEvenly) { this: RowScope
                Column(

                    horizontalAlignment = Alignment.CenterHorizontally,
                    verticalArrangement = Arrangement.Center
                ) { this: ColumnScope
                    Icon(
                        imageVector = Icons.Default.DateRange,
                        contentDescription = "Log",
                        modifier = Modifier
                            .size(114.dp, 114.dp)
                            .padding(16.dp)
                            .fillMaxSize()
                    )
                }
                Column(modifier = Modifier.padding(vertical = 16.dp)) { this: ColumnScope
                    Text(
                        text = item.tagId,
                        fontWeight = FontWeight.Bold,
                    )
                    Text(text = "(${item.inventoryCode}) ${item.equipmentName}")
                    Text(text = "Borrowed by ${item.name}")
                    Spacer(modifier = Modifier.height(16.dp))

                    Text(text = "Last borrowed on ${formattedIn}", fontSize = 12.sp)
                    Text(text = "To be returned by ${formattedOut}", fontSize = 12.sp)

                }
            }
        }
    }

    @Preview(showBackground = false)
    @Composable
    fun HistoryCardPreview() {
        HistoryCard(
            HistoryItem(
                tagId = "123456", date = "2022-01-01T08:00:00Z", dateOut = "2022-01-01T08:00:00Z"
            )
        )
    }

    @Composable
    fun ConnectScreen(appState: AppState, onAuthorize: () -> Unit) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
```

```
284             horizontalAlignment = Alignment.CenterHorizontally,
285             verticalArrangement = Arrangement.Center,
286 
287             ) { this: ColumnScope
288             Column(
289                 modifier = Modifier.weight(1f),
290                 verticalArrangement = Arrangement.Bottom,
291                 horizontalAlignment = Alignment.CenterHorizontally
292             ) { this: ColumnScope
293                 Image(
294                     modifier = Modifier.weight(3f),
295                     painter = painterResource(id = R.drawable.appicon),
296                     contentDescription = "App Logo"
297                 )
298                 Text(
299                     text = "BORROWAPP",
300                     color = Color.White,
301                     fontSize = 36.sp,
302                     fontWeight = FontWeight.Bold
303                 )
304 
305 
306             }
307             Column(
308                 modifier = Modifier.weight(0.5f),
309                 verticalArrangement = Arrangement.Center,
310                 horizontalAlignment = Alignment.CenterHorizontally
311             ) { this: ColumnScope
312                 Spacer(modifier = Modifier.height(32.dp))
313 
314                 when {
315                     appState.loading -> {
316                         CircularProgressIndicator(color = Color.Green)
317                         Spacer(modifier = Modifier.height(32.dp))
318 
319                         Text(text = "Connecting to the device. Please, wait...", color = Color.White)
320                     }
321 
322                     appState.error != null && !appState.loading -> {
323                         Text(
324                             textAlign = TextAlign.Center,
325                             text = "Failed to connect to the device: ${appState.error}",
326                             color = Color.Red
327                         )
328                     }
329                 }
330             }
331             Column(
332                 modifier = Modifier.weight(0.5f),
333                 verticalArrangement = Arrangement.Bottom,
334                 horizontalAlignment = Alignment.CenterHorizontally
335             ) { this: ColumnScope
336                 when {
337                     appState.connected -> {
338                         Text(
339                             text = "Successfully connected to the device with tag ID: ${appState.currentID}",
340                             textAlign = TextAlign.Center,
341                             color = Color.White
```

```
344
345
346                 }
347                 Button(modifier = Modifier
348                     .fillMaxWidth()
349                     .padding(32.dp),
350                     colors = ButtonDefaults.buttonColors(containerColor = Color( color: 0xF32CB111)),
351                     onClick = { onAuthorize() }) { this: RowScope
352                     Text(text = "CONNECT", color = Color.White)
353                 }
354                 Spacer(modifier = Modifier.height(32.dp))
355                 Row(
356                     modifier = Modifier.fillMaxWidth(),
357                     horizontalArrangement = Arrangement.SpaceEvenly,
358                     verticalAlignment = Alignment.CenterVertically
359                 ) { this: RowScope
360                     Image(
361                         modifier = Modifier.size(72.dp),
362                         painter = painterResource(id = R.drawable.dhvsu),
363                         contentDescription = "App Logo"
364                     )
365                     Text(
366                         text = "Developed by Electronics Engineering 3A \n" + "Don Honorio Ventura State University",
367                         color = Color.White,
368                         fontSize = 12.sp,
369                         textAlign = TextAlign.Center
370                     )
371                 }
372
373
374             }
375         }
376     }
```

ConnectScreenVM.kt

```kotlin
package com.example.borrowapp


import ...


class MainViewModel : ViewModel() {
    private val _appState = mutableStateOf(AppState())
    val appState: State<AppState> = _appState

    fun authorize() {
//        _appState.value = _appState.value.copy(connected = true)
        viewModelScope.launch { this: CoroutineScope
            try {
                _appState.value = _appState.value.copy(loading = true)
                val response = apiService.authorize()
                _appState.value = _appState.value.copy(
                    connected = true,
                    loading = false,
                    loaded = true,
                    currentID = response.id,
                    error = null
                )
                println("YEL: Response is $response")
            } catch (e: Exception) {
                _appState.value = _appState.value.copy(
                    connected = false,
                    loaded = false,
                    loading = false,
                    error = e.message
                )
                println("YEL: Error is ${e.message}")

            }
        }
    }

    fun fetchHistory() {
        viewModelScope.launch { this: CoroutineScope
            try {
                _appState.value = _appState.value.copy(loading = true, loaded = false)
                val response = apiService.getHistory()
                val decoded = response.history?.let { decodeCSVLinesToHistoryItems(it) }
                _appState.value = _appState.value.copy(
                    loading = false,
                    history = decoded ?: emptyList(),
                    loaded = true,
                    error = null,
                    historyError = null
                )
                Log.d( tag: "YEL",  msg: "Found ${decoded?.size} items")
                for (item in decoded ?: emptyList()) {
                    Log.d( tag: "YEL",  msg: "Item: $item")
                }
```

```
            } catch (e: Exception) {
                _appState.value = _appState.value.copy(
                    loading = false,
                    loaded = false,
                    error = e.message,
                    historyError = e.message,
                )
                Log.e( tag: "YEL",  msg: "Error fetching history", e)
            }
        }
    }

    fun logout() {
        _appState.value = _appState.value.copy(loading = true)
        viewModelScope.launch { this: CoroutineScope
            try {
                val response = apiService.done()
                _appState.value = _appState.value.copy(
                    connected = false,
                    loading = false,
                    loaded = false,
                    currentID = null,
                    error = null,
                    history = emptyList(),
                )
                Log.d( tag: "YEL",  msg: "Logout response: $response")
            } catch (e: Exception) {
                _appState.value = _appState.value.copy(
                    connected = false,
                    loaded = false,
                    loading = false,
                    error = e.message
                )
                println("YEL: Error is ${e.message}")
                Log.e( tag: "YEL",  msg: "Error logging out", e)
            }
        }
    }
}
```

Decoder.kt

```kotlin
package com.example.borrowapp

import ...

val formatter: DateTimeFormatter =
    DateTimeFormatter.ofPattern( pattern: "MMMM d, yyyy (EEEE) 'at' hh:mm a").withZone(ZoneId.systemDefault())
fun decodeCSVLinesToHistoryItems(csvLines: String): List<HistoryItem> {
    val lines = csvLines.split( ...delimiters: "\n")
    Log.d( tag: "YEL",  msg: "Found ${lines.size} lines")
    Log.d( tag: "YEL",  msg: "Lines: $lines")
    // filter lines that are empty
    val filteredLines = lines.filter { it.isNotEmpty() }
    val items = filteredLines.map { csvLine ->
        val decodedLine = URLDecoder.decode(csvLine,  enc: "UTF-8")
        val parts = decodedLine.split( ...delimiters: ";")
        Log.d( tag: "YEL",  msg: "Parts: $parts")
        Log.d( tag: "YEL",  msg: "Parts size: ${parts.size}")
        // return empty HistoryItem if parts is empty
        if (parts.size < 10) {
            return@map HistoryItem()
        } else {
            HistoryItem(
                tagId = parts[2],
                date = parts[0],
                dateOut = parts[1],
                name = parts[3],
                professor = parts[4],
                purpose = parts[5],
                subject = parts[6],
                inventoryCode = parts[7],
                equipmentName = parts[8],
                remarks = parts[9]
            ) ^map
        }
    }

    // return reversed list
    return items.reversed()
}
```

MainActivity.kt

```kotlin
package com.example.borrowapp

import ...

class MainActivity : ComponentActivity() {
    private val mainViewModel: MainViewModel by viewModels()
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)



        setContent {
            BorrowAppTheme { // A surface container using the 'background' color from the theme
                Surface(modifier = Modifier.fillMaxSize(),
                        color = MaterialTheme.colorScheme.background) {
                    MainScreen(appViewModel = mainViewModel)

                }
            }
        }
    }

    override fun onResume() {
        super.onResume()
        Log.d( tag: "YEL",  msg: "onResume")
        if (mainViewModel.appState.value.connected) {
            mainViewModel.fetchHistory()
        }
    }
}

```

build.gradle.kts

```
plugins {
    alias(libs.plugins.androidApplication)
    alias(libs.plugins.jetbrainsKotlinAndroid)
}

android {
    namespace = "com.example.borrowapp"
    compileSdk = 34

    defaultConfig {
        applicationId = "com.example.borrowapp"
        minSdk = 29
        targetSdk = 34
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
        vectorDrawables {
            useSupportLibrary = true
        }
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_1_8
        targetCompatibility = JavaVersion.VERSION_1_8
    }
    kotlinOptions {
        jvmTarget = "1.8"
    }
    buildFeatures {
        compose = true
    }
    composeOptions {
        kotlinCompilerExtensionVersion = "1.5.1"
    }
    packaging {
        resources {
            excludes += "/META-INF/{AL2.0,LGPL2.1}"
        }
    }
}

```

```
dependencies { this: DependencyHandlerScope

    // Bundling the model with the app
    implementation ("com.google.mlkit:barcode-scanning:17.2.0")
    implementation(libs.androidx.appcompat)
    implementation(libs.material)
    implementation(libs.androidx.activity)
    implementation(libs.androidx.constraintlayout)

    val nav_version = "2.7.4"

    implementation("androidx.navigation:navigation-compose:${nav_version}")

    // Compose ViewModel
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.7.0")

    // Network Calls
    implementation("com.squareup.retrofit2:retrofit:2.9.0")

    // JSON to Kotlin Object Mapping
    implementation("com.squareup.retrofit2:converter-gson:2.9.0")

    // Image Loading
    implementation("io.coil-kt:coil-compose:2.4.0")

    // CameraX
    val cameraxVersion = "1.3.0-rc01"

    implementation ("androidx.activity:activity-compose:1.3.0-alpha08")

    implementation("androidx.camera:camera-core:$cameraxVersion")
    implementation("androidx.camera:camera-camera2:$cameraxVersion")
    implementation("androidx.camera:camera-lifecycle:$cameraxVersion")
    implementation("androidx.camera:camera-video:$cameraxVersion")

    implementation("androidx.camera:camera-view:$cameraxVersion")
    implementation("androidx.camera:camera-extensions:$cameraxVersion")

//    // CSV Parsing
//    implementation("com.github.doyaaaaaken:kotlin-csv-jvm:1.9.3") // for JVM platform
//    implementation("com.github.doyaaaaaken:kotlin-csv-js:1.9.3") // for Kotlin JS platform

    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.activity.compose)
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.ui)
    implementation(libs.androidx.ui.graphics)
    implementation(libs.androidx.ui.tooling.preview)
    implementation(libs.androidx.material3)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(platform(libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.ui.test.junit4)
    debugImplementation(libs.androidx.ui.tooling)
    debugImplementation(libs.androidx.ui.test.manifest)
}
```

Returning Function:

```
Text(text = "Purpose: ${currentlySelectedItem.value?.purpose}", fontSize = 16.sp)

Spacer(modifier = Modifier.height(16.dp))

val formattedIn = formatter.format(Instant.parse(currentlySelectedItem.value?.date))
var formattedOut = ""
if (currentlySelectedItem.value?.dateOut != "") {
    formattedOut =
        formatter.format(Instant.parse(currentlySelectedItem.value?.dateOut))
}
Text(text = "Date borrowed: ${formattedIn}", fontSize = 16.sp)
when {
    currentlySelectedItem.value?.dateOut != "" -> {
        Text(text = "Date returned: ${formattedOut}", fontSize = 16.sp)
    }
```

```
            else -> {
                LazyVerticalGrid(columns = GridCells.Fixed( count: 1)) { this: LazyGridScope
                    items<HistoryItem>(items = appState.history) { this: LazyGridItemScope item ->
                        HistoryCard(item = item, onClick = {
                            currentlySelectedItem.value = item
                            showBottomSheet = true
                        })
                    }
                }
            }
        }
    }

}
```

```
Row(horizontalArrangement = Arrangement.SpaceEvenly) { this: RowScope
    Column(

        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) { this: ColumnScope
        Icon(
            imageVector = Icons.Default.DateRange,
            contentDescription = "Log",
            modifier = Modifier
                .size(114.dp, 114.dp)
                .padding(16.dp)
                .fillMaxSize()
        )
    }
```

```

    val groupedItems = items.groupBy { it.logID }
    val groupedItemsList = groupedItems.values.map { group ->
        val inItem = group.find { it.direction == "in" }
        Log.d( tag: "YEL-GROUP",  msg: "inItem: $inItem")
        val outItem = group.find { it.direction == "out" }
        if (inItem != null) {
            inItem.date = inItem.date
            inItem.dateOut = outItem?.dateOut ?: ""
            inItem ^map
        } else {
            inItem ?: HistoryItem() ^map
        }
    }
```

APPENDIX E

SOURCE CODES FOR HARDWARE

```
#include <WiFi.h>     //libraries needed for ESP 32, SD card, and RFID
#include <SPI.h>      //Software SPI for RFID
#include <MFRC522.h>
#include <SD.h>

#define RST_PIN 21    // RFID pins
#define SS_PIN 5

#define SD_MOSI 13    // SD Card pins
#define SD_MISO 34
#define SD_SCK 14
#define SD_CS 15

SPIClass spi(HSPI);         // Hardware SPI for SD card

WiFiServer server(80);     // Create Wifi Server and Client to handle connection at port 80
WiFiClient client;

const char *NETWORKID = "ESP32";                // Wifi SSID
const char *PASSWORD = "ConnectKaSakin";        // Wifi password

const String READ_AUTHORIZE = "GET /authorize"; // Strings for HTTP requests
const String READ_HISTORY = "GET /history";
const String READ_DONE = "GET /done";
const String READ_ADD = "GET /add";

int GRANTED = -1;                               // status of various operations, -1 as initial value
int DENIED = -1;
int HISTORY = -1;
int ADD = -1;

unsigned long lastActivityTime = 0;
unsigned long timeoutDuration = 5 * 60 * 1000;         // 5 minutes of inactivity in the app

String request = "";                             // String to hold incoming data from the client
String csvHistory = "";
String profTag = "";

byte readCard[4];
String MasterTag1 = "71456227";                  // ID of registered RFID tags
String MasterTag2 = "FAFE6080";
String MasterTag3 = "E28C219";
String MasterTag4 = "7933F27A";
String MasterTag5 = "6983327A";
String MasterTag6 = "793BFD7A";
String MasterTag7 = "69E2BC7A";
String MasterTag8 = "69D7987A";
String MasterTag9 = "7914417A";
String MasterTag10 = "69F61B7A";

// create an array containing all tag values
String MasterTags[] = {MasterTag1, MasterTag2, MasterTag3, MasterTag4,
MasterTag5, MasterTag6, MasterTag7, MasterTag8, MasterTag9, MasterTag10};

String tagID = "";                               // String to hold UID of tags

MFRC522 mfrc522(SS_PIN, RST_PIN);                // Create instances for RFID
```

```
// RFID Functions
boolean getID() {                             // Read new tag if available


  if (!mfrc522.PICC_IsNewCardPresent()) {     // If a new PICC placed to RFID reader continue
    return false;
  }

  if (!mfrc522.PICC_ReadCardSerial()) {       // Since a PICC placed get Serial and continue
    return false;
  }

  tagID = "";                                 // reset String tagID to empty before reading new RFID tag

  for (uint8_t i = 0; i < 4; i++) {           // The MIFARE PICCs that we use have 4 byte UID
    readCard[i] = mfrc522.uid.uidByte[i];
    tagID.concat(String(mfrc522.uid.uidByte[i], HEX));      // Adds the 4 bytes in a single String variable
  }

  tagID.toUpperCase();
  mfrc522.PICC_HaltA();             // Stop reading
  return true;
}

String grantedResponse() {          // response if ID tag is accepted
  String response;
  response.reserve(1024);
  String body = "{\"id\":\"";
  body += tagID;
  body += "\",";
  body += "\"profName\":\"";
  body += profTag;
  body += "\"}";
  response = "HTTP/1.1 202 Accepted\r\nContent-Type: application/json\r\nContent-Length: ";
  response += body.length();
  response += "\r\nConnection: close\r\n\r\n";
  response += body;
  return response;
}

String deniedResponse() {           // response if ID tag is denied
  String response;
  response.reserve(1024);
  String body = "{}";
  response = "HTTP/1.1 403 Forbidden\r\nContent-Type: application/json\r\nContent-Length: ";
  response += body.length();
  response += "\r\nConnection: close\r\n\r\n";
  response += body;
  return response;
}

String RESPONSE = deniedResponse(); // default response is denied/forbidden

String historyResponse()  {         // response if History is requested
  String response;
  response.reserve(1024);
  String body = "{\"history\":\"";
  body += csvHistory;
  body += "\"}";
  response = "HTTP/1.1 200 History\r\nContent-Type: application/json\r\nContent-Length: ";
  response += body.length();
  response += "\r\nConnection: close\r\n\r\n";
  response += body;
  return response;
}
```

```
String addResponse()  {           // response to when a new equipment is added to the borrowed list
  String response;
  response.reserve(1024);
  String body = "{}";
  response = "HTTP/1.1 200 Add\r\nContent-Type: application/json\r\nContent-Length: ";
  response += body.length();
  response += "\r\nConnection: close\r\n\r\n";
  response += body;
  return response;
}

String doneResponse() {           // response when all transactions in the app is done, return to connect
  String response;
  response.reserve(1024);
  String body = "{}";
  response = "HTTP/1.1 200 Done\r\nContent-Type: application/json\r\nContent-Length: ";
  response += body.length();
  response += "\r\nConnection: close\r\n\r\n";
  response += body;
  return response;
}

// SD Card Functions
void createCSV(fs::FS &fs, const char *path)  {           // create a csv file inside the SD card
  Serial.printf("File Created: %s\n", path);

  File file = fs.open(path, FILE_WRITE);
  if (!file)
  {
    Serial.println("Failed to create file");
    return;
  }
  file.close();
}
void appendHistory(fs::FS &fs, const char *path, const char *message) {     // append all requests in the csv file created
  Serial.printf("Logging history to: %s\n", path);

  File file = fs.open(path, FILE_APPEND);
  if (!file)
  {
    Serial.println("Failed to open file");
    return;
  }
  if (!file.print(message))
  {
    Serial.println("Failed to log history");
  }

  file.close();
}

void readHistory(fs::FS &fs, const char *path)  {       // read history written in the SD card
  Serial.printf("Reading file: %s\n", path);

  csvHistory = "";

  File file = fs.open(path);
  if (!file)
  {
    Serial.println("Failed to open file for reading");
    return;
  }

  Serial.print("Read from file: ");
  while (file.available())
  {
    char c = (char)file.read();                     // Read a character from the file
    csvHistory += c;
  }
  file.close();
}
```

```
String parseAdd(String request) {                    // String parsing of the request

  int index = 0;
  String parts[14];
  while (request.indexOf('/') != -1) {               // Split the URL by '/'

    int slashIndex = request.indexOf('/');
    parts[index++] = request.substring(0, slashIndex);
    request = request.substring(slashIndex + 1);
  }

  String datetimein = parts[2];                      // Extract values from the URL request
  String timeused = parts[3];
  String tagid = tagID;
  String student_name = parts[5];
  String prof_name = profTag;
  String subject = parts[7];
  String purpose = parts[8];
  String inventory_code = parts[9];
  String equipment_name = parts[10];
  String logID = parts[11];
  String logDirection = parts[12];
  String remarks = parts[13];

  // Format values into CSV string
  String csvtext = datetimein + ";" + timeused + ";" + tagid + ";" + student_name + ";" + prof_name + ";" + subject + ";"
  + purpose + ";" + inventory_code + ";" + equipment_name + ";" + logID + ";" + logDirection + ";" + remarks;

  return csvtext;
}

void printHistory() {                                // append or print the parsed request string into the CSV file
  String callHistory = request;
  String csvHistory = parseAdd(callHistory);
  String nextrowHistory = csvHistory + "\n";
  Serial.println(csvHistory);
  appendHistory(SD, "/history.csv", nextrowHistory.c_str());
}
void setup()  {
  Serial.begin(115200);
  SPI.begin();                                       // SPI bus for RFID
  spi.begin(SD_SCK, SD_MISO, SD_MOSI, SD_CS);        // SPI bus for SD card

  mfrc522.PCD_Init(); // MFRC522

  WiFi.softAP(NETWORKID, PASSWORD);                  // Create ESP32 Access Point

  server.begin();                                    // Start ESP32 server

  if (!SD.begin(SD_CS, spi))  {                      // If SD card mounted
    Serial.println("Card Mount Failed");
    return;
  }

  Serial.println("SD Card initialized");
  Serial.println("Waiting for a client to connect...");

  if (!SD.exists("/history.csv")) {
    createCSV(SD, "/history.csv");
  }
}
```

```
void loop() {
  WiFiClient client = server.available();                  // Listen for incoming clients

  if (millis() - lastActivityTime > timeoutDuration) {      // Check for timeout

    RESPONSE = deniedResponse();
    GRANTED = 0;
    client.print(RESPONSE);
    client.flush();
    lastActivityTime = millis();                           // Reset last activity time
    Serial.println("Timeout occurred. Resetting authorization status.");
  }

  if (!GRANTED) {                       // Default response
    RESPONSE = deniedResponse();
  }

  while (getID()) {                     // reads RFID tags that are tapped in the sensor

    bool isTagValid = false;           // flag to be true if tagID is in the array of tags

    for (int i = 0; i < 10; i++) {     // check the 10 registered tags

      if (MasterTags[i] == tagID) {    // check if tagID is in array MasterTags
        isTagValid = true;
        break;
      }
    }
      if (tagID == MasterTag1) {          //1
        profTag = "Sir JP";
      }

      else if (tagID == MasterTag2) {    //2
        profTag = "Sir Enmar";
      }

      else if (tagID == MasterTag3) {     //3
        profTag = "Sir Nilo";
      }

      else if (tagID == MasterTag4) {    //4
        profTag = "Ma'am Anne";
      }

      else if (tagID == MasterTag5) {    //5
        profTag = "Sir Emman";
      }

      else if (tagID == MasterTag6) {    //6
        profTag = "Sir Lester";
      }

      else if (tagID == MasterTag7) {    //7
        profTag = "Sir Elias";
      }

      else if (tagID == MasterTag8) {    //8
        profTag = "Sir Christian";
      }

      else if (tagID == MasterTag9) {    //9
        profTag = "Ma'am Dorothy";
      }

      else if (tagID == MasterTag10) {    //10
        profTag = "Sir Anthony";
      }
```

```
    if (isTagValid) {                 // if tag is registered
      Serial.println(" Access Granted!");
      Serial.println(" Handler : " + profTag);
      Serial.println(" ID : " + tagID);
      RESPONSE = grantedResponse();
      GRANTED = 1;
    }

    else  {                           // if tag is not registered
      Serial.println(" Access Denied!");
      Serial.println(" ID : " + tagID);
      RESPONSE = deniedResponse();
      DENIED = 1;
      GRANTED = 0;
    }
    break;
  }
if (client) {                     // If a client (web browser) connects
  Serial.println("A Client is connected to ESP32");

  while (client.connected())  {  // loop while the client's connected

    if (client.available()) {   // read line by line what the client (web browser) is requesting

      request = client.readStringUntil('\r');          // reads the request of client and store it in String request
      Serial.println();
      Serial.println("RECEIVED REQUEST: " + request);
      lastActivityTime = millis();                     // Update last activity time

      if (request.length() > 0) {                      // ensures valid requests from the client

        if (GRANTED) {
          Serial.println("HANDLING THE REQUEST NOW...");

          if (request.startsWith(READ_AUTHORIZE)) {      // handles GET /authorize request
            Serial.println("REQUEST TO AUTHORIZE");
            //   Serial.println("Sending response: " + RESPONSE);
            client.print(RESPONSE);
            client.flush();
          }
          else if (request.startsWith(READ_HISTORY)) {   // handles GET /history request
            Serial.println("CALLING HISTORY");
            //   Serial.println("Sending response: " + RESPONSE);
            readHistory(SD, "/history.csv");
            RESPONSE = historyResponse();
            HISTORY = 1;
            client.print(RESPONSE);
            client.flush();
          }
          else if (request.startsWith(READ_ADD))  {     // handles GET /add request
            Serial.println("ADD");
            //     Serial.println("Sending response: " + RESPONSE);
            printHistory();
            RESPONSE = addResponse();
            ADD = 1;
            client.print(RESPONSE);
            client.flush();
          }
```

```
            else if (request.startsWith(READ_DONE)) {     // handles GET /done request
              Serial.println("DONE");
              //     Serial.println("Sending response: " + RESPONSE);
              RESPONSE = doneResponse();
              GRANTED = 0;
              tagID = "";
              client.print(RESPONSE);
              client.flush();
            }

            else  {
              client.print(RESPONSE);
              client.flush();
            }

          } // end of GRANTED

          else  {
            RESPONSE = deniedResponse();
            client.print(RESPONSE);
            client.flush();
          }
        }
      }
      break;
    }
  }
}
```

APPENDIX F

QR CODES

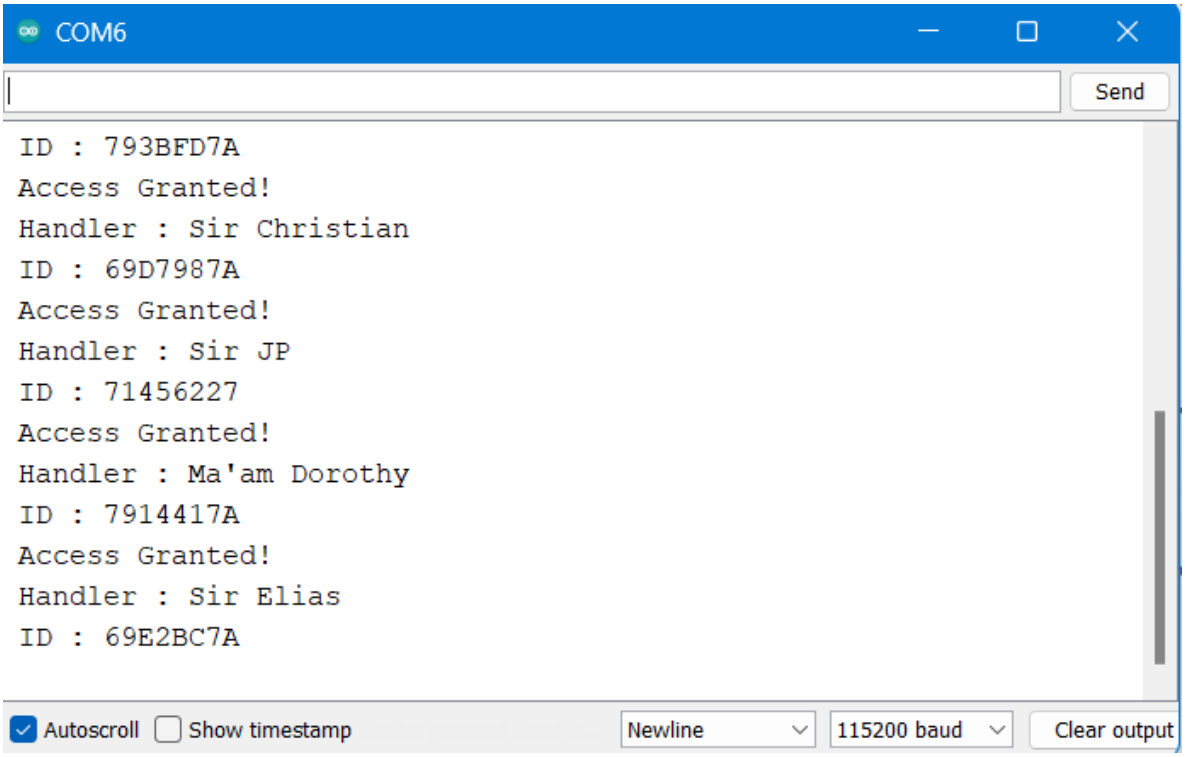
DT2013-08	ATF20B-06	ATF20B-01	3ARB2017-02	MT2013-04
LLT2013-03	DT2013-06	DT2013-05	DT2013-04	DT2013-02
DT2013-01	ATF20B-05	AT2013-08	AT2013-05	AT2013-04
AT2013-03	AT2013-02	AT2013-01	ADS1102CML-04	ADS1102CML-06
MT2018-06	ADS1102CML-02	MT2018-05	MT2018-04	MT2018-01
LM2330-04	LM2230-01	LLT2013-05	LLT2013-04	DT2013-03

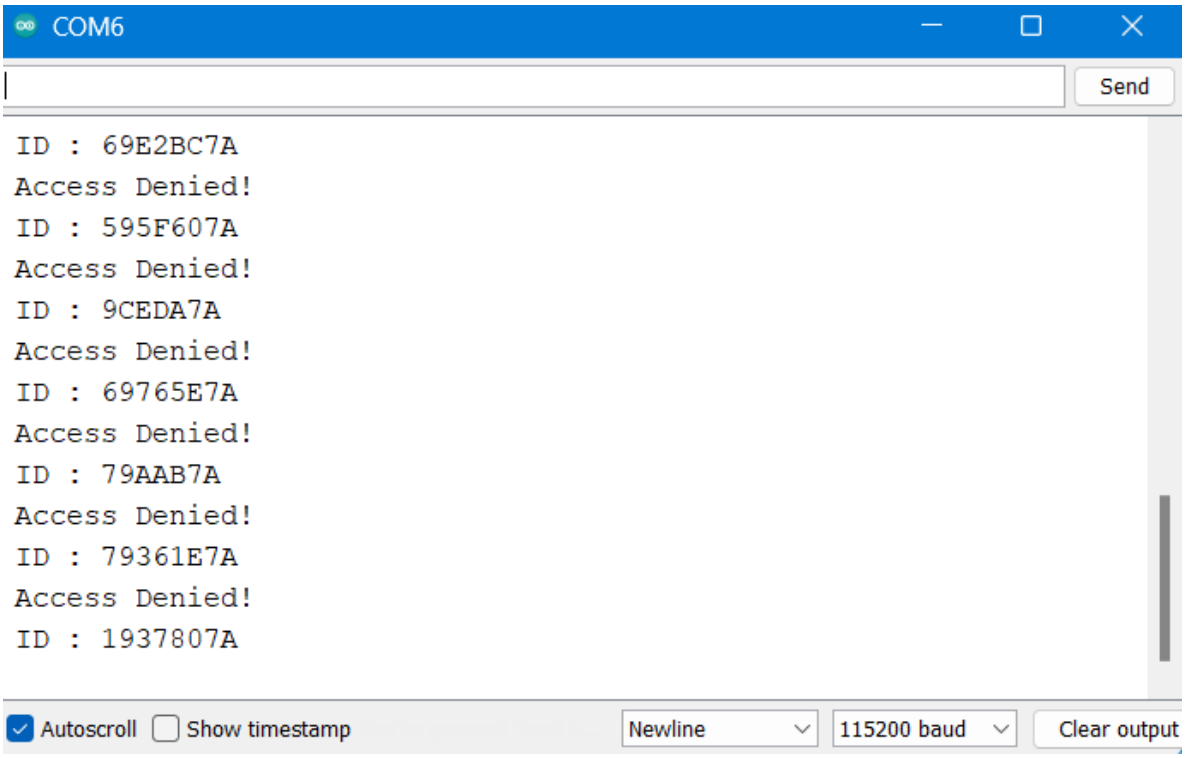
				
ADS1102CML-05	ATF20B-04	ATF20B-03	AT2013-07	AT2013-06
				
MT2018-07	MT2018-03	ADS1102CML-03	MT2018-02	MT2013-03
				
MT2013-02	MT2013-01	LM2330-03	LM 2321-01	LLT2013-06
				
LLT2013-02	LLT2013-01	ADS1102CML-01	DT-AT 2017-03	DT-AT 2017-02
				
3ARB2017-01				

APPENDIX G

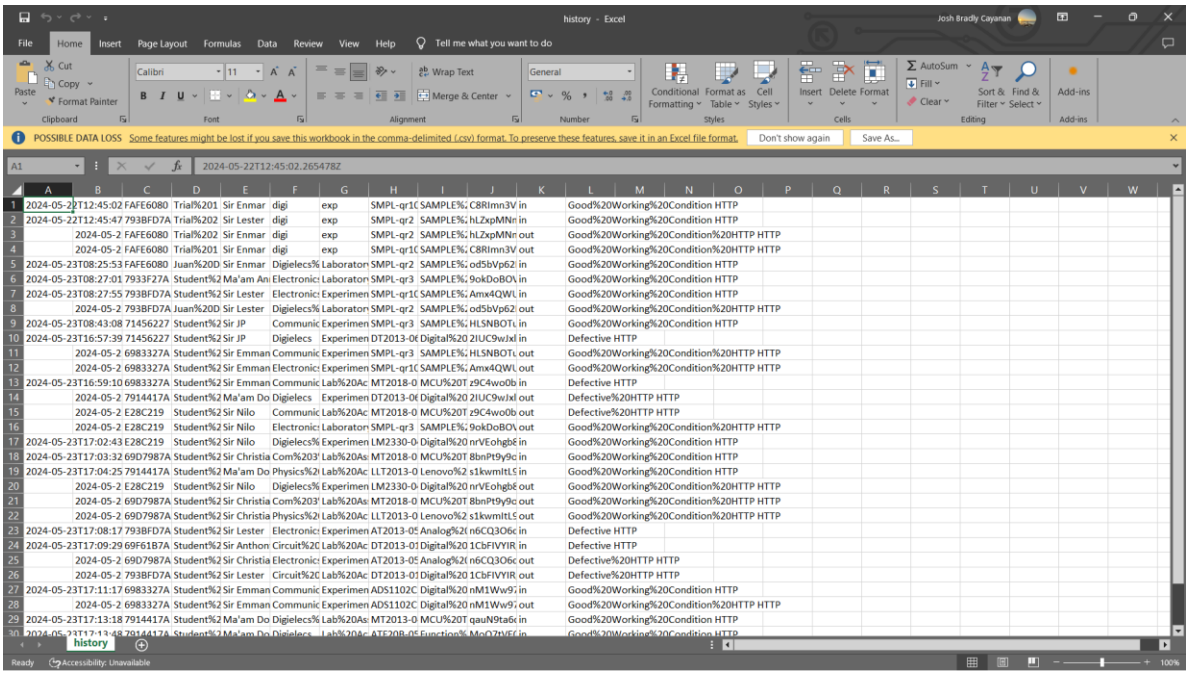
DOCUMENTATIONS

RFID

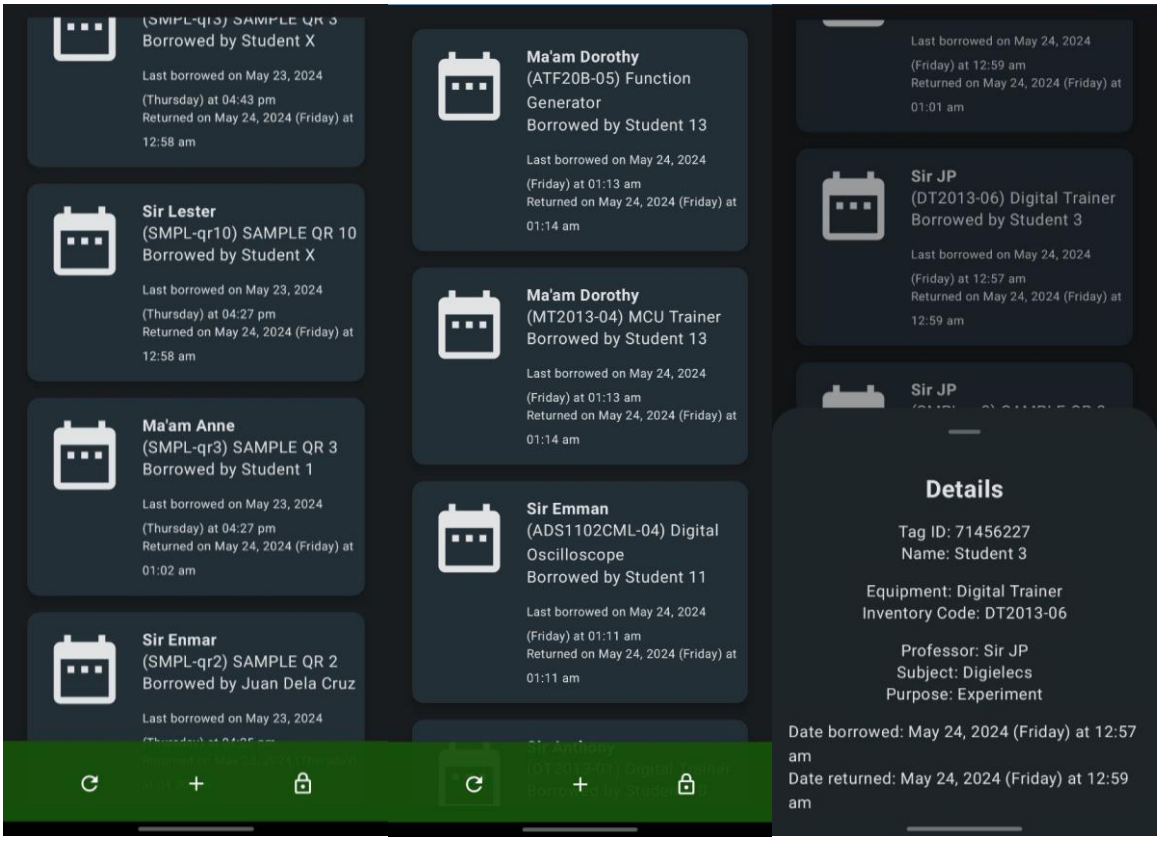




CSV



Android Application



Sample QR Code

	Inventory Code: SMPL-cd1 Equipment Name: SAMPLE code1 Remarks: Good Working Condition
	Inventory Code: SMPL-cd2 Equipment Name: SAMPLE code2 Remarks: Good Working Condition
	Inventory Code: SMPL-cd3 Equipment Name: SAMPLE code3 Remarks: Good Working Condition
	Inventory Code: SMPL-cd4 Equipment Name: SAMPLE code4 Remarks: Good Working Condition
	Inventory Code: SMPL-cd5 Equipment Name: SAMPLE code5 Remarks: Good Working Condition